

IL LIVELLO RETE

Il livello di rete si fa carico di **trasferire pacchetti provenienti dal livello di trasporto** a partire dalla sorgente fino alla destinazione finale, attraversando tanti sistemi intermedi (solitamente router), nella sottorete di comunicazione.

I principali compiti di questo livello sono:

- ✚ conoscere la topologia della sottorete di comunicazione;
- ✚ scegliere di volta in volta il cammino migliore (**routing**);
- ✚ gestire il flusso dei dati e le congestioni;
- ✚ gestire le problematiche derivanti dalla presenza di più reti diverse.

Nel progetto e nella **realizzazione del livello di rete** di un'architettura di rete si devono prendere decisioni in merito a:

- ✚ servizi offerti a livello superiore;
- ✚ **organizzazione interna** della sottorete di comunicazione.

In merito ai servizi offerti a livello superiore, ci sono due tipologie fondamentali di servizi:

- ✚ **servizi connection oriented**: che lo intendiamo come servizio suddiviso in tre fasi → *instaurazione della connessione, trasmissione delle informazioni e rilascio della connessione*;
- ✚ **servizi connectionless**: inteso come servizio con un'unica fase.

~~In realtà le decisioni da effettuare riguardano essenzialmente se offrire un servizio affidabile con uno orientato alla connessione. Le scelte più comuni sono di offrire servizi connection oriented affidabili oppure servizi connectionless non affidabili.~~

Nei **servizi orientati alla connessione** per il livello di trasporto dovranno verificarsi le seguenti condizioni:

- ✚ le **peer entities** stabiliscono una connessione, negoziandone i **parametri**, alla quale viene associato un identificatore;
- ✚ tale **identificatore** viene inserito in ogni pacchetto che verrà inviato;
- ✚ la comunicazione è bidirezionale e i pacchetti viaggiano, **in sequenza, lungo il cammino assegnato alla connessione**;
- ✚ il controllo di flusso è fornito automaticamente.

Le condizioni per la creazione di un **servizio non orientato alla connessione** sono:

- ✚ la sottorete è considerata inaffidabile, per tale motivo gli host devono provvedere per conto proprio alla correzione degli errori e al controllo del flusso;
- ✚ i **pacchetti sono inoltrati in maniera indipendente** nella sottorete, in questo contesto i pacchetti si chiamano **datagram**;
- ✚ ogni pacchetto deve contenere un **identificatore che sarà l'indirizzo di destinazione**;
- ✚ ~~la complessità è dovuta ai vari host i cui livelli di trasporto devono fornire la necessaria affidabilità e l'orientamento alla connessione.~~

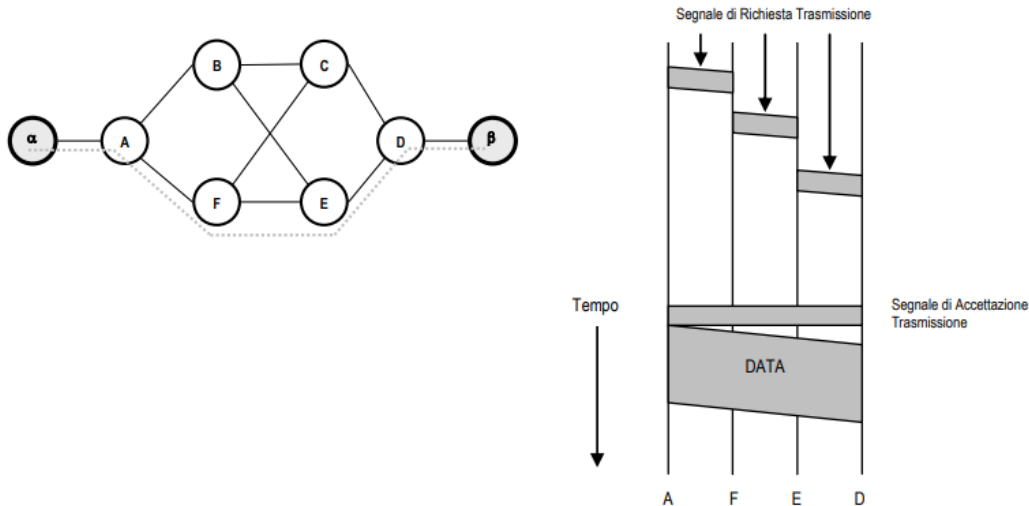
PRINCIPI DI COMMUTAZIONE

La **commutazione** è la funzione che consente di trasferire l'informazione proveniente dagli host di origine agli host di destinazione variabili di volta in volta secondo le esigenze del servizio. Tale funzione è svolta da appositi dispositivi collocati nei nodi della rete.

La scelta della tecnica di commutazione da adottare in una sottorete è ristretta in tre opzioni principali, ed influenza la QoS:

- ✚ commutazioni di circuito;
- ✚ commutazione di messaggio;
- ✚ commutazione di pacchetto.

COMMUTAZIONE DI CIRCUITO



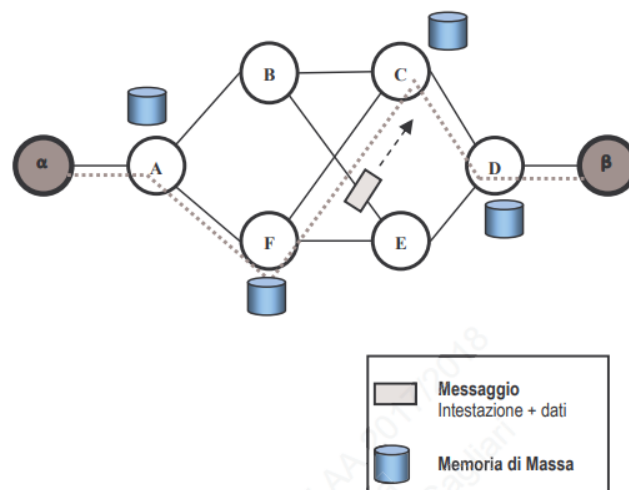
Abbiamo 6 nodi di collegamento di commutazione che possiamo chiamare **router instradatori**. Gli host sono alpha e beta collegati rispettivamente ad A e a B, nella fase di instaurazione l'host chiamante invia alla sottorete le informazioni necessarie per l'individuazione del destinatario.

È previsto un **ritardo di instaurazione**, variabile con le caratteristiche della rete, dovute ai tempi di elaborazione per la scelta dei nodi del percorso origine-destinazione, ed ai tempi di trasmissione delle informazioni di segnalazione. Per questo motivo nello schema a *destra il tempo è un 'ordinata negativa*, ed è spostato perché rappresenta il tempo che ci metto a collegarmi, F non parte subito ma dipende dal momento di commutazione, D ci mette molto più tempo perché deve confermare tutta la filiera. Tutta la linea di trasmissione è emessa con un leggere di ritardo dovuto all'attraversamento dei nodi.

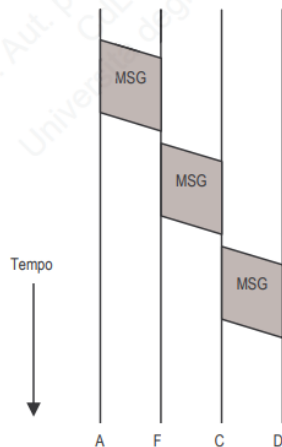
In sostanza viene resa disponibile una connessione fisica dedicata. I principali limiti di questo sistema sono che impiego tutte le risorse per un singolo collegamento, anche quando non si trasferisce.

COMMUTAZIONE DI MESSAGGIO

Per rendere disponibile il canale solo quando lo si deve utilizzare si preferisce *dare più importanza al messaggio* rispetto al circuito per questo si passa alla **commutazione di messaggio**, viene inserita una risorsa di identificazione che è una memoria di massa, nella quale viene registrata l'informazione e se viene ricevuta correttamente viene cancellata dalla memoria, questa tecnica si chiama **store and forward**.



I vantaggi sono che occupo le singole linee solo per il tempo per trasferire quel singolo dato, questo messaggio è un dato ampio e deve viaggiare autonomamente quindi avrà un'intestazione. *Non c'è instaurazione del canale*, fisica o logica, tra host di origine e host di destinazione, quindi è una comunicazione senza connessione.



Il **ritardo di trasferimento** dei messaggi tra origine e destinazione dipenderà da una serie di fattori, quali la lunghezza dei messaggi, la velocità di trasmissione, il tempo di attraversamento dei singoli nodi. A causa dei fattori in gioco appena elencati, il servizio di trasferimento delle informazioni in una rete con questa commutazione *non può consentire comunicazione di informazioni interattive*.

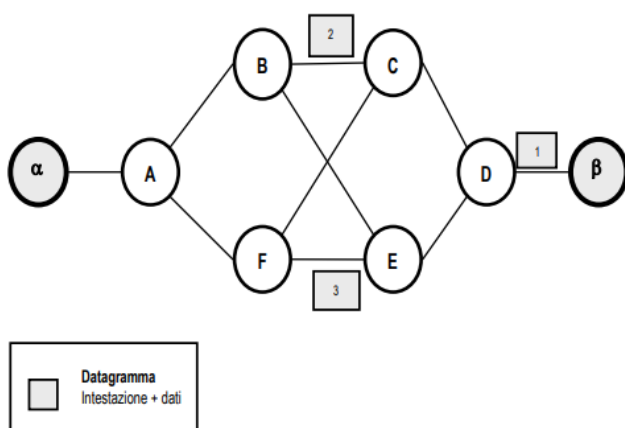
Questo sistema non è più efficiente quando i dati sono veramente molto grandi, quindi è meglio suddividerlo in pacchetti, da qui nasce la commutazione di pacchetto in modo tale che i canali possano essere sfruttati anche per mandare più comunicazioni contemporaneamente.

COMMUTAZIONE DI PACCHETTO

Come detto prima, in questa commutazione l'informazione viene segmentata in unità di lunghezza massima predefinita, chiamate **pacchetti**. Questo tipo di reti possono funzionare attraverso due modalità differenti:

- ✚ datagramma;
- ✚ circuito virtuale.

La prima è da considerarsi l'evoluzione della commutazione di messaggi. L'informazione viene segmentata in una serie di pacchetti chiamati **datagrammi**.



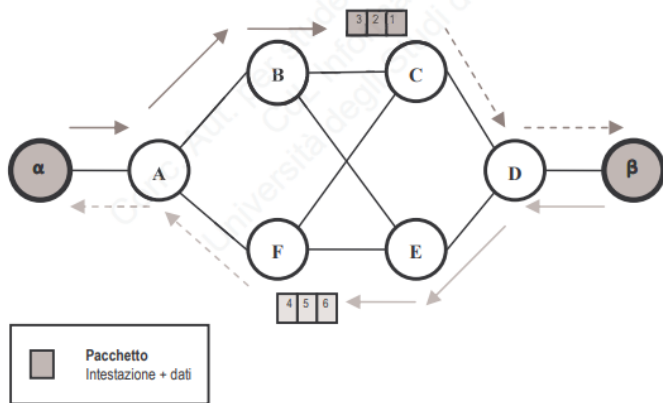
Ciascun datagramma, visto che viaggiano per conto proprio, è provvisto di **intestazione** contenente delle informazioni aggiuntive quali indirizzo di origine e destinazione dell'informazione.

I pacchetti possono essere consegnati con un ordine diverso rispetto a quello con cui sono stati spediti, quindi sarà compito dell'host destinatario riordinare i pacchetti, potrebbe capitare anche che alcuni si perdano, o addirittura che arrivino dei duplicati.

Rispetto alla commutazione di messaggio si riesce a mandare più informazioni contemporaneamente,

ma c'è un alto rischio che i pacchetti si perdano o che vengano duplicati.

Per risolvere si usa la **commutazione a circuito virtuale**, in cui tutti i pacchetti seguiranno lo stesso percorso, grazie a questo aspetto viene garantita la sequenzialità delle informazioni e l'integrità del messaggio a destinazione.



Il problema è che se uso sempre lo stesso circuito man mano che lo si utilizza inizia a degradare e quindi l'attraversamento inizia a rallentare.

Per decidere il migliore percorso si utilizzano dei protocolli chiamati **algoritmi di routing** che possono essere **statici o dinamici**, nei **primi** le decisioni di routing sono prese precedentemente all'avvio della rete, sono adatti per sottoreti di piccole dimensione ed hanno il vantaggio di permettere il controllo totale sui flussi del traffico al gestore. In quelli **dinamici** le decisioni di routing sono frequentemente riformulate sulla base di diversi fattori, sono adatti alle reti di grandi dimensioni in quanto hanno un'alta tolleranza alle condizioni di errore.

In maniera più precisa gli algoritmi dinamici (o adattivi) differiscono fra loro per:

- ✚ come ricevono le informazioni;
- ✚ quanto spesso rivedono le decisioni;
- ✚ quale metrica di valutazione adottano.

L'algoritmo di routing cambia a seconda del tipo di sottorete:

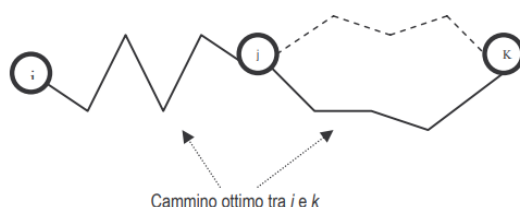
- ✚ in una **sottorete datagram** viene applicato un nuovo percorso ad ogni nuovo pacchetto o serie di pacchetti;
- ✚ in una **sottorete basata su circuiti virtuali** l'algoritmo viene applicato solo nella fase di creazione del circuito, in questo contesto si usa spesso il termine *sessione routing*.

Da un algoritmo di routing ci si attende:

- ✚ **correttezza**: inoltrare il pacchetto nella giusta destinazione;
- ✚ **semplicità**: avere un'implementazione non troppo complicata;
- ✚ **robustezza**: bisogna avere la garanzia di funzionamento anche in caso di cadute di linee o malfunzionamento o blocco del router;
- ✚ **stabilità**: convergere e possibilmente in modo rapido;
- ✚ **equità**: non ci deve essere nessun privilegio nell'inoltro;
- ✚ **ottimalità**: bisogna scegliere la soluzione globalmente migliore.

La **metrica** è lo strumento per *misurare come stiamo utilizzando la rete*, può essere il numero di pacchetti che invio, la capacità di invio bidirezionale, il ritardo della rete, il numero di nodi che salto, ecc.. Solitamente viene scelto il percorso con una **metrica inferiore**.

Alcuni protocolli sono in grado di utilizzare *differenti metriche per il calcolo del percorso migliore*. È possibile fare una considerazione generale sul **percorso ottimale dei cammini**, indipendentemente dallo specifico algoritmo adottato per selezionarli. Il principio di ottimalità dice che se il **router j** è nel cammino ottimo fra **i** e **k**, allora anche il cammino ottimo fra **j** e **k** è sulla stessa strada:



Se così non fosse, ci sarebbe un altro cammino fra j e k migliore di quello che è parte del cammino ottimo fra i e k, ma allora ci sarebbe anche un cammino fra i e k migliore di quello ottimo. Una diretta conseguenza è che l'insieme dei cammini ottimi da tutti i router a uno specifico router di destinazione costituiscono un albero detto **sink tree**.

La capacità di distribuire il traffico su differenti percorsi, verso la stessa destinazione viene denominata **bilanciamento del carico** (load balancing). Questo permette una maggiore efficienza sull'utilizzo delle risorse di rete.

Le due principali modalità di load balancing vengono identificate come:

- ✚ **equal-cost**: che distribuisce il traffico su più percorsi con stessa metrica;
- ✚ **unequal-cost**: che distribuisce il traffico su più percorsi aventi differenti metriche. Il traffico viene distribuito in maniera inversamente proporzionale al costo dei percorsi.

Viene definito con il termine **convergenza** il processo nel quale le tabelle di routing vengono portate ad uno stato di consistenza. Il tempo di consistenza è il tempo che occorre per l'aggiornamento delle tabelle di routing a seguito di una modifica della topologia di rete. (si parla di convergenza solo per gli algoritmi dinamici)

Come detto prima esistono **algoritmi di routing statici e dinamici**, differiscono perché statico viene progettato una volta, testato e rimane in piedi regolarmente con quella categoria di protocollo, viene utilizzato quando abbiamo una sottorete di comunicazione dove sono definiti i carichi, i nodi e i tempi con i quali pacchetti attraversano la sottorete. Quando si ha la necessità di tener conto delle condizioni del momento, del traffico del momento e del funzionamento del momento si utilizzano invece gli algoritmi dinamici.

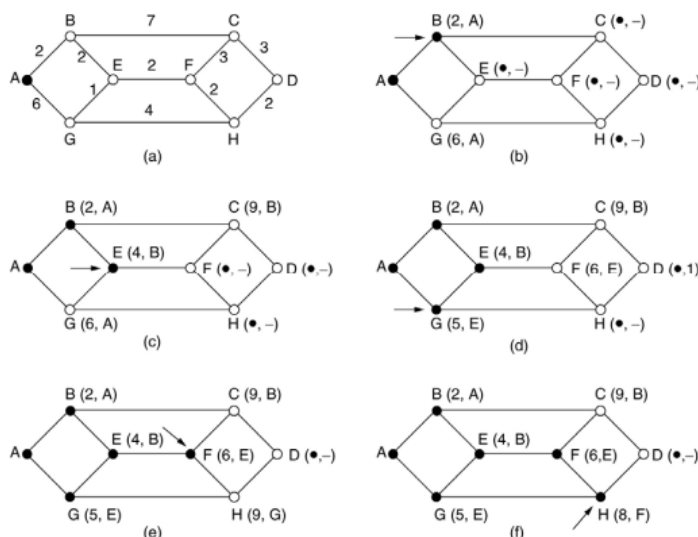
Esistono **tre principali algoritmi statici**:

SHORTEST PATH ROUTING

Lo **shortest path routing** ha 8 nodi con una metrica di peso o di velocità che è assegnata su ognuna di quelle linee, ad ogni linea quindi gli attribuisco un'etichetta che riporta la sua distanza dal nodo d'origine lungo il miglior percorso conosciuto; la metrica scelta normalmente è quella di velocità.

Inizialmente nessun percorso è noto, perciò tutti i nodi sono etichettati con il simbolo che rappresenta *l'infinito*. Mano a mano che l'algoritmo procede e si trovano i percorsi, le etichette cambiano rispecchiando i percorsi migliori. Un'etichetta può essere *provvisoria o permanente*. Inizialmente tutte le etichette sono provvisorie, e quando si scopre che rappresenta il percorso più breve possibile dall'origine a quel nodo viene resa permanente smettendo di modificarla.

Quando si attribuisce un peso, e lo si dà molto alto si fa in modo che pochi pacchetti passino di lì, viceversa se gli attribuisco un peso minore.



La scelta del **percorso più breve** si inizia dal nodo di partenza che in questo caso è il nodo **A** e si va a verificare in quanto tempo raggiunge un altro nodo, ad esempio per raggiungere B ci vorrà 2, per raggiungere G invece ci metto 6; questi valori sono chiamate **interdistanze**.

Questa mappa di interdistanze va caricata per ogni nodo, quindi faccio un test di partenza e creo una tabella che rimane sempre quella, quindi statica. Che verrà immessa in rete e tutti la possono vedere.

FLOODING

La tecnica del **flooding** consiste nell'inviare ogni pacchetto su tutte le linee del router eccetto quella da cui è arrivato. Presenta l'inconveniente di generare un numero enorme di pacchetti, per questo motivo può essere utilizzata solo in reti molto piccole. Per limitare il traffico generato si può ricorrere a diverse tecniche:

- ✚ inserire in ogni pacchetto un **contatore** che viene decrementato ad ogni hop. Quando il contatore arriva a zero, il pacchetto viene scartato;
- ✚ inserire la coppia source *router ID-sequence number* in ogni pacchetto. Ogni router esamina tali informazioni e ne tiene traccia, e quando le vede per la seconda volta scarta il pacchetto;
- ✚ **selective flooding**: i pacchetti vengono duplicati solo sulle linee che vanno all'incirca nella giusta direzione (per questo si devono mantenere apposite tabelle a bordo).

FLOW BASED ROUTING

Quest'algoritmo è basato sull'idea di *calcolare in anticipo il traffico atteso su ogni linea*, da questi calcoli si deriva una stima del ritardo medio atteso per ciascuna linea ed infine si basano su tali informazioni le decisioni di routing.

Le informazioni necessarie per poter applicare l'algoritmo sono la *topologia di rete*, la *matrice delle quantità di traffico $T(i,j)$ stimate fra ogni coppia (i,j) di router*, e le *capacità delle linee punto a punto*.

Vengono fatte le seguenti assunzioni:

- ✚ il traffico è stabile nel tempo e noto in anticipo;
- ✚ il ritardo su ciascuna linea aumenta all'aumentare del traffico sulla linea e diminuisce all'aumentare della velocità della linea secondo le leggi della Teoria delle Code.

Dai ritardi calcolati per le singole linee si può calcolare il **ritardo medio dell'intera rete**, espresso come somma pesata dei ritardi delle singole linee. Il peso di ogni linea è dato dal traffico su quella linea diviso il traffico totale sulla rete.

Il metodo nel suo complesso, considerata una topologia nota, si basa sui seguenti punti:

- ✚ si considera una matrice di traffico;
- ✚ si determinano i percorsi che verranno seguiti per il collegamento fra ogni coppia di router;
- ✚ si calcola il traffico che incide su ogni linea (la somma di tutti i $T(i,j)$ instradati su quella linea, con i e j nodi della rete);
- ✚ si calcola il ritardo di ogni linea;
- ✚ si calcola il ritardo medio della rete;
- ✚ si determina un algoritmo di routing che minimizza il ritardo medio dell'intera rete.

Gli **algoritmi dinamici** si adattano automaticamente alla rete, ne esistono principalmente due:

DISTANCE VECTOR

Nel **distance vector** compongo un **vettore dei ritardi** (tabella) per ciascun nodo che ho a disposizione, ogni nodo avrà un suo vettore. Il vettore di A rappresenta il ritardo per raggiungere un nodo, il ritardo di A su A sarà 0, su B sarà 12 e così via. Quindi il vettore ha al suo interno la *distanza che lo separa dal router in oggetto e la linea in uscita da usare per arrivarci*.

Per i suoi vicini immediati il router stima direttamente la distanza dei collegamenti corrispondenti, mandando speciali **pacchetti ECHO** e misurando quanto tempo ci mette la risposta a tornare. Questa operazione verrà condotta un numero sufficiente di volte, per poter stimare il tempo di raggiungibilità.

Ad intervalli regolari ogni router *manda la sua tabella a tutti i vicini*. Quando un router riceve le nuove informazioni, calcola una nuova tabella scegliendo fra tutte la concatenazione migliore: *se stesso* → *vicino immediato* → *router remoto di destinazione* per ogni destinazione.

Ovviamente la miglior concatenazione è quella che produce la minor somma di:

- ✚ distanza fra il router stesso ed un suo vicino immediato;
- ✚ distanza fra quel vicino immediato ed il router di destinazione.

	A	I	H	K	
A	0	24	20	21	8 A
B	12	36	31	28	20 A
C	25	18	19	36	28 I
D	40	27	8	24	20 H
E	14	7	30	22	17 I
F	23	20	19	40	30 I
G	18	31	6	31	18 H
H	17	20	0	19	12 H
I	21	0	14	22	10 I
J	9	11	7	10	0 -
K	24	22	22	0	6 K
L	29	33	9	9	15 K

Il ritardo da J ad A è 8
 Il ritardo da J a I è 10
 Il ritardo da J a H è 12
 Il ritardo da J a K è 6

Vettori ricevuti dai quattro vicini di J

Nuovo ritardo stimato da J

Nuova tabella di routing di J

Se aggiungiamo un nodo anch'esso avrà un ritardo originario derivato da **A** per il primo vettore, da **I** per il secondo vettore e così via, ogni qual volta utilizzo un sistema di tipo dinamico devo essere in grado di poter mettere e togliere dei nodi e la rete deve continuare a convergere. *Questo algoritmo non funziona sempre bene* perché è efficiente nel dare le buone notizie ma lentissimo nel dare le brutte notizie.

Esempio: abbiamo una rete con al massimo 2 porte per nodo, dopo un primo tempo **t0** tutto è a 0 perché non è passato ancora nessun pacchetto, al tempo **t1** dopo 1 scambio ho che B si rende conto che per raggiungere A ha bisogno di un salto, e così via fino a completare il distance vector.

A	B	C	D	E	
•	•	•	•	•	Inizialmente
	1	•	•	•	Dopo 1 scambio
	1	2	•	•	Dopo 2 scambi
	1	2	3	•	Dopo 3 scambi
	1	2	3	4	Dopo 4 scambi

A	B	C	D	E	
•	•	•	•	•	Inizialmente
	1	2	3	4	Dopo 1 scambio
	3	2	3	4	Dopo 2 scambi
	3	4	3	4	Dopo 3 scambi
	5	4	5	4	Dopo 4 scambi
	5	6	5	6	Dopo 5 scambi
	7	6	7	6	Dopo 6 scambi
	7	8	7	8	Dopo 7 scambi

Se A dovesse smettere di funzionare B sa che non può più raggiungere A ma nel frattempo C gli comunica che raggiungerà A in due colpi, e dato che B raggiunge C in un colpo ne conclude che raggiungerà A con 3 colpi. Dopodiché anche a C arriverà la comunicazione che A non è raggiungibile e ripete lo stesso

fatto, questo si chiama il **paradosso dello scambio all'infinito**, ci dice che dato che le informazioni sono demandate ad ogni altro nodo non si riuscirà ad avere un aggiornamento completo di tutti i nodi.

LINK STATE ROUTING

Il **link state routing** supera la lentezza di convergenza del distance vector routing. Ogni router tiene sott'occhio lo stato dei collegamenti *fra sé ed i suoi vicini immediati*, misurando il ritardo di ogni linea, e distribuisce tali informazioni a tutti gli altri. Sulla base di tali informazioni, ogni *router ricostruisce localmente la topologia completa dell'intera rete* e calcola il cammino minimo fra sé tutti gli altri.

È uno degli algoritmi più utilizzati, è suddiviso in 5 passaggi:

1. un nodo non sa chi ha vicino quindi deve indagare sui vicini e rilevarne l'indirizzo, per farlo manda dei **pacchetti HELLO** su tutte le linee in uscita, in risposta riceve dai vicini i loro indirizzi;
2. inviando diversi **pacchetti ECHO** e misurando il tempo di arrivo della risposta e mediando su vari pacchetti si deriva il ritardo della linea. La tabella costruita viene chiamata **Neighbour Table**;
3. a tale punto viene costruito una pacchetto chiamato **LSP** (Link State Packet) contenente l'identità del mittente, la lista dei vicini con i relativi ritardi, la metrica associata, il numero di sequenza del pacchetto, il tempo di vita.
4. *questo pacchetto viene poi mandato a tutti gli altri router* con il flooding, tutti questi vanno a convergenza su un'unica tabella e generano la tabella dinamica di link state packet;
5. attraverso l'invio e la ricezione di tali pacchetti da parte di ciascun router, viene costruita la topologia dell'intera rete. La fase finale consiste nel *calcolo del cammino minimo* tra tutti i router.

All'interno del link state packet troviamo quindi qual è il nodo, qual è il numero di sequenza per non confondere il prossimo che verrà generato, e l'età cioè il tempo che impiego per raggiungere un nodo.

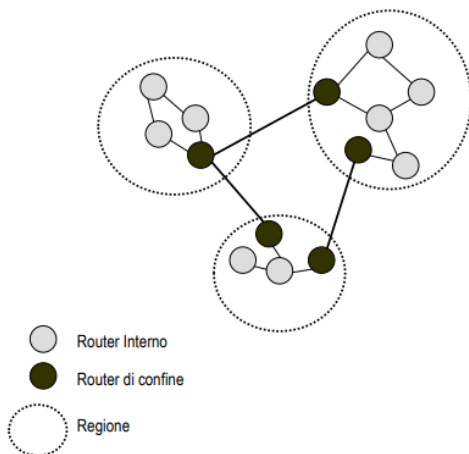
Questi valori vengono agganciati ad una tabella che riporterà gli stessi dati che ha dei flag che dicono non devo ripassare su un nodo in cui sono già passato. La rete è enorme quindi esce una tabella troppo grande. Gli algoritmi dinamici vengono generati per livelli perché se no sarebbero troppo grandi.

Alla fine, si ottengono dei protocolli che definiscono i nuovi percorsi minimi per raggiungere i router (OSPF, IS-IS).

ROUTING GERARCHICO

Quando la dimensione della sottorete cresce fino a contenere un numero elevato di nodi, mantenere la completa topologia in ogni router risulta essere particolarmente oneroso. Il routing viene pertanto impostato in **maniera gerarchica**, la rete viene quindi *divisa in zone* spesso chiamate regioni:

- ✚ all'interno di una regione i router, detti **router interni**, conoscono il percorso per il raggiungimento di tutti gli altri router della stessa regione;
- ✚ quando un router interno ha necessità di inviare informazioni verso un'altra regione, questo inoltra tali informazioni verso un particolare router della propria regione, chiamato **router di confine**;
- ✚ il router di confine inoltra le informazioni verso il router di confine della regione di destinazione.



Di conseguenza si possono considerare due livelli di routing:

- 1) un primo livello di **routing all'interno** di ogni regione;
- 2) un secondo livello di *routing fra tutti i router di confine*.

I **router interni** mantengono nelle loro tabelle di routing:

- ✚ un'entrata per ogni altro router interno, con la relativa linea da usare per arrivarci;
- ✚ un'entrata per ogni altra regione, con l'indicazione del relativo router di confine e della linea da usare per arrivarci.

I **router di confine** invece mantengono un'entrata per ogni altra regione, con l'indicazione del prossimo router di confine da contattare e della linea da usare per arrivarci.

BROADCAST ROUTING

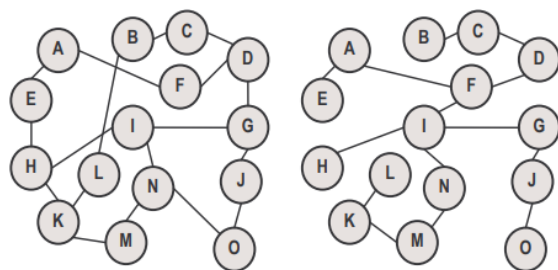
Alcune applicazioni necessitano *dell'invio delle informazioni a molti o a tutti gli host presenti in una sottorete*. La trasmissione contemporanea di un pacchetto a tutte le destinazioni viene chiamata **trasmissione broadcast**, e ci sono diverse metodologie per ottenerla.

La metodologia più semplice consiste nell'invio di un *pacchetto distinto a ciascun destinatario*, possiede il vantaggio di un basso utilizzo di banda ma obbliga il sorgente a possedere una lista completa dei destinatari.

Una seconda metodologia consiste nell'utilizzare il meccanismo di **flooding**, gli svantaggi sono la generazione elevata di pacchetti e largo consumo di banda.

Un terzo meccanismo consiste nell'utilizzo del **multidestination routing**, in cui ogni pacchetto è dotato di una lista di destinazioni, il router che riceve il pacchetto controlla tutte le destinazioni e determina l'insieme delle linee di trasmissione richieste per inoltrarlo. A questo punto il router genera una copia del pacchetto che inoltra in ogni linea creata, includendoci esclusivamente le destinazioni su tale linea. Con questa metodologia, dopo un certo numero di salti, ogni pacchetto conterrà esclusivamente un unico indirizzo di destinazione e sarà quindi trattato come pacchetto normale.

Un quarto algoritmo utilizza il **sink tree** del router di trasmissione, ed utilizza lo spanning tree per inoltrare i pacchetti. Lo **spanning tree** consiste nel sottoinsieme di percorsi comprendenti tutti i router ma senza che si verifichino collegamenti ridondanti e quindi cicli. Ogni router appartenente allo spanning tree può copiare un pacchetto broadcast in arrivo su tutte le linee, ad eccezione di quella di ingresso. Questo metodo ha il



Una Sottorete

Uno Spanning Tree

vantaggio di un eccellente utilizzo della banda disponibile e della minima generazione di pacchetti necessari per il compimento del lavoro.

Lo svantaggio principale è la necessità della conoscenza da parte del router di uno spanning tree, cosa impossibile per alcuni algoritmi. Per ovviare a questo problema si può utilizzare una metodologia chiamata **reverse path forwarding**.

In questa metodologia il router al ricevimento di un pacchetto broadcast verifica se questo è giunto dalla *linea utilizzata per inviare i pacchetti alla sorgente della trasmissione broadcast*. In caso affermativo esiste un'alta probabilità che il pacchetto broadcast abbia seguito il percorso migliore per giungere al router e che *perciò sia la prima copia arrivata*. Pertanto, il router invia il pacchetto verso tutte le linee, ad eccezione di quella entrante.

Viceversa, nel caso in cui il pacchetto broadcast non sia giunto dalla linea utilizzata per inviare pacchetti alla sorgente, *quest'ultimo viene scartato in quanto supposto duplicato*.

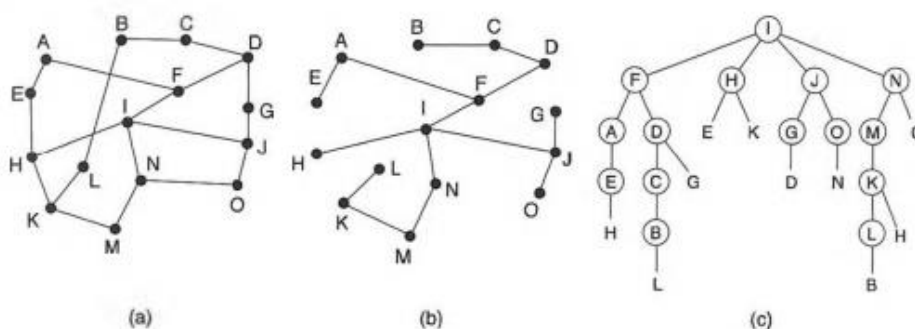
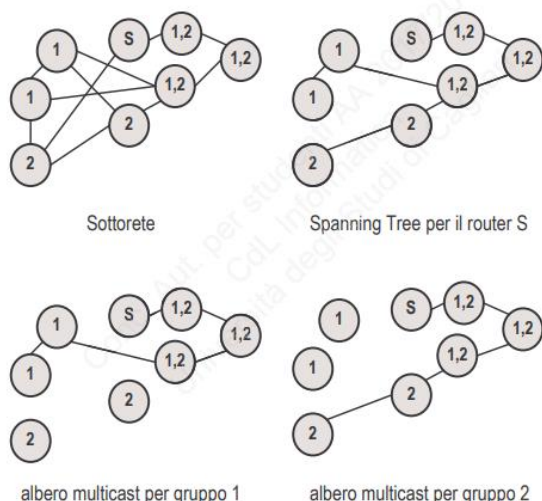


Figura 5.16. Inoltro a percorso inverso. (a) Una sottorete. (b) Un sink tree. (c) L'albero realizzato mediante l'inoltro a percorso inverso.

I vantaggi del reverse path forwarding risiedono principalmente nella sua *efficienza e nella sua facilità di implementazione*, come visto non richiede da parte dei router la conoscenza di uno spanning tree o di alcuna mappa.

MULTICAST ROUTING

La possibilità di inviare pacchetti a sottoinsiemi di destinatari di un insieme viene chiamato **multicast** e l'algoritmo utilizzato per l'instradamento di tali pacchetti viene chiamato **multicast routing**.



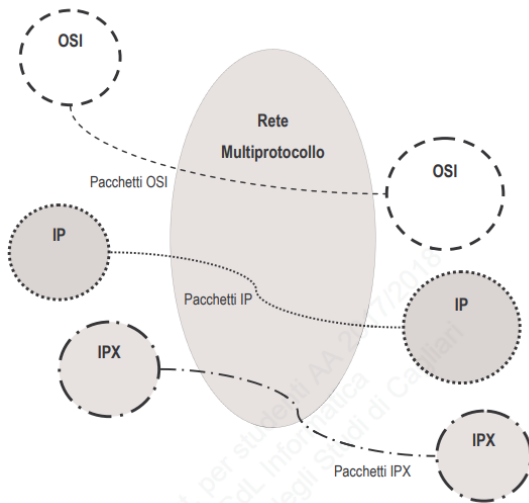
La trasmissione multicast richiede la gestione dei **gruppi**: occorre un sistema che consenta di creare e distruggere i gruppi e che permetta ai processi di entrare e uscire dai gruppi. Il modo in cui queste operazioni sono eseguite non riguarda l'algoritmo di routing; ciò che importa è che quando si unisce a un gruppo, il processo informi il suo host di questo fatto.

È importante che i router sappiano quali sono gli host appartenenti a ogni gruppo, quindi gli host *devono comunicare ai loro router i cambi di gruppo* oppure i *router devono interrogare periodicamente i loro host*. In entrambi i casi, i router scoprono i gruppi di appartenenza dei loro host. I router comunicano con i loro vicini, perciò le informazioni si propagano attraverso la sottorete.

Per ottenere una trasmissione multicast, ogni *router elabora uno spanning tree che copre tutti gli altri router*.

Quando un processo invia un processo multicast ad un gruppo, ogni router rimuove tutte le linee dello spanning tree che non conducono agli host del gruppo, inoltrando i pacchetti esclusivamente attraverso lo spanning tree appropriato.

INTERNETWORKING

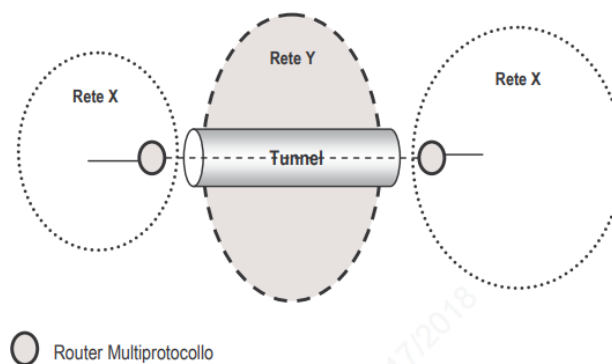


L'**internetworking** è il collegamento attraverso una rete multiprotocollo di reti con protocolli differenti, due reti IP uguali possono essere collegate attraverso un operazione di internetworking su una rete multiprotocollo, l'importante è avere un gateway multiprotocollo che può essere un bridge oppure un operazione di tunneling.

Il **tunneling** viene utilizzato nel caso che host sorgente e host di destinazione utilizzino lo stesso tipo di rete, ma sono separati da una rete con caratteristiche differenti.

In questo caso la rete di tipo Y non è dotata di router multiprotocollo, invece si ha un router designato in ciascuna delle due reti di tipo X che è multiprotocollo, e incapsula i pacchetti delle reti di tipo X dentro pacchetti di tipo Y,

consegnandoli poi alla rete di tipo X.



STRUTTURA DI UN GENERICO PACCHETTO

Pur senza entrare nella specificità della ripartizione dei campi previsti da suite protocollari differenti, è utile fare un breve cenno a quali campi debbano essere comunque contenuti nel pacchetto inoltrato lungo una sottorete di comunicazione:

- ✚ **campi sorgente e destinazione:** soggiacciono a regole di indirizzamento specifiche; saranno presenti tra i primi byte del pacchetto, e possono rappresentare sia un indirizzo WAN univoco che un indirizzo di sottorete locale;
- ✚ **campo lunghezza:** un campo che specifica la dimensione del pacchetto nel suo insieme o della sola intestazione dello stesso;
- ✚ **campo protocollo:** specifica quale è la suite di protocolli tipici della sottorete attraversata;
- ✚ **campo percorso:** individua particolarità relative a circuiti predefiniti o permanenti di attraversamento della rete.
- ✚ **campo versione-servizi:** individua particolarità di impiego di protocollo specifici, sperimentali, o relativi a etichettature di instradamento, ovvero servizi particolari erogabili;
- ✚ **campo frammentazione:** riporta informazioni sul progressivo frammentazione del pacchetto, indicando anche l'eventuale non frammentabilità, o la non ulteriore frammentabilità;

- ✚ **campo Time To Live:** definisce il tempo di vita del pacchetto, allo scopo di evitare che un pacchetto non correttamente inoltrato continui a circolare all'infinito nella sottorete; tale campo di norma rappresenta il numero massimo di secondi di vita del pacchetto dal momento della sua generazione; il valore viene decrementato dai router che trova lungo il suo percorso; il pacchetto viene eliminato, quando il valore raggiunge lo zero;
- ✚ **campo checksum:** rappresenta l'algoritmo di controllo dell'intestazione o dell'intero pacchetto e che viene calcolato di norma alla sorgente ed alla destinazione;
- ✚ **campo dati:** dipende dalla dimensione più o meno flessibile di byte accettati dalla sottorete.
- ✚ Può essere presente inoltre un **campo timestamp**, che fissa data e ora di creazione o di transito del pacchetto su un nodo della sottorete.

CONGESTIONE

Dato che la quantità di pacchetti è molto elevata, bisogna limitare il numero di pacchetti che vanno in collisione tra di loro, maggiori collisioni → maggiori pacchetti persi → maggiori ritardi, generano congestioni. Quindi affianco agli algoritmi di routing abbiamo anche bisogno degli **algoritmi di controllo della congestione**.

Ciascun router possiede un *buffer* di una certa grandezza, in grado di conservare i pacchetti giunti ma non ancora spediti o reinoltrati. Durante il fenomeno della congestione, la quantità di pacchetti in ingresso nel router è superiore alla quantità di pacchetti in grado di essere instradati dallo stesso, aumentando la lunghezza della coda e di conseguenza il ritardo di trasmissione.



A causa del meccanismo di Timeout and Retransmission, tipico della maggioranza dei protocolli di routing, al timeout generato dal ritardo di trasmissione del router, il nodo risponde ritrasmettendo i pacchetti non inoltrati, aggravando quindi il livello della congestione globale della rete. La condizione critica della congestione derivata dall'aumento esponenziale del traffico viene chiamata **Congestion Collapse**, ovvero collasso dovuto alla congestione.

La congestione in un router può derivare da diversi fattori, ad esempio dimensione dei singoli buffer, numero limitato di buffer nel router, processore troppo lento nel router, linea di trasmissione troppo lenta.

Ogni cosa interna di un router viene gestita da una **politica di congestion avoidance** che caratterizza il tipo di reazione che la stessa ha al verificarsi di una congestione. Nel corso di questi anni sono state elaborate diverse politiche, man mano più sofisticate, per fronteggiare il sempre più serio problema della congestione.

Gli algoritmi di controllo della congestione possono operare essenzialmente all'interno degli host della rete di comunicazione, gestendone le comunicazioni, ovvero al livello link, attraverso l'individuazione della congestione da parte dei router, e la successiva segnalazione del problema alla sorgente.

L'approccio al problema della congestione si può ricondurre alla due categorie:

- 1) l'**open loop**: basato su una politica di prevenzione delle congestioni;
- 2) il **closed loop**: basato su una politica di controllo della congestione attraverso azioni opportune.

CHOKER PACKET

Abbiamo sei nodi, in un nodo c'è A che deve trasmettere a un altro nodo B, A manda molti pacchetti, se ne rende conto il router di B che manda un **chock packet** che viene inviato al router più vicino ad A, con il quale chiede di aumentare il buffer sulla porta seriale di collegamento e di diminuire la velocità di invio.

Sostanzialmente è previsto che un router tenga d'occhio il grado di utilizzo delle sue linee di uscita. In dettaglio il router misura, per ciascuna linea, l'**utilizzo istantaneo U** e accumula, entro una **media esponenziale M**, la storia passata:

$$M_{\text{nuovo}} = a M_{\text{vecchio}} + (1 - a)U$$

Dove:

- ✚ il parametro **a**, compreso tra 0 e 1, è il **peso della storia passata**;
- ✚ **(1-a)** è il peso dato dall'informazione più recente.

Quando, per una delle linee in uscita, M si avvicina a una soglia di pericolo prefissata, il router esamina i pacchetti in ingresso per vedere se sono destinati alla linea d'uscita che è in allarme. In caso positivo, invia all'host di origine del pacchetto un **choke packet** per avvertirlo di diminuire il flusso. Quando l'host sorgente riceve il choke packet *diminuisce il flusso ed ignora i successivi choke packet per un tempo prefissato*. Trascorso il tempo prefissato, l'host si rimette in attesa di altri choke packet, se ne arrivano altri dovrà ancora ridurre il flusso, altrimenti potrà aumentare di nuovo il flusso.

HOP-BY-HOP CHOKe PACKET

L'evoluzione del choke packet è l'**hop by hop choke packet** che va a ritroso e manda un pacchetto di rallentamento su ogni router intermedio fino a raggiungere A, dove ormai sarà già rallentata l'onda di pacchetti.

Questa tecnica permette di sbloccare rapidamente la congestione nel punto di creazione della stessa, ma richiede un maggiore utilizzo del buffer nei router sul percorso dall'host originario a quel router.

DROP-TAIL

Se ho un eccesso di pacchetti inizio a scartarli, quindi *cancella i pacchetti quando si raggiunge il massimo di pacchetti del buffer*, è utile nelle comunicazioni real time.

Tale schema non reagisce dinamicamente alle condizioni di traffico della rete, e presenta inoltre diversi svantaggi:

- ✚ *Problema della sincronizzazione dei flussi e della non equa distribuzione della perdita di pacchetti tra le connessioni*: la congestione in atto porta ad una cancellazione indiscriminata di tutti i pacchetti in arrivo, da cui scaturisce la non equa distribuzione della perdita tra le connessioni;
- ✚ *Problema dello scarso utilizzo delle risorse di rete*: le connessioni che subiscono perdite di dati rispondono con un contemporaneo decremento della finestra, che a sua volta genera un calo di traffico in reti, e di conseguenza delle risorse a disposizione.

LOAD SHEDDING

Un router quando viene raggiunto il massimo limite del buffer, può decidere di scartare i pacchetti seguendo precise regole, e dipendenti dall'applicativo in esecuzione:

- ✚ **Regola del wine**: il vecchio è migliore del nuovo. In applicazioni quali il File Transfer è importante la sequenza dei pacchetti, e perciò è preferibile scartare i pacchetti nuovi, tale regola può essere paragonata al **FIFO**;
- ✚ **Regola del milk**: il nuovo è migliore del vecchio, nel caso di applicazioni real time i pacchetti vecchi possono essere scartati, tale regola può essere paragonata al **LIFO**.

ACTIVE QUEUE MANAGEMENT (AQM)

AQM è un **algoritmo dinamico**, è una tecnica che marca i pacchetti in transito e vede com'è l'andamento e se diventa preoccupante, inizia a marcare e a scartare qualche pacchetto con il protocollo che comunica che vengono scartati. Questo scarto permette di gestire la comunicazione, ed è un metodo pre-reattivo.

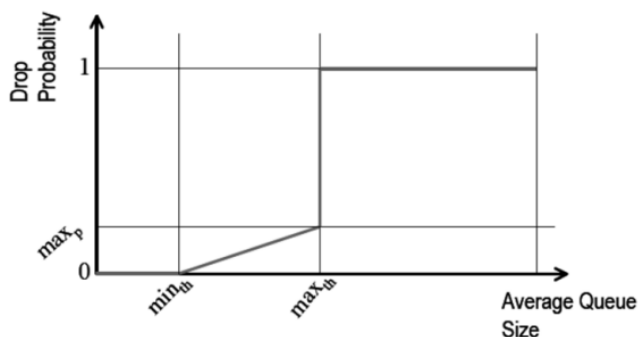
Quest' algoritmo viene implementato tramite l'utilizzo di **Dynamic Buffer Limiting (DBL)** che tiene traccia della lunghezza della coda per ogni flusso di traffico entrante su singolo ingresso del router. Quando la lunghezza di coda di un flusso eccede il suo limite, il DBL comincia a cancellare pacchetti o a impostare i bit **Explicit Congestion Notification (ECN)** nell'intestazione dei pacchetti IP a 1.

RANDOM EARLY DETECTION (RED)

Uno dei più popolari schemi AQM utilizzati in reti TCP/IP viene chiamato **RED**, lavora su un anticipo di questa valutazione misurando la dimensione media delle code con l'utilizzo di un filtro pesato, e reagisce con la cancellazione di pacchetti, in base ad una determinata probabilità.

La probabilità che un pacchetto sia cancellato o contrassegnato (**packet-marking probability**) dipende:

- ✚ dal valore dell'AQM;
- ✚ dal tempo trascorso dall'ultima cancellazione;
- ✚ da un parametro: la probabilità di massima cancellazione (indicata con p_{max} o con max_p) che non deve mai essere superata.

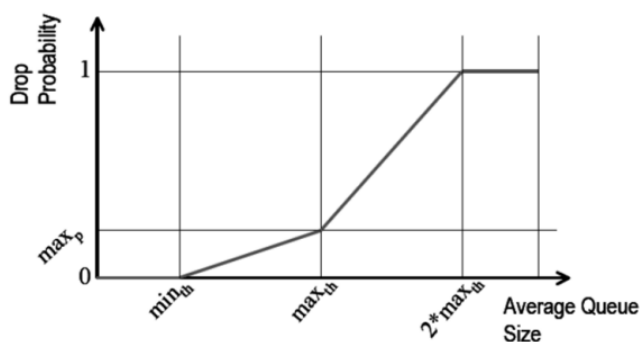


Se la lunghezza media della coda è maggiore di una data soglia massima t_{max} oppure max_{th} , tutti pacchetti vengono cancellati o contrassegnati.

Quest' algoritmo viene chiamato **RED in versione classica**.

L'altra versione è la **gentle-version**: quando la lunghezza media delle code supera la soglia massima, la probabilità di marcatura non diventa unitaria ma continua a crescere, anche se più

velocemente rispetto a prima. Essa diventa unitaria solo dopo che lunghezza media della code supera $2 * max_{th}$.



La gentle-version ha due *linee di inclinazioni diverse*, con una inizio a marcare, se non cambia nulla incremento quella marcatura. L'obiettivo è quello di marcare pacchetti in maniera abbastanza distanziata nel tempo in modo da evitare picchi di traffico, e in maniera sufficientemente frequente da evitare la crescita sconsiderata della lunghezza media della coda.

Questo metodo non riesce a evitare la congestione ma riesce a diminuirla.

Un'insufficienza rilevante si verifica quando la *lunghezza media della coda cresce proporzionalmente al numero di connessioni attive nel sistema*, fino a quando non viene raggiunta la soglia massima, dopo la quale tutti i pacchetti vengono cancellati o contrassegnati, questo è un tentativo radicale di eliminare la congestione che può però sfociare in una sincronizzazione globale.

Con RED, solitamente, si decide di misurare la lunghezza della coda in byte, invece che in pacchetti. Con questa opzione la lunghezza media della coda riflette accuratamente il ritardo medio al router. In questo caso, ad esempio, un grosso pacchetto FTP ha più probabilità di essere marcato rispetto ad un piccolo

pacchetto TELNET. Questa logica tiene in conto *la relazione tra dimensione del pacchetto e la congestione potenziale indotta*.

L'algoritmo di RED, inoltre, può essere modificato per assicurare che la packet-marking probability (probabilità di marcatura) di un pacchetto, sia proporzionale alla sua grandezza.

Un altro degli aspetti caratterizzanti di RED è il non reagire alle **congestioni temporanee**; esse, infatti, sono accompagnate da un *momentaneo aumento di lunghezza della coda* (Queue Length). La **congestione di lunga durata**, invece, si riflette *nell'aumento di lunghezza media della coda* (Average Queue Length: la grandezza monitorata da RED) e porta, quindi, alla richiesta di decremento della finestra per alcune delle connessioni.

Anche in questo caso lo scopo è di privilegiare i carichi duraturi come fautori di congestione potenziale indotta, trascurando i soli picchi di traffico.

Nonostante i difetti che RED presenta, si può dire che la *gestione dinamica e preventiva della congestione aumenta di molto le prestazioni della rete* e risolve quelli che erano i più grandi problemi dell'algoritmo di Drop-Tail, garantendo:

- ✚ **l'evitare della congestione:** con RED la rete viene costantemente monitorata e la sua politica di marcatura preventiva dei pacchetti porta ad una minore perdita di dati, con conseguente minore numero di pacchetti rispediti. La scelta della modalità di marcatura ottiene risultati migliori a seconda della situazione: in caso di cancellazione si ha un immediato alleviamento della congestione, ma una conseguente ritrasmissione dei dati; quando invece i pacchetti vengono contrassegnati, si deve fare più affidamento sulla "onestà" dei mittenti, cioè sul fatto che effettivamente decrementino la grandezza della loro finestra.
- ✚ **tempi appropriati:** dopo aver reso nota una congestione in seguito alla marcatura (tramite contrassegno) di un pacchetto, occorre almeno un Round Trip Time per vedere una diminuzione dell'Arrival Rate (la quantità di dati in arrivo).
- ✚ **nessuna sincronizzazione globale:** l'intervallo di marcatura dei pacchetti dipende dal livello di congestione. Quando il livello di congestione è *basso*, il router ha una bassa probabilità di marcare ogni pacchetto che arriva, e quando aumenta il livello di congestione la probabilità di marcare pacchetti aumenta. I router RED evitano la sincronizzazione globale marcando meno pacchetti possibile. Inoltre RED elimina questo pericolo tramite la casualità della scelta del pacchetto da cancellare o contrassegnare.
- ✚ **semplicità:** l'algoritmo di RED può essere implementato con un carico minimo nelle reti.
- ✚ **massimizzazione del potere globale** (rapporto tra la velocità dei dati spediti ed il ritardo): in simulazioni con grande utilizzo della rete si notano prestazioni migliori di RED rispetto a quelle di Drop-Tail.
- ✚ **gestione equa:** un obiettivo del meccanismo per evitare la congestione è l'equità. Una gestione equa fa in modo da non discriminare nessuna connessione o classe di connessione. La *quantità di pacchetti marcati per ogni connessione è proporzionale all'utilizzo che essa fa della larghezza di banda*. Comunque, non si può assicurare che, effettivamente, ogni utente riceva lo stesso servizio, e d'altronde RED non si preoccupa neanche di controllare "onestà" degli utenti. L'algoritmo ha però dei meccanismi che permettono di identificare le connessioni che stanno utilizzando una grossa fetta della banda disponibile.
- ✚ **appropriatezza per una gran quantità di ambienti:** il meccanismo casuale di marcatura dei pacchetti è appropriato per reti con un grosso range di RTT diversi e larghezze di banda differenti, oltre ad un gran numero di connessioni attive allo stesso tempo. Inoltre, cambiamenti nel carico portano a cambiamenti della lunghezza media della coda e quindi la probabilità di cancellazione viene calcolata di conseguenza.

Un'ulteriore evoluzione è **PI** (Proportional Integral) che consente di mantenere una lunghezza tipo della coda entro un certo range (**lunghezza campione**). Campiona la lunghezza istantanea della coda ad intervalli prestabiliti e costanti calcolando la variazione della probabilità di cancellazione al k-esimo campione. Tuttavia, PI offre una risposta meno rapida ad improvvise oscillazioni del traffico in rete.

ALGORITMO RED

L'algoritmo che calcola il **valore medio della lunghezza della coda** determina il grado massimo di trasmissione che il router può supportare. L'algoritmo che calcola la probabilità di marcare un pacchetto determina *quanto frequentemente il router marca i pacchetti in relazione al livello di congestione*. Tramite una cancellazione di pacchetti a determinati intervalli di tempo si tiene sotto controllo la lunghezza media della coda.

L'algoritmo dettagliato di funzionamento dei router RED è il seguente:

In questo algoritmo possiamo distinguere: **avg** è la lunghezza media della coda, **q_time** è il momento di inizio dello stato di *idle della coda* (cioè il periodo in cui la coda rimane vuota), **count** è il numero di pacchetti instradati senza problemi prima che venisse marcato l'attuale pacchetto, **w_q** è il peso della coda, **min_{th}** è il limite minimo della coda, **max_{th}** è il limite massimo della coda, **max_p** è il valore massimo di **p_b**, **p_a** è la probabilità che l'attuale pacchetto venga marcato, **q** è la lunghezza attuale della coda, **time** è l'istante attuale e **f(t)** è una funzione lineare del tempo.

Il calcolo di questo algoritmo prende in considerazione il periodo di idle della coda, e calcola il numero di pacchetti che potrebbero essere messi in coda durante quel periodo, come se fossero realmente arrivati al router.

In alternativa a questa implementazione dell'algoritmo RED, si può considerare la grandezza della coda in byte piuttosto che in pacchetti.

Inizializzazione:

$avg = 0$

$count = -1$

Per ogni pacchetto in arrivo calcola la nuova lunghezza media della coda **avg**:

se la coda non è vuota

$$avg = (1 - w_q)avg + w_q q$$

altrimenti

$$m = f(time - q_time)$$

$$avg = (1 - w_q)^m avg$$

se $min_{th} \leq avg \leq max_{th}$

incrementa **count**

calcola la probabilità **p_a**:

$$p_b = \max_p (avg - min_{th}) / (max_{th} - min_{th})$$

$$p_a = p_b / (1 - count \cdot p_b)$$

con probabilità **p_a**:

marca il pacchetto in arrivo

$count = 0$

altrimenti se $max_{th} \leq avg$

marca il pacchetto in arrivo

$count = 0$

altrimenti $count = -1$

Quando la coda diventa vuota

$q_time = time$

Con tale modifica, l'algoritmo può essere riespresso in questo modo:

$$p_b = \max_p (avg - min_{th}) / (max_{th} - min_{th})$$

$$p_b = p_b \cdot PacketSize / MaximumPacketSize$$

$$p_a = p_b / (1 - count \cdot p_b)$$

In questo caso verrà marcato con più probabilità un pacchetto grande piuttosto che uno piccolo. Il peso della coda **w_q** è determinato dalla grandezza della coda e dalla durata del periodo con un traffico più consistente del normale.

CALCOLO DELLA LUNGHEZZA MEDIA DELLA CODA

I router RED utilizzano un **filtro passa-basso** per il calcolo della *lunghezza media della coda*. Anche se si verifica un inizio di congestione o un aumento improvviso del traffico, la lunghezza media della coda non varia considerevolmente, coerentemente con quanto detto sul privilegiare carichi duraturi come fattori di congestione potenziale indotta.

Il filtro passa-basso è una *media dinamica pesata esponenziale* (**EWMA** - Exponential Weighted Moving Average):

$$avg = (1 - w_q)avg + w_q q$$

Il peso w_q è una costante del tempo del filtro pass-basso.

LIMITE SUPERIORE PER w_q

Se w_q è *troppo grande*, il metodo del calcolo della media può non filtrare *la congestione iniziale*. Si supponga che inizialmente la coda sia vuota e quindi la sua lunghezza media sia zero, e che successivamente la coda cresca da 0 sino ad L, dove L è il numero di pacchetti arrivati. Perciò all'arrivo dell'L-esimo pacchetto la lunghezza media della coda avg, è:

$$avg_L = \sum_{i=1}^L i w_q (1 - w_q)^{L-i} = w_q (1 - w_q)^L \sum_{i=1}^L i \left(\frac{1}{1 - w_q} \right)^i = L + 1 + \frac{(1 - w_q)^{L+1} - 1}{w_q}.$$

Dato un limite minimo $\min_{th}$, e considerato che si vuole riuscire a gestire un aumento di carico che arrivi fino a L pacchetti in arrivo al router, allora w_q dovrebbe essere scelto in modo da avere che **avg, < $\min_{th}$** , cioè:

$$L + 1 + \frac{(1 - w_q)^{L+1}}{w_q} < \min_{th}$$

LIMITE INFERIORE PER w_q

È ragionevole scegliere un buon limite inferiore w_q per in quanto non è desiderabile che la lunghezza media della coda avg non reagisca troppo rapidamente e che il router non riesca a rendersi conto della congestione nascente.

Si consideri che in un primo momento la coda passi da essere vuota a contenere un pacchetto, e che poi i pacchetti arrivino e vengano rispediti alla stessa velocità, perciò che nella coda rimanga sempre un pacchetto. Inoltre si consideri che inizialmente la *lunghezza media della coda sia zero*. In questo caso la coda prende $-1/\ln(1 - w_q)$ pacchetti in arrivo (con la lunghezza della coda che rimane a uno) sino a che la lunghezza media della coda giunge a $0.63 = 1 - 1/e$. Nella maggior parte dei simulatori si utilizza $w_q = 0.002$.

ASSEGNAZIONE DI $\min_{th}$, E $\max_{th}$

I valori dei limiti minimo e massimo, $\min_{th}$ e $\max_{th}$, dipendono dalla lunghezza media della coda che si vuole mantenere. Se ad esempio si vuole mantenere un traffico molto intenso per abbastanza tempo, il *limite minimo* deve essere abbastanza alto, come anche quando si vuole che il ritardo di un pacchetto non influenzi eccessivamente l'affidabilità del collegamento.

Per quanto riguarda il *limite massimo*, questo dipende largamente dal massimo ritardo medio che il router è disposto ad accettare.

I router RED lavorano meglio quanto la differenza $\max_{th}$ - $\min_{th}$, è maggiore del tipico aumento della lunghezza media della coda in un RTT (Round Trip Time, tempo di andata e ritorno). Una regola che quasi sempre è adatta è quella di scegliere un $\max_{th}$ uguale al doppio del $\min_{th}$.

CALCOLO DELLA PROBABILITÀ DI MARCARE UN PACCHETTO

La probabilità iniziale di marcare un pacchetto p_b è calcolata come una funzione lineare della lunghezza media delle code, con la seguente formula:

$$p_b = \max_p (avg - \min_{th}) / (\max_{th} - \min_{th}).$$

dove $\max_p$, rappresenta il valore massimo che la probabilità di marcare un pacchetto p_b può assumere.

Un metodo per calcolare la probabilità finale di marcare un pacchetto utilizza una *variabile geometrica random*. Assumiamo che X sia il tempo che intercorre tra due pacchetti che vengono marcati, ossia il numero di pacchetti che arrivano dopo un pacchetto marcato senza essere a loro volta marcati. Dato che ogni pacchetto viene marcato con una probabilità p_b allora:

Dato che X è una variabile geometrica standard di parametro p_b , avrà una media $E(X) = 1 / p_b$.

Avendo una *lunghezza media di coda costante*, è auspicabile che non vi sia un intervallo né troppo breve né troppo grande tra due pacchetti marcati. Entrambi questi effetti possono portare alla sincronizzazione globale, cioè la situazione che si presenta quando molte connessioni riducono la grandezza della propria finestra contemporaneamente, che è appunto ciò che accade con l'utilizzo di questo primo metodo di calcolo della probabilità.

Un altro approccio è quello di utilizzare una *variabile uniforme random* X , che assuma i valori $\{1, 2, \dots, 1/p_b\}$ assumendo per semplicità che p_b sia un intero. Per ottenere ciò, la probabilità di marcare un pacchetto deve essere $p_b / (1 - \text{count} * p_b)$, dove count è il numero di pacchetti non marcati che sono arrivati dopo l'ultimo pacchetto marcato. In questo caso si ha che:

$$\begin{aligned} \text{Prob}[X = n] &= \frac{p_b}{1 - (n-1)p_b} \prod_{i=0}^{n-2} \left(1 - \frac{p_b}{1 - ip_b}\right) \\ &= p_b \text{ con } 1 \leq n \leq 1/p_b, \\ \text{Prob}[X = n] &= 0 \text{ con } n > 1/p_b. \end{aligned}$$

Con questo secondo metodo la media $E[X] = 1/(2 p_b) + 1/2$

IMPLEMENTAZIONE DELL'ALGORITMO RED

L'implementazione da parte dei router dello schema RED porta all'attuazione di alcuni degli scopi dell'algoritmo RED stesso. La **prevenzione della congestione** si attua grazie alla cancellazione di alcuni pacchetti quando viene raggiunto il limite massimo. In questo modo il calcolo della lunghezza media della coda non supererà il limite massimo.

$$\text{Prob}[X = n] = (1 - p_b)^{n-1} p_b.$$

Se viene assegnato un giusto valore a w_q per la procedura descritta, il router controllerà la reale lunghezza media della coda. Se invece di cancellare il pacchetto, quando viene raggiunto il limite massimo, vengono impostati i bit dell'intestazione, si renderà possibile una cooperazione nella prevenzione della congestione da parte degli altri router.

Un'appropriata bilancia temporale permette che, dopo che si notifica una congestione con il marcamento del pacchetto, in un tempo pari al Round Trip Time (RTT), il router noti una diminuzione di velocità di arrivo dei pacchetti. La bilancia temporale, quindi, misura il tempo di reazione alla congestione delle connessioni.

La *sincronizzazione globale è da evitare*, e per fare questo si devono marcare il minor numero di pacchetti possibile se la congestione è bassa, tramite il calcolo della probabilità di marcare un pacchetto. Questa infatti è più bassa quanto più è bassa la congestione. In questo modo non si avrà un crollo del traffico improvviso da parte di tutti le connessioni.

È necessario che gli algoritmi dei router RED siano implementati con una spesa abbastanza ridotta per le reti attuali, quindi l'implementazione stessa deve essere semplice.

La *massimizzazione del rapporto tra larghezza di banda e ritardo globale* si ottiene tramite il controllo esplicito della lunghezza media della coda. Infatti è dimostrato che il suddetto rapporto è più alto con l'utilizzo dei gateway RED piuttosto che di quelli con normale Drop Tail.

Uno degli scopi della prevenzione della congestione è *il corretto bilanciamento nella condivisione della coda*. I router RED non devono discriminare certe connessioni rispetto alle altre, infatti il numero di pacchetti marcati di una certa connessione *sarà proporzionale alla larghezza di banda di tale connessione*.

I router RED non controllano che ogni connessione sfrutti uguale larghezza di banda, ma è un controllo che si può aggiungere. In particolare, un controllo del genere è utile proprio durante un periodo di congestione.

I router RED devono essere *adatti ad un grande insieme di ambienti*. Il meccanismo di scelta dei pacchetti da marcare fa sì che si marchi con più probabilità un pacchetto di una connessione che sfrutta più larghezza di banda o con diversi valori di RTT, o quello di un host che sfrutta più connessioni contemporaneamente.

QUALITÀ DEL SERVIZIO

In una rete connection oriented tutti i pacchetti di un flusso seguono lo stesso percorso, in una rete connectionless i pacchetti possono seguire percorsi differenti. A ciascun flusso è possibile associare una determinata **Qualità del Servizio** classificabile con i seguenti parametri:

- ✚ **affidabilità**: nessun bit può essere trasmesso in modo scorretto non voglio perdere nessun pacchetto. Quante volte la rete riesce a garantire il servizio senza down? Se avviene quante volte va in down?
- ✚ **ritardo**: il quale deve essere valutabile e controllabile. Si verifica la differenza che c'è dal valore di Mega forniti dall'ISP secondo contratto, rispetto a quelli che realmente giungono al cliente;
- ✚ **jitter** (tremolio): indica i picchi di malfunzionamento con i quali si ha una variazione del tempo di arrivo di ciascun pacchetto;
- ✚ **banda**: bisogna garantire una banda minima garantita (BMG).

La QoS dipende dalla tipologia delle reti:

- ✚ *Reti a Velocità Costante*: fornisce banda uniforme e ritardo uniforme (quali ad esempio quelle telefoniche);
- ✚ *Reti a Velocità Variabile in tempo reale* (quali la videoconferenza);
- ✚ *Reti a Velocità Variabile non in tempo reale* (quali video on-demand);
- ✚ *Velocità Disponibile*: per applicazioni non sensibili al ritardo o allo jitter (per esempio il file transfer).

Diverse tecniche sono state adottate per ottenere una buona qualità del servizio:

- ✚ Bufferizzazione;
- ✚ leaky bucket (secchio bucato);

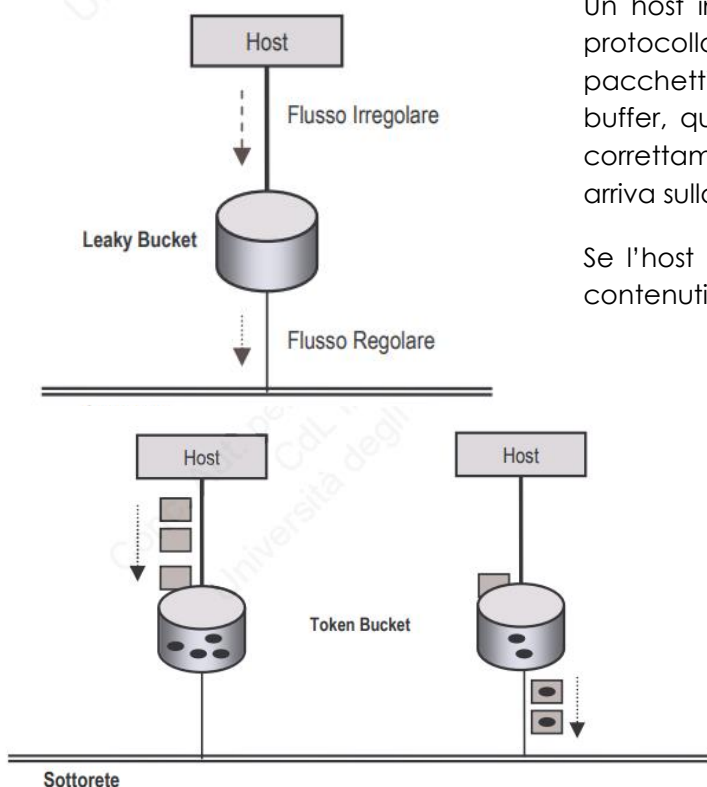
- ✚ token bucket (secchio di gettoni);
- ✚ flow specification (specifica o parametrizzazione del flusso);
- ✚ routing adattivo;
- ✚ sovradimensionamento.

BUFFERIZZAZIONE

Tecnica che consiste nella **memorizzazione del flusso**, prima della consegna, in un **buffer**. Ho una quantità di pacchetti prodotti da una porta seriale di un router in maniera costante, i pacchetti arrivano e hanno una loro bufferizzazione che è tale da rilasciare i pacchetti in modo tale che la macchina ricevente li riceva con continuità di servizio, il limite è il costo delle risorse.

Altri problemi possono essere il ritardo, il jitter e la creazione di code anomale

LEAKY BUCKET – TOKEN BUCKET



Un host invia tutti i pacchetti che vuole e in mezzo ha un protocollo **leaky bucket** autorizzato a riservare sulla rete pacchetti con un fissato data rate e che mantiene, nei suoi buffer, quelli accodati per la trasmissione. Se tutto funziona correttamente, tale è il flusso dell'host e tale è il flusso che arriva sulla sottorete.

Se l'host genera più pacchetti di quelli che possono essere contenuti nei buffer, questi vengono perduti.

Se l'host ha degli incrementi di velocità questa tecnica funziona così: il secchio genera un gettone per ogni unità di tempo stabiliti se io ho tanti pacchetti quanti gettoni posso trasmettere, se io ho meno pacchetti che gettoni posso reggere un eventuale picco di traffico, serve per tarare i picchi di traffico. (**secchio di gettoni** o token bucket).

FLOW SPECIFICATION

Il **traffic shaping** è molto efficace se sorgente, sottorete e destinazione si accordano in merito. Un modo di ottenere tale accordo consiste nello specificare:

- ✚ le caratteristiche che si vuole inviare (ad esempio, data rate, grado di bruschetta);
- ✚ la qualità del servizio (ad esempio ritardo massimo, frazione di pacchetti che si può perdere).

Se io so che devo fare una trasmissione e mantenere un data rate prestabilito cerco di pre-allocare la banda e specifico qual è il tipo di flusso, posso farlo anche per un determinato blocco di tempo.

Tale accordo viene identificato con il nome di **flow specification** e consiste in una struttura dati che descrive le grandezze in questione. Sorgente, sottorete e destinatario si accordano di conseguenza per la trasmissione, questo accordo viene preso prima della trasmissione, e può essere stipulato sia in *sottoreti connesse* sia in *sottoreti non connesse*.

In generale nelle reti connesse è più facile il controllo della congestione, perché le risorse per una connessione sono allocate in anticipo. Per evitare la congestione è *possibile negare l'attivazione di nuovi circuiti virtuali dove non ci sono sufficienti risorse per gestirli*. Questa tecnica va sotto il nome di **admission control**.

ROUTING ADATTIVO

Molti degli algoritmi di routing tentano l'instradamento di tutto il traffico per ciascuna destinazione, attraverso il percorso migliore. I router non hanno in generale una visione completa dei path di traffico di tutta la rete.

È nato così un approccio diverso per garantire una qualità del servizio elevata, il **routing adattivo**. Tale metodologia si basa nella *suddivisione in percorsi differenti del traffico diretto in ciascuna direzione*, utilizzando le informazioni disponibili localmente.

Il metodo più semplice consiste nella suddivisione del traffico in parti uguali nei percorsi alternativi, o proporzionali alla banda disponibile per ciascun percorso. In tal modo, per ogni collegamento diretto, ci si potrà attendere più frazioni di banda uguali ed in grado di essere gestite alla stregua di una moltiplicazione statistica.

Sostanzialmente viene fatto quando il routing ha dei suoi protocolli ulteriori rispetto a quelli normali di rete che consentono di adattare anche per porzioni di rete il routing relativo.

SOVRADIMENSIONAMENTO

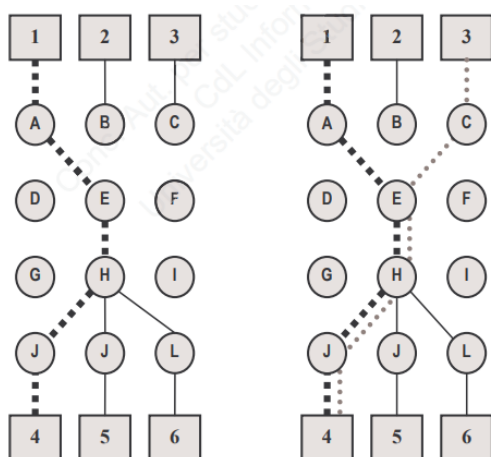
Prevede un'attribuzione di QoS in ambito di servizi essenziali e/o militari, è una tecnica che *consiste nell'aumentare la capacità di calcolo e le dimensioni del buffer dei router e l'ampiezza di banda* per singole tratte della sottorete (esempio: *necessito di 5Mb/s e ne richiedo di più per sicurezza*).

Risulta evidente costosa, e spesso inapplicabile. Tuttavia, può rappresentare una valida soluzione se circoscritta nel tempo e per percorsi definiti.

SERVIZI INTEGRATI

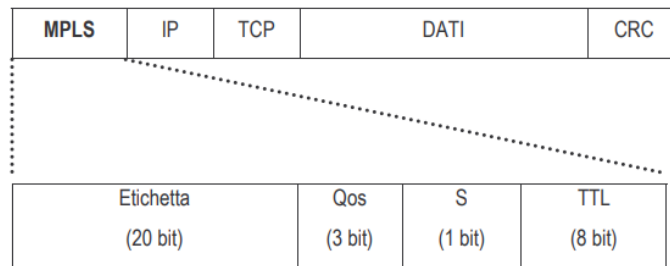
Quando i flussi di dati sono *multimediali o real time* ho bisogno di una capacità di rete più elevata, allora qui la QoS ci ha fornito un nuovo tipo di algoritmi: quelli basati sui flussi o i **servizi integrati**. Tali algoritmi sono rivolti principalmente ad applicazioni unicast e multicast.

Il primo algoritmo che consideriamo è il **protocollo RSVP** (Resource reSerVation Protocol), il quale è in grado di occuparsi principalmente della gestione delle prenotazioni, ed ottimizza l'uso della banda e contemporaneamente elimina le collisioni. Il protocollo utilizza il routing multicast basato su strutture **spanning tree**.



Abbiamo una rete con 3 host che stanno in alto e 3 host che stanno in basso, se io devo fare il collegamento fra 1 e 4 passando per A-E-H-J tutto va bene, ma se devo fare anche un altro collegamento da 3 a 4 abbiamo i 3/5 del percorso occupato, forse non avrei abbastanza risorse. **Quindi cosa fa il protocollo?** Sa che questa è una comunicazione che avverrà in una determinata ora quindi prealloca la risorsa, mettendo a disposizione di quel canale quella dimensione di banda che gli consente di utilizzare anche successivamente quella determinata tratta.

Un'altra efficiente metodologia consiste *nell'aggiungere al pacchetto un'etichetta*, al fine di permettere ai router l'instradamento del traffico in base ad etichette e non a indirizzi di destinazione, e permettendo in tal modo una gestione più efficace delle risorse. Tale tecnica prende differenti nomi, **label switching** o **tag switching**. Il nome assegnato dall'IETF, e così standardizzato è **MPLS** (MultiProtocol Label Switching).



La sequenza dei campi contenuti nel pacchetto MPLS, fa sì che, a rigore nella pila gerarchica ISO-OSI, la sua collocazione debba intendersi tra i livelli di collegamento e di rete, tali atipicità e flessibilità forse sono le ragioni del suo impiego sempre più ampio.

L'intestazione MPLS ha 4 campi:

- ✚ **etichetta**(label): contiene l'indice che serve ad individuare la linea di trasmissione;
- ✚ **QoS**: indica la classe di servizio del pacchetto;
- ✚ **S**: bit di accatastamento di più etichette, permette di conoscere l'esistenza di etichette incapsulate in altre etichette, il suo valore posto a 1 sta a indicare l'appartenenza ad un circuito virtuale, o un circuito virtuale concatenato, che il MPLS rende disponibile anche tra sottoreti diverse tra loro;
- ✚ **TTL** (time to live): viene decrementato ad ogni passaggio su un router, se il suo valore è nullo il pacchetto viene scartato.

Due router possono inviare sulla stessa linea di uscita dei pacchetti non in rapporto tra loro, ma con stessa etichetta, creando quindi dei circuiti virtuali. È possibile associare un gruppo di etichette a ciascun flusso per la sottorete, o associare un'etichetta collettiva a più flussi. In quest'ultimo caso, tali flussi sono chiamati **Forwarding Equivalence Class** (FEC).

SERVICE LEVEL AGREEMENT

Sempre più spesso la garanzia di un apprezzabile Qualità di Servizio è frutto non esclusivo dell'impiego di un solo protocollo ad essa deputato. Come già trattato in precedenza, nelle metodiche proprie del Traffic Shaping, la QoS è significativamente dipendente da un impiego concordato di risorse non esclusivamente riconducibili ai protocolli attivi sulla sottorete di comunicazione.

Poter commisurare il carico in ingresso in un router al throughput ottimale dello stesso, risulta essenziale concordare il complesso delle prestazioni desiderate dall'Host, o dagli Host, in ingresso sulla sottorete con quelle di fatto erogabili dalla sottorete stessa.

A tal fine, da alcuni si è cercato di regolamentare tale rapporto *definendo delle soglie di garanzia minima, o massima, concordata e misurabile su specifici parametri*.

La definizione dei livelli di servizio in tal modo introdotti prende nome di **Service Level Agreement** (SLA).

Le SLA hanno la duplice funzione, verso l'Host e verso la sottorete, di raggruppare al loro interno un insieme di servizi, spesso implementati nei protocolli di livello rete, che rendono misurabili prestazioni specifiche sia dell'Host che della sottorete. Le SLA hanno una flessibilità di parametrizzazione che ne consente l'impiego con architetture e tecnologie di rete differenti.

Le prestazioni che le SLA standard prevedono vengono assicurate principalmente sulla sottorete, considerando di norma influente il contributo del singolo Host.

I livelli di servizio tipicamente considerano:

- ✚ **throughput per port** (bit/sec, frame/sec, packets/sec): garanzia sul livello minimo di produttività all'oltro assicurato per porta del router di sottorete;
- ✚ **data delivery ratio, DDR** (bit/sec): garanzia sul livello minimo assicurato di consegna dei dati;

- ✚ **constant bit rate, CBR** (bit/sec): garanzia sulla mantenimento di una velocità minima concordata di consegna di dati per applicazioni particolari, tipico dell'ATM;
- ✚ **back-up di linea** (a caldo, dinamico) (a freddo, statico): garanzia della continuità trasmissiva, in caso di caduta del collegamento principale, con linea di back-up già predisposta (a caldo) con la quale si può direttamente switchare perdendo solo gli ultimi pacchetti , o predisposta al momento (a freddo), e, in quest'ultimo caso, i tempi massimi e le risorse stimate per tale operazione;
- ✚ **back-up router** (a caldo, dinamico) (a freddo, statico): garanzia della continuità trasmissiva, in caso di malfunzionamento grave o non funzionamento del router di interfacciamento, con router di back-up già predisposto (a calde) o predisposto al momento (a freddo), e in quest' ultimo caso, i tempi massimi e le risorse stimate per tale operazione;
- ✚ **uptime** (percentuale/mese-anno): garanzia di una percentuale minima stimata di mantenimento del servizio in essere per un tempo di riferimento;
- ✚ **report service unit** (RSU): garanzia di poter usufruire di strumenti di controllo delle prestazioni da parte dell'Host, tipicamente non interattivi e di sola visualizzazione;
- ✚ **management service unit** (MSU): garanzia di poter usufruire di strumenti di configurazione delle prestazioni da parte dell'Host, tipicamente gestione della banda a propria disposizione, e generazione o modifica di circuiti virtuali permanenti.

La complessità derivante dalla verifica e/o dalla misurabilità dei livelli di servizio delle SLA, hanno portato alla definizione di più semplici e strutturati parametri di indagine denominati **KPI** (Key Performance Indicator) che consentono di raggruppare in un unico reporting set l'insieme di KPI necessari alla descrizione del Service Level Agreement desiderato. Se scende sotto 1, il servizio non sta funzionando come vorremmo.

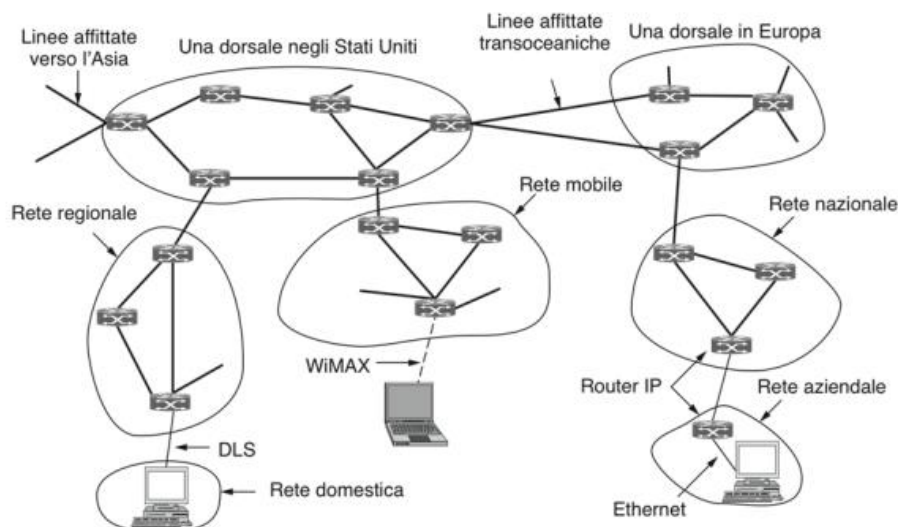
LIVELLO RETE IN INTERNET

Quello che abbiamo trattato sinora è la generalità che regolano le comunicazioni di rete, che non comprende quella che è l'infrastruttura di rete più usata al mondo, ovvero la rete **internet**. Per poter far funzionare questo protocollo correttamente, già dall'evoluzione che parte dal modello ARPANET, siamo obbligati ad avere delle garanzie, ovvero:

- ✚ **Mantenimento della semplicità all'aumentare delle funzioni:** bisogna continuare a garantire la funzionalità e semplicità dei protocolli elementari di livello più basso, in maniera tale che siano sempre presenti e sempre in funzione mantenendo la stessa logica su cui si sono sempre basati anche all'aumentare delle funzioni, più o meno complesse, che si possono effettuare;
- ✚ **Sfruttamento della modularità (indipendenza dei livelli):** Il livello di rete, come altri livelli deve, essere progettato in maniera tale che si possa introdurre o rimuovere protocolli senza che siano alterate le funzioni dell'intero sistema, in maniera tale da non richiedere ulteriori funzionalità dal livello sottostante e soprastante. Inoltre, i servizi devono essere tipici di un determinato livello e non devono essere effettuati da altri livelli;
- ✚ **Tener conto dell'eterogeneità degli utilizzatori:** quando abbiamo più sistemi operativi o più dispositivi differenti , oppure credenziali diverse, dobbiamo rendere il tutto connettibile, non basterà utilizzare dei sistemi responsive o adattivi;
- ✚ **Riduzione dell'impiego di parametri statici:** dato che gli algoritmi dinamici sono molto più efficienti si cerca di diminuire sempre di più l'impiego di parametri statici;
- ✚ **Rigore nella trasmissione e tolleranza nella ricezione:** bisogna essere abbastanza rigorosi nel fatto che la trasmissione debba avvenire comunque, io devo inviare quei dati attraverso un protocollo di invio senza il quale non esisterebbe la filiera TCP/IP, ma devo essere tollerante nella ricezione assumendomi il rischio di ricevere duplicati, pacchetti in ritardo, ecc;
- ✚ **Garantire la scalabilità per garantire l'espansione della rete:** la rete deve essere in grado di potersi espandere, è nella obiettività di Internet. La rilevanza storica della rete internet, che prima comprendeva poche decine di macchine in tutta Italia adesso ne include decine di milioni, per questo è importante;

- ✚ **Considerare il rapporto costo/prestazioni:** risulta chiaro che si possono avere delle prestazioni eccezionali spendendo una quantità di denaro elevato, ma non è l'obiettivo di Internet, per questo il rapporto è importante, poiché Internet è da sempre composto da una comunità no profit.

INTERNET COME INSIEME DI RETI INTERCONNESSE



All'interno è come se avessimo l'oceano Atlantico, a destra troviamo la *dorsale europea* con dei router che collegano le varie capitali, sotto di questa ci sono le *reti nazionali* tipo la GAR, al di sotto esistono dei router specifici chiamati *boundary router*, che vedono dei router ancora più piccoli chiamati *router di confine*, e alla fine troviamo l'utilizzatore. Ho delle linee per **comunicazioni di lunga distanza** che si *chiamano linee affittate transoceaniche*, dall'altra parte abbiamo un'altra dorsale nazionale, e su quello stato esistono delle singole *reti regionali*, con cui si intende una zona molto grande.

Posso costruire una rete in capo ad un provider che si sovrappone territorialmente ad una rete ad un capo di un altro provider.

Al di sotto delle reti regionali c'è una **DLS** che ci porta in una rete domestica. Alla destra troviamo una rappresentazione di *rete mobile*, dove il **WiMAX** mi permette di collegare dei dispositivi mobili che sono agganciati ad una rete fisica che è appunto la rete mobile.

Il collante che tiene unita Internet è il protocollo a livello di rete: **IP** (Internet protocol). A differenza della maggior parte dei protocolli di livello di rete più vecchi, IP è stato progettato fin dall'inizio pensando alla comunicazione tra reti.

Un buon modo di pensare al livello di rete è il seguente: il suo compito è *fornire un servizio di tipo best effort* (perciò non garantito) *per trasportare i datagrammi inviati da una sorgente a una destinazione*, senza tener conto della posizione delle macchine che potrebbero trovarsi su reti diverse e della eventuale presenza di reti intermedie poste tra loro.

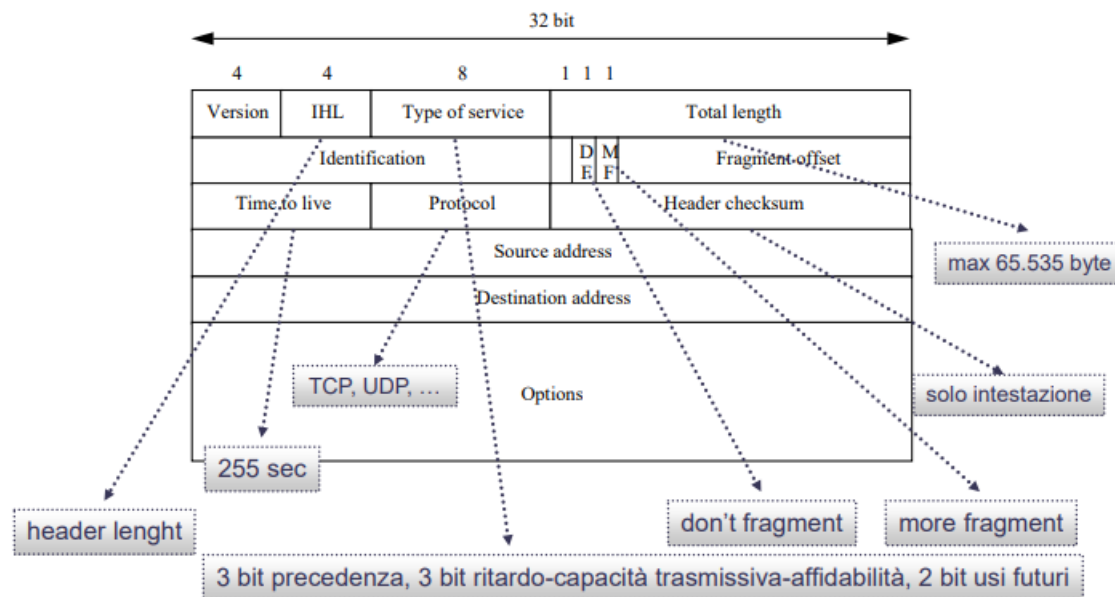
La comunicazione in Internet funziona in questo modo: il livello di trasporto prende i flussi di dati e li divide in modo tale da poter essere inviati in pacchetti IP. In teoria, ogni pacchetto può essere grande **64 KB**, ma all'atto pratico la loro dimensione di solito non supera i **1500 byte** (così possono essere inseriti in un frame Ethernet, abbiamo una correlazione 1:1 con ethernet). I router inoltrano ogni pacchetto attraverso Internet lungo il percorso da un router all'altro finché non viene raggiunta la destinazione, dove il livello di rete passa i dati al livello di trasporto, che li dà al processo destinatario.

Nell'esempio della figura soprastante un pacchetto generato dall'host della rete domestica deve attraversare quattro reti e molti router IP per raggiungere la rete aziendale dell'host di destinazione. Questo non è inusuale nella pratica, anzi, spesso il numero di reti attraversate è maggiore. C'è anche *molta ridondanza di connettività in Internet*, perché le dorsali e gli ISP si connettono tra di loro in più punti. Questo

significa che ci sono molti percorsi possibili tra due host e sono i protocolli di routing per IP che devono decidere quali utilizzare.

Di norma localmente viene utilizzata la versione IP quattro **IPv4**, mentre a livello di dorsali è utilizzata la versione sei **IPv6**.

INTESTAZIONE DI IPV4



Dimensione massima intestazione 60 byte (Opzioni 40 byte), ogni indirizzo IP è costituito da 32 bit, che la immaginiamo come sequenza di 4 volte di 8 bit. Questo è l'header ovvero l'intestazione, che il router deve analizzare una volta ricevuto un pacchetto. Fondamentalmente, è come se lo dividessimo in 5 parti.

PARTE 1

- ✚ **Versione:** IPv4 o IPv6 abbiamo i quattro bit che rappresentano la versione per indicare la versione di protocollo;
- ✚ **IHL (header length):** indica la lunghezza dell'intestazione espressa in parole di 32, questo serve perché per dare informazioni su un pacchetto IP, che è un pacchetto datagramma, si aggiungevano dei campi options che andavano a saturare enormemente il campo header, quindi si sono stati separati, e serve a indicare che ci sono altri campi sotto e quindi bisogna avere dello spazio anche per loro. Il valore minimo, 5, si applica quando non sono presenti altre opzioni. Il valore massimo di questo campo a 4 bit è 15, quindi la dimensione massima dell'intestazione è 60 byte e il campo Options (opzioni) non può occupare più di 40 byte;
- ✚ **Type of service:** era (ed è ancora) usato per distinguere diverse classi di servizio; sono ammesse svariate combinazioni di affidabilità e velocità;
- ✚ **Total length:** lunghezza totale data dalla intestazione dai dati che sono di 1500 byte, il checksum e il trailer. La sua lunghezza totale può arrivare a **65535 byte**, perché abbiamo 16 bit ($2^{16}=65536$).;

PARTE 2 (RECORD DELLA FRAMMENTAZIONE):

In alcuni casi il nostro campo da 1500 byte (intestazione + CRC) diventa pesante, anziché inviare 1500 byte, invio 3 pacchetti da 500. Questo record, quindi, serve a dire che la frammentazione è relativa a quel blocco di pacchetti e che quel pacchetto frammentato è per esempio 1 di 3, 2/3 o 3/3 in modo che arrivato a destinazione lo ricostruisco. Abbiamo:

- ✚ **Identification:** serve all'host di destinazione per determinare a quale datagramma appartiene il frammento appena arrivato. Tutti i frammenti di un datagramma contengono lo stesso valore di Identification (16 bit);
- ✚ **Bit nullo;**
- ✚ **DF:** dice di non frammentare ulteriormente, se c'è il DF posto a 1 il pacchetto non verrà frammentato ulteriormente;
- ✚ **MF:** consente di frammentare ulteriormente se il suo bit è posto a 1;
- ✚ **Fragment offset:** indica la posizione del frammento nel datagramma corrente che consente di ricostruire il pacchetto com'era all'origine. Tutti i frammenti tranne l'ultimo devono essere multipli di 8 byte, che è la dimensione del frammento elementare. Questo campo è lungo 13 bit, dunque possiamo avere 2^{13} frammenti, ossia 8192 frammenti per datagramma.;

PARTE 3:

- ✚ **Time to live:** quanto deve rimanere in vita il pacchetto, è calcolato con un decremento che parte da **255 secondi**, che si decrementa a ogni salto. Nella realtà però, questo campo riflette solo il numero di salti, e quando il contatore arriva a zero il pacchetto viene scartato in modo da non lasciarlo in giro per sempre se dovesse bloccarsi per qualche motivo;
- ✚ **Protocol:** indica quale processo di trasporto è in attesa di quei dati. TCP è una possibilità, ma ci sono anche UDP e altre soluzioni;
- ✚ **Header checksum:** somma di controllo dell'intestazione, l'header è talmente importante che ha bisogno di una sua somma di controllo;

PARTE 4: indirizzi di destinazione e indirizzo sorgente a 32 bit;

PARTE 5: campo options da 32 bit che posso attaccare all'intestazione. Alcune opzioni sono:

- ✚ **Security:** Specifica il livello di sicurezza del datagramma;
- ✚ **Strict source routing:** definisce il percorso completo dalla sorgente alla destinazione, attraverso una sequenza di indirizzi IP. Il datagramma deve seguire quel percorso preciso. Per lo più è utilizzato dagli amministratori di sistema per inviare pacchetti di emergenza in caso di danneggiamento dei router o per eseguire misure di tempi;
- ✚ **Loose source routing:** Elenco dei router dove non passare;
- ✚ **Record router:** Fa sì che ogni router aggiunge il proprio indirizzo IP (ci dice il pacchetto per quali router è passato);
- ✚ **Timestamp:** Fa sì che ogni router aggiunge indirizzo e ora (ci dice quando il pacchetto c'è passato).

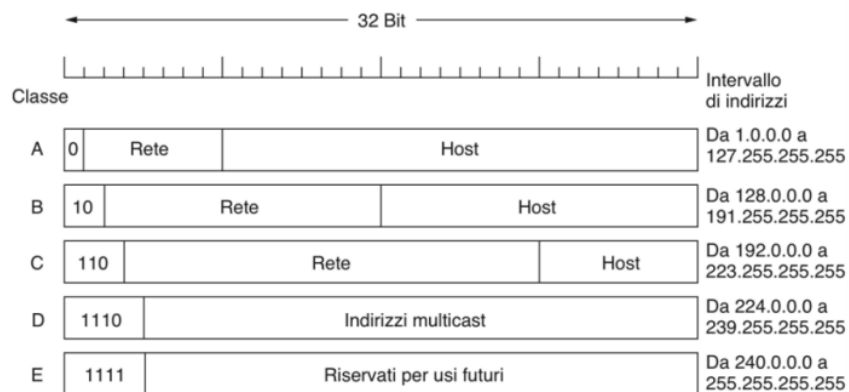
INDIRIZZI IP (RFC 791)

Al contrario degli indirizzi Ethernet, gli indirizzi IP sono **gerarchici**. Ogni indirizzo a 32 bit è composto di una parte di rete di lunghezza variabile nei primi bit e di una parte per l'host negli ultimi. La parte di rete ha lo stesso valore per tutti gli host di una singola rete, come in una LAN Ethernet. Questo significa che una rete corrisponde a un blocco contiguo di indirizzi IP; tale blocco è chiamato **prefisso**.

Gli indirizzi di rete sono scritti in **notazione decimale puntata**. In questo formato ognuno dei 4 byte è rappresentato in forma decimale con un numero che varia tra 0 e 255. Per esempio, l'indirizzo esadecimale di 32 bit 80D00297 è scritto come 128.208.2.151. I prefissi sono scritti dando l'indirizzo IP più basso del blocco e la grandezza del blocco. La dimensione è determinata dal numero di bit nella porzione di rete; i bit rimanenti nella parte dedicata all'host possono variare.

Prima del 1993 gli indirizzi IP erano suddivisi nelle cinque categorie mostrate nella figura sottostante, classificazione chiamata **classfull addressing** (indirizzamento per classi):

Esiste una suddivisione canonica degli indirizzi, che tipicamente ha 3 livelli di classi principali (A, B e C), che si basano sull'attribuzione da parte degli enti proposti a livello mondiale o a livello regionale, nel consentire di avere degli indirizzi a disposizione dell'utente che li ha richiesti.



Gli indirizzi che vengono assegnati dall'ente sono indirizzi di rete e sono dei **Network Identification**, gli indirizzi che vengono gestiti da chi ha ottenuto la rete sono quelli propri e quindi degli **Host Identification**. La batteria canonica è suddivisa in 3 classi principali, A B e C, in funzione degli indirizzi che vengono dati monte da chi assegna gli indirizzi.

Le prime tre classi sono quelle che ci interessano:

- ✚ **A:** con il primo bit bloccato a 0, il Net ID sono i primi 8 bit, gli altri 24 vengono gestiti all'interno di un'azienda. Con una classe A, sono disponibili nell'Host ID più di 17 milioni indirizzi, pertanto ad oggi non vengono assegnate classi A.;
- ✚ **B:** i primi due bit sono bloccati al valore 10, hanno i primi 16 bit come Net ID, l'Host ID è di 16 bit posso indirizzare 65.536 macchine;
- ✚ **C:** hanno i primi 3 bit bloccati al valore 110, hanno un Net ID di 24 bit e 8 bit assegnabili nel campo Host ID, con 256 macchine indirizzabili.

Gli altri indirizzi sono indirizzi multicast ed indirizzi per ora lasciati ad usi futuri

Quindi *il router per capire con chi a che fare verifica i primi bit.*

ESEMPIO: Se io avessi una azienda con 100 nodi, quindi 100 macchine che devono essere collegate in rete e devono essere rese pubbliche, faccio una richiesta e mi assegneranno la classe C, non utilizzerò tutti i nodi disponibili ma comunque mi assegnano questa classe. All'interno della classe C attribuisco i vari indirizzi alle mie macchine.

Questo è uno schema gerarchico in cui la **dimensione dei blocchi di indirizzi è fissa**. Esistono più di 2 milioni di indirizzi, ma organizzare gli indirizzi in classi ne spreca milioni. In particolare, il vero cattivo della situazione è rappresentato dalla rete di classe **B**: per la maggior parte di enti e aziende, una rete di classe A con 16 milioni di indirizzi è troppo grande e una rete di classe C con 256 indirizzi è troppo piccola. Una rete di classe B (con 65.536 indirizzi) invece è perfetta. In Internet questa situazione è chiamata **three bears problem** (problema dei tre orsi).

In realtà, un *indirizzo di classe B è ancora di gran lunga troppo grande per la maggior parte delle aziende*. Le ricerche hanno dimostrato che più della metà di tutte le reti di classe B hanno meno di 50 host. Una rete di classe C sarebbe stata sufficiente, ma senza dubbio ogni azienda che ha chiesto un indirizzo di classe B pensava che un giorno gli 8 bit del campo host non sarebbero più stati sufficienti. In retrospettiva, sarebbe stato meglio assegnare al numero di host delle reti di classe C 10 bit invece di 8, in modo da supportare 1022 host per rete. Con una scelta del genere, *la maggior parte delle aziende avrebbe richiesto una rete di classe C e gli indirizzi di questo tipo sarebbero stati mezzo milione*.

Nel 1990, come abbiamo comunicato in precedenza vi fu una grande crisi che di indirizzi che mise il sistema di indirizzamento quasi al collasso, lasciando la possibilità di indirizzamento per pochi indirizzi di Classe B e alcuni di Classe C. Quello che si fece, fu tentare di recuperare questa situazione attraverso tre fasi:

- ✚ **Ripulitura:** verificare tutte le assegnazioni che venivano fatte in maniera doppia e pasticciona degli indirizzi;
- ✚ **Requisizione:** di tutte quelle classi C B A che non erano utilizzate o sottoutilizzate e riassegnazione in maniera più intelligente della risorsa (In Italia gestito dalla GARR);

- ✚ **Superfetazione:** l'impiego di sottoreti, in questo caso sottorete è utilizzato come frazionamento dell'indirizzo di rete.

INDIRIZZI IP SPECIALI

Tutte le assegnazioni degli indirizzi non seguono regole fisse, **esistono indirizzi IP speciali:**

0 0	Questo host
0 0 ... 0 0 Host	Un host su questa rete
1 1	Trasmissione broadcast sulla rete locale
rete 1 1 1 1 ... 1 1 1 1	Trasmissione broadcast su una rete remota
127 (Qualsiasi cosa)	Loopback

- 1) Tutti 0 rappresentano l'host stesso;
- 2) se metto a 0 tutto il net-id e 1 il resto indico un host su questa rete;
- 3) con tutti 1 si ha broadcast su una rete locale;
- 4) se ho una parte di net-id e poi tutti 1 indico una trasmissione broadcast su una rete remota,
- 5) tutti gli indirizzi espressi nella forma 127 sono riservati per effettuare controlli all'interno dell'host stesso (**loopback**). I pacchetti diretti a quell'indirizzo non sono immessi nel cavo, ma elaborati localmente e trattati come pacchetti in arrivo. Questo permette di inviare pacchetti all'host locale, senza che chi trasmette conosca il suo numero, cosa utile per effettuare dei test.

INDIRIZZI IP PRIVATI

Se io *devo lavorare solo all'interno della mia rete posso* utilizzare degli indirizzi **IP privati** che quindi non contrastino con quelli pubblici, i quali non vengono mai annunciati all'esterno. Ci sono 3 classi disponibili, ognuna ha diversi range di indirizzi che specificano diversi host configurabili:

10.0.0.0	10.255.255.255/8	(16.777.216)
172.16.0.0	172.31.255.255/12	(1.048.576)
192.168.0.0	192.168.255.255/16	(65.536)

- ✚ La classe 10.0.0.0, che è generosissima tanto che offre quasi 17 milioni di indirizzi a disposizione. Il Net ID è 10, tutti gli altri 3 valori li posso indirizzare da 0 a 255. Mi viene quindi un numero gigantesco di indirizzi possibili. Lo slash seguito da un numero, indica il **mascheramento** (la parte del Net ID che teniamo bloccata)
- ✚ 172.16.0.0 circa 1 mln di indirizzi;
- ✚ 192.168.0.0 ho gli ultimi due campi impiegabili, ho 16 bit di Net ID bloccato e 16 bit di Host ID disponibili.

Può capitare che io faccia una rete e la voglia indirizzare attraverso degli indirizzi IP privati, ma prima di garantire a questa rete una comunicazione attraverso l'internet, essendo gli host assegnati agli indirizzi privati, *dobbiamo convertire ogni nostro indirizzo privato in un indirizzo pubblico.*

SOTTORETI IP (SUBNETTING)

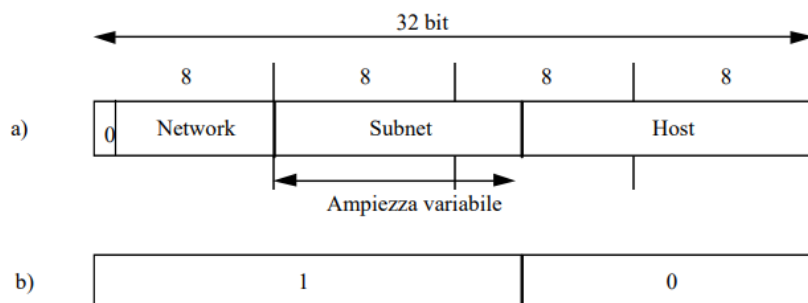
È possibile modificare, parzialmente, la struttura dell'indirizzamento utilizzando le sottoreti (subnet). Il campo definito come indirizzo locale host, può essere suddiviso arbitrariamente in due campi: **sottorete e host.**

Se io ho un certo numero di indirizzi net-id che mi sono stati annunciati, e dietro ne ho un sottoinsieme mio privato, io creo un **mascheramento** in modo tale che sfruttando quest'ultimo è come se creassi tante **sottoreti** all'interno. Per *decidere in modo più veloce la destinazione dei messaggi* (pacchetti) verso i computer della

rete, si usano le **subnet mask** (maschera di sottorete), per indicare ai dispositivi di instradamento (router) della rete quali numeri dell'indirizzo IP devono essere controllati e quali no.

La maschera di sottorete è un **indirizzo di 32 bit** (simile al numero IP) che consente di dividere la parte host da quella di rete; questa è fondamentale perché tramite essa, si riesce a determinare se un IP fa parte della rete o va ricercato in una rete remota. Gli **1** definiscono la porzione di indirizzo che identifica la sottorete, gli **0** definiscono la porzione di indirizzo che identifica l'host

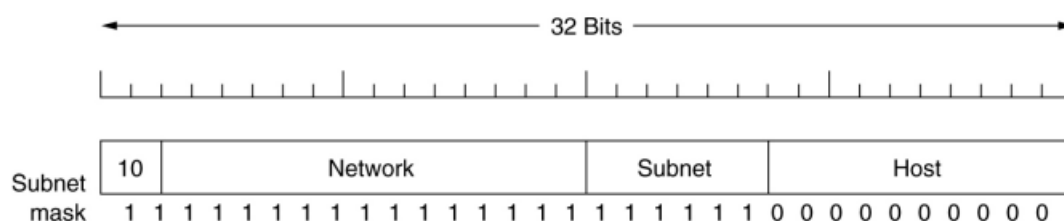
Questo sotto è l'esempio su una classe A operando subnetting sui 24 bit a disposizione:



Quindi, se io volessi suddividere in più reti questo unico indirizzo, quello che vado a fare è inserire una *sottorete visibile solamente all'interno*. L'annuncio dell'intera rete non sarà più di 8 bit ma 8 bit + altri (pari all'ampiezza variabile nello schema) e allora gestisco questi bit rimanenti come host id.

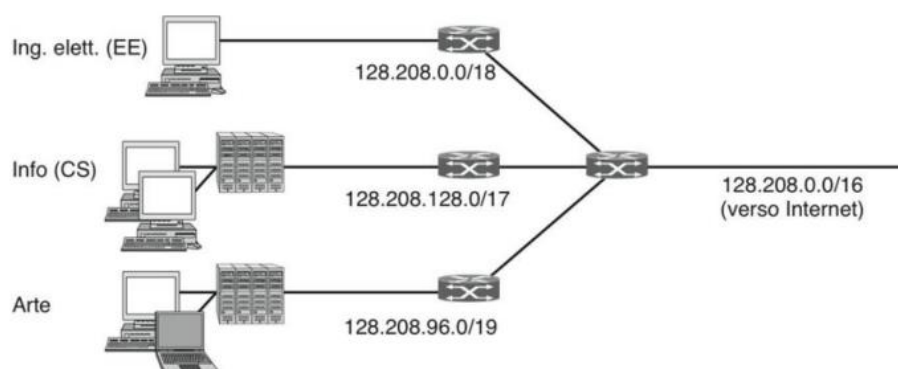
La suddivisione non è visibile all'esterno e, pertanto, per riconoscerla all'esterno basta una semplice comparazione dell'indirizzo e della maschera, applicando un'**AND** logico tra il numero IP e la sua maschera.

Una rete classe B divisa in 64 sottoreti



Negli indirizzi di classe B, ad esempio, si può pensare di suddividere i 16 bit di indirizzamento locale (host) in 12 bit dedicati all'indirizzamento delle sottoreti per un massimo di 4096 subnet, (2^{12}) ciascuna delle quali con massimo di 16 host. Questo esempio potrebbe riferirsi ad una catena di supermercati, caratterizzata da molte filiali, ciascuno dei quali in possesso di un numero di computer/casse non superiori a 16.

Suddividere una rete in sottoreti ci consente di *allocare in maniera efficiente gli indirizzi*, migliorando al tempo stesso le prestazioni (il traffico relativo ad una sottorete non viene introdotto nelle altre).



Sappiamo che l'ente **ICANN**, ma non solo questo, gestiscono gli indirizzi IP. Questo processo di assegnazione degli indirizzi però, può essere problematico man mano che una rete incrementa le sue dimensioni. Supponiamo che un'università (che parte con un prefisso /16), voglia utilizzare questa rete per connetterci i suoi host. Supponiamo che poi l'anno seguente, anche altri dipartimenti dell'università necessitino di una rete. L'utilizzo di un altro blocco di indirizzi IP assegnabili ai singoli dipartimenti è una soluzione possibile ma dispendiosa, quindi *mi conviene spezzare in sottoreti interne come nella figura sopra*, e nel passaggio verso internet viene effettuato un mascheramento con il quale annuncio un'unica rete.

Questa operazione è definita sempre definita **subnetting** e le reti che risultano dalla divisione di una rete più grande sono chiamate sottoreti. Blocco i primi 16 bit (/16) che ci danno il Net ID.

Per esempio, nel dipartimento di Arte ho un mascheramento maggiore, quindi ne utilizzo meno. Come si vede in figura, non è tanto importante che il mascheramento venga applicato in modo uniforme tra le parti, ma è tanto importante che tutti i bit nella parte più bassa degli host siano indirizzabili.

CIDR (CLASSLESS INTERDOMAIN ROUTING)

Anche se i blocchi di indirizzi IP vengono allocati in maniera efficiente rimane tuttavia un problema: *l'esplosione delle dimensioni delle tabelle di routing*.

I router di enti e aziende ai bordi di una rete, quali le università, hanno bisogno di una informazione per ognuna delle loro subnet; queste *informazioni indicano al router quale linea usare per andare verso una specifica rete*. Per percorsi la cui destinazione si trovi fuori dell'organizzazione dove opera il router essi possono semplicemente usare la regola standard di inviare i pacchetti sulla linea verso l'ISP che connette tale organizzazione al resto di Internet.

Gli ISP e le dorsali al centro di Internet non godono di questo lusso; devono conoscere come raggiungere ogni rete e nessuna regola standard funziona. Questi **core router** sono detti essere nella **zona default-free** (zona senza default) di Internet.

Nessuno sa veramente quante reti sono connesse a Internet, ma sono tante, probabilmente almeno 1 milione. Quindi *le tabelle di routing possono essere veramente molto grandi*. Potrebbero non sembrare così grandi per gli standard di un computer, ma si pensi che i router devono effettuare una ricerca in questa tabella per inoltrare ogni pacchetto e che quelli di grandi ISP possono inoltrare anche milioni di pacchetti al secondo. Con questi numeri sono necessari hardware specializzato e memoria veloce; un computer normale non è sufficiente. Inoltre, vari algoritmi di routing richiedono che *i router si scambino informazioni sugli indirizzi che possono raggiungere*. Più sono grandi le tabelle, più informazioni devono essere comunicate ed elaborate; l'aumento dei tempi di elaborazione cresce almeno linearmente con la dimensione della tabella. Più sono i dati da comunicare, più è probabile che alcune parti si perdano lungo la strada, generando instabilità.

Fortunatamente per *ridurre la dimensione delle tabelle di routing* si può fare qualcosa: possiamo applicare lo stesso meccanismo del **subnetting**; i router in posizioni differenti possono riconoscere un fissato indirizzo IP come appartenente a prefissi di dimensione diversa. Tuttavia, invece di suddividere un blocco di indirizzi in subnet, in questo caso si aggregano prefissi piccoli in un unico prefisso più grande. Questo processo è chiamato **route aggregation** (aggregazione di percorsi).

Il risultante prefisso, più grande, è chiamato a volte **supernet**, in contrasto con le subnet che sono invece una suddivisione dei blocchi di indirizzo. Con questo metodo di aggregazione gli indirizzi IP sono contenuti in prefissi di dimensione variabile. Lo stesso indirizzo IP che un router tratta come parte di uno /22 (un blocco contenente 210 indirizzi) può essere trattato da un altro router come parte di un più grande /20 (che contiene 212 indirizzi). Compito di ogni router è avere la corrispondente **informazione sul prefisso aggregato**. Questo schema opera con il subnetting ed è chiamato **CIDR** (Classless InterDomain Routing).

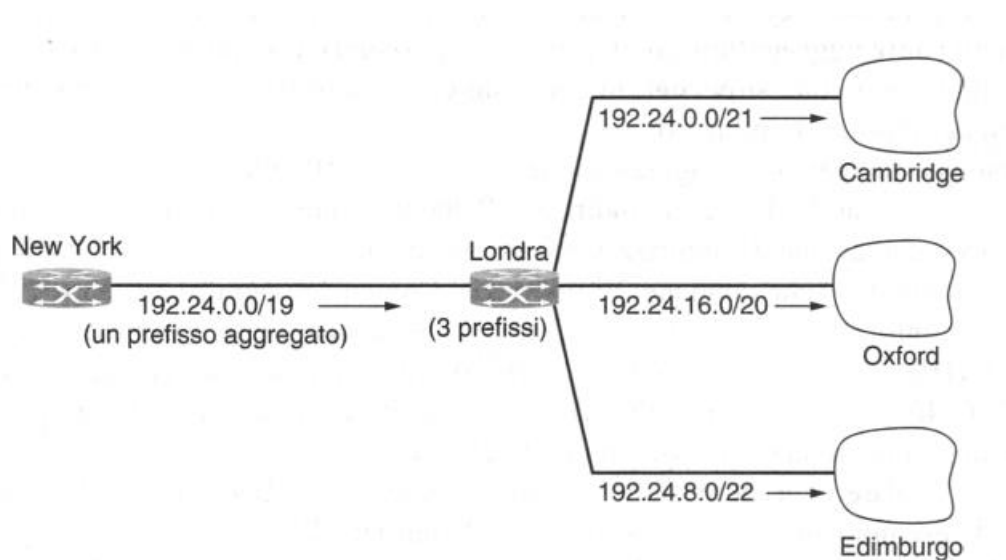
Università	Primo indirizzo	Ultimo indirizzo	Totale indirizzi	Prefisso
Cambridge	194.24.0.0	194.24.7.255	2.048	194.24.0.0/21
Edimburgo	194.24.8.0	194.24.11.255	1.024	194.24.8.0/22
(disponibili)	194.24.12.0	194.24.15.255	1.024	194.24.12.0/22
Oxford	194.24.16.0	194.24.31.255	4.096	194.24.16.0/20

Nella figura sopra sono rappresentate alcune Università inglesi, ogni università ha bisogno di un certo numero di indirizzi IP (totale indirizzi).

Se io assegnassi a Cambridge tutto 194.24.0.0 e come se gli dessi circa 65536 mila Host ID, ma ne basterebbero circa 2000. Allora prendo 192.24.0.0 e effettuo un mascheramento a 21 (/21), e ciò significa che il Net ID è pari a 21; avrò quindi 11 bit di Host ID e quindi 2048 indirizzi (2^{11}) che ricoprono le macchine richieste da Cambridge. Questo mi consente di ricoprire verso lo specifico il numero di macchine necessarie senza avere un eccesso di indirizzi non utilizzati.

Edimburgo ha bisogno di 1024, quindi effettuo un mascheramento a 22 ovvero 32-10 perché dai totali (32) sottraggo i necessari per coprire le macchine 10 ($2^{10}=1024$) (meno generoso). Oxford ha bisogno di 4096 macchine, quindi avrà una maschera di 20 bit (2^{12}).

La logica è che esiste un router in testa che fa questi mascheramenti, con questi riesco a sfruttare al massimo la potenzialità di assegnazione dei bit.



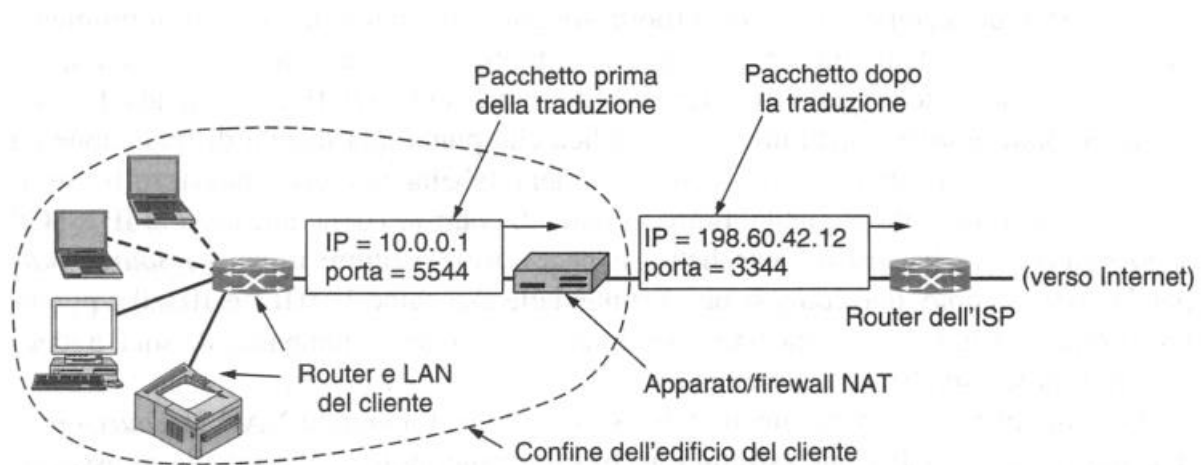
NETWORK ADDRESS TRANSLATION

La **NAT** (Network Address Translation) è una *tecnica irrispettosa dell'Internet Protocol*, perché prende parte dell'indirizzo di trasporto e lo usa come quello di rete.

Io ho un impiego di un indirizzo privato all'interno di una determinata LAN **10.0.0.1**. Questi fanno parte degli **indirizzi privati** di cui abbiamo parlato precedentemente, quindi all'interno della linea tratteggiata in figura abbiamo macchine private (facente quindi parte del "dominio" 10.0.0.0), le quali non sono rese note all'esterno. L'ultima macchina di confine ha indirizzo 10.0.0.1 e può interfacciarsi all'esterno tramite l'**indirizzo pubblico 198.60.42.12**, che può essere visto dall'intera rete Internet.

L'idea di base è proprio quella di *interfacciare un certo numero di macchine che hanno un loro sistema di indirizzamento privato tramite questo unico indirizzo pubblico* che mi fornisce il provider.

Il funzionamento di **NAT** è mostrato nella figura sottostante, dentro i locali dell'azienda, ogni macchina ha un unico indirizzo espresso nella forma **10.x.y.z**, ma quando un pacchetto lascia la rete locale, passa attraverso un apparato NAT che converte l'indirizzo IP interno (10.0.0.1 nella figura) nel vero indirizzo IP assegnato all'azienda (198.60.42.12 in questo esempio). Quando la risposta ritorna (per esempio da un Web server), i dati sono naturalmente indirizzati a 198.60.42.12; in che modo, allora, l'apparato NAT sceglie l'indirizzo di destinazione interno? È qui che si presenta il problema di NAT.



Se ci fosse un campo di riserva nell'intestazione IP lo si potrebbe utilizzare per tener traccia del trasmittente reale, ma è rimasto inutilizzato solo 1 bit. I progettisti di NAT hanno osservato che la *maggior parte dei pacchetti IP trasporta al suo interno TCP o UDP*, entrambi i protocolli hanno **intestazioni che contengono una porta sorgente e una porta di destinazione**. Le porte sono numeri interi lunghi 16 bit che indicano dove inizia e finisce la connessione TCP e offrono il campo che permette di far funzionare NAT.

Quando un processo desidera stabilire una connessione TCP con un processo remoto, si lega a una porta TCP inutilizzata sulla sua macchina; questa porta è chiamata **porta sorgente** e indica al codice di TCP dove devono essere inviati i pacchetti in arrivo appartenenti alla connessione. Il processo fornisce anche una porta di destinazione che indica chi deve ricevere i pacchetti sulla parte remota, non utilizzerà le porte da 0 a 1023 perché sono le porte con indirizzi di uso standard, le cosiddette **Well-known ports**.

Quindi si genera una tabella all'interno della LAN che crea una corrispondenza tra un **numero fittizio privato interno** (la porta sorgente) e un numero di porta di destinazione (porta TCP/UDP).

Quindi il NAT *nasconde nell'indirizzo a livello superiore l'indirizzo di rete che non potrebbe mettere perché è un indirizzo privato*. Permette dunque alle macchine di interfacciarsi come 198.60.42.12 all'esterno del "confine", e quando la risposta torna indietro dalla tabella interna decide per chi è quella determinata risposta. Dalla porta UDP si capisce quale sistema ha fatto la richiesta cosicché la risposta possa essere riconsegnata al computer che aveva inviato la richiesta.

Questo sistema prende una funzione che dovrebbe essere svolta al livello 3 e la alloca al livello 4 violando le precedenti norme descritte nei livelli (non è una cosa che si dovrebbe fare). **Viola quindi l'autonomia dei livelli**, che è una delle basi su cui progettare una rete.

Un altro aspetto è che NAT rompe il **modello di connettività end-to-end di Internet**, che afferma che ogni host può inviare un pacchetto in ogni momento a ogni altro host. Poiché l'associazione viene effettuata dai pacchetti in uscita, i pacchetti in entrata non possono essere accettati se non dopo quelli in uscita. In pratica questo significa che un utente domestico con una NAT può effettuare una connessione TCP/IP a un server Web remoto, ma un utente remoto non può connettersi a un server sulla rete domestica. Per supportare queste situazioni è necessario avere speciali configurazioni o tecniche di **NAT traversal** (attraversamento NAT).

Inoltre, NAT *trasforma Internet da una rete non orientata alla connessione in un tipo particolare di rete orientata alla connessione*, perché l'apparato NAT deve conservare le informazioni (le associazioni) relative a ogni connessione che lo attraversa. Il mantenimento dello stato della connessione è una proprietà delle

reti orientate alle connessioni. Se l'apparato NAT si blocca e la sua tabella di mappatura si perde, tutte le sue connessioni TCP vanno distrutte.

NAT *viola la principale regola della stratificazione dei protocolli*: il livello k non deve essere costretto a formulare alcuna ipotesi su ciò che il livello $k + 1$ ha inserito nel payload. Questo principio fondamentale serve a rendere i livelli indipendenti.

I *processi su Internet non sono obbligati a utilizzare TCP e UDP*. Se un utente sulla macchina A decidesse di utilizzare qualche nuovo protocollo di trasporto per parlare con un utente seduto davanti alla macchina B, l'introduzione di un apparato NAT non farebbe funzionare l'applicazione perché quest'ultimo non sarebbe in grado di individuare in modo corretto il campo Source port TCP.

Infine, *alcune applicazioni usano più connessioni TCP o porte UDP con modalità prefissate*. Per esempio, FTP (file transfer protocol), il protocollo standard di trasferimento file, inserisce indirizzi IP nel corpo del pacchetto dal quale il ricevente li estrae e li usa. Poiché NAT non conosce il formato non può gestirlo.

IPV6

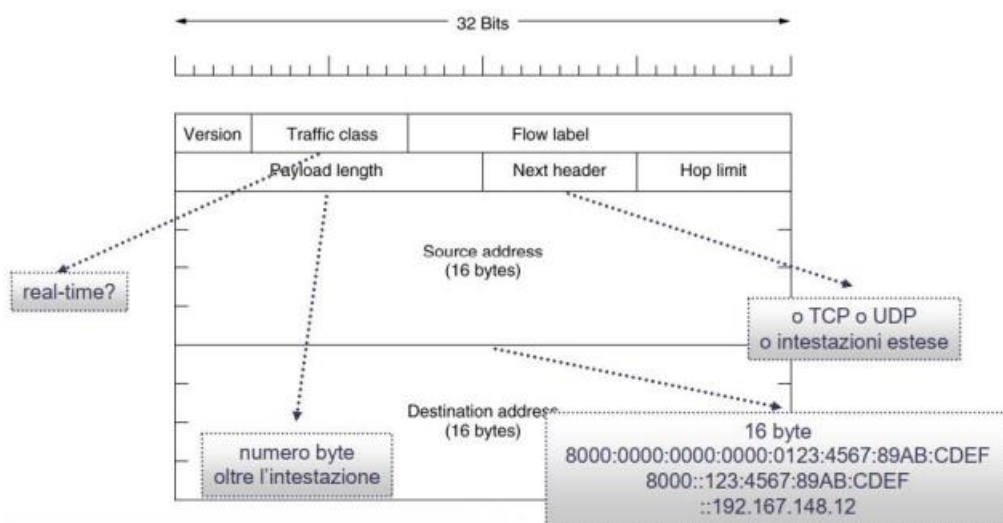
Dato che ci si aspetta che prima o poi gli indirizzi IPv4 termineranno, è stato necessario preparare in modo preventivo un nuovo indirizzamento IP con *indirizzi più grandi*, che è chiamato appunto **IPv6**. Gli indirizzi IPv6 sono a **128 bit** e per ora, non è ipotizzabile che questi indirizzi si saturino in un futuro prossimo. L'**intestazione** è un indirizzo a **32 bit** ma ha una struttura *esadecimale pari a 16 byte* (es indirizzo 8000:0000:...CDEF).

Per scrivere gli indirizzi a 16 byte è stata escogitata una nuova notazione. Gli indirizzi sono scritti come otto gruppi di quattro cifre esadecimali separate da due punti (8000:0000:0000:0000:0123:4567:89AB:CDEF). Poiché molti indirizzi possono contenere numerosi zero, sono state autorizzate tre ottimizzazioni.

Primo, *è possibile omettere gli zero iniziali di un gruppo*, perciò 0123 può essere scritto come 123. Secondo, uno o più gruppi contenenti 16 bit a zero possono essere sostituiti da due punti. Perciò, l'indirizzo precedente diventa: 8000::<123:4567:89AB:CDEF. Se l'indirizzo da cui parto è un vecchio indirizzo IPv4, può essere scritto in notazione decimale a punti dopo una coppia di due punti: ::192.167.148.11. Tutti gli *IPv4 possono essere annunciati quindi da una rete IPv6*.

Come per IPv4 abbiamo anche qui un'intestazione particolare (sopra gli indirizzi in figura che abbiamo già descritto), la quale è molto più semplificata perché contiene solo 7 campi rispetto ai 13 di IPv4:

IPv6 (SIPP Simple Internet Protocol Plus) RFC 2460-2474



🚩 **Version**: posto a 6 per indicare IPv6;

- ✚ **Traffic class:** specifica qual è la classe di traffico, è utilizzato per distinguere i pacchetti con diversi requisiti di distribuzione in tempo reale;
- ✚ **flow label:** specifica del tipo di trasmissione fatta che consente a una sorgente e ad una destinazione di marcare un gruppo di pacchetti che devono essere trattati nello stesso modo, formando una *pseudo connessione*. Ad esempio, un insieme di pacchetti che servono a svolgere un servizio da un host verso un altro potrebbero avere bisogno di una corsia preferenziale, e di un certo ordine di consegna. Un pacchetto con la Flow Label con valore diverso da zero, fa in modo che tutti i router cerchino l'etichetta nelle loro tabelle interne per scoprire il tipo di trattamento che questo pacchetto ha richiesto;
- ✚ **Payload length:** indica il numero di byte che segue l'intestazione minima di 40 byte. Non si chiama più total length come in IPv4 perché ora non si conteggiano più nella dimensione totale i byte di intestazione;
- ✚ **Next Header:** indica quale tra le sei intestazioni estese disponibili, sta utilizzando il pacchetto corrente. Se c'è una presenza di intestazione del livello sovrastante, ci dirà quale (se ad esempio a livello sovrastante c'è un protocollo TCP). Ci indica sostanzialmente cosa si attende la prossima intestazione (ad esempio un UDP, TCP, ecc);
- ✚ **Hop limit:** è il numero di salti massimo, utile per evitare che per un qualche tipo di errore un pacchetto rimanga in attesa per sempre. Funziona come il campo Time to Live di IPv4, e il valore è decrementato ad ogni salto. Formato da 8 bit, quindi un numero massimo di 256 che scala ad ogni salto compiuto.

Dato che i campi mancanti di IPv4 sono ancora saltuariamente necessari, IPv6 ha introdotto il concetto di **intestazione estesa** chiamate extension header. Queste possono essere usate per fornire informazioni aggiuntive codificate in modo efficiente. Alcune di queste hanno un formato fisso, altre hanno campi di lunghezza variabile. Al momento sono definite 6 intestazioni estese:

Intestazione estesa	Descrizione
Opzioni hop per hop	Informazioni varie per i router
Opzioni di destinazione	Informazioni aggiuntive relative alla destinazione
Routing	Lista di router da visitare
Frammentazione	Gestione dei frammenti di datagramma
Autenticazione	Verifica dell'identità del trasmittente
Carico utile cifrato	Informazioni relative al contenuto cifrato

Ogni estensione è opzionale, ma quando appaiono devono farlo dopo l'intestazione fissa e in questo preciso ordine:

- ✚ **Hop-by-hop options:** è utilizzata per le informazioni che tutti i router lungo il percorso devono esaminare. Finora è stata definita una sola opzione: supporto di datagrammi che superano i 64 KB. La Figura mostra il formato di questa intestazione.

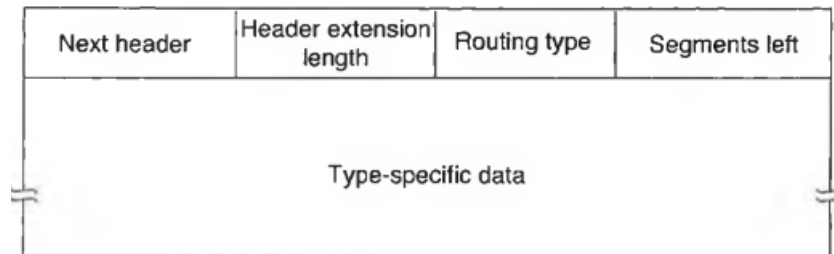
Intestazione successiva	0	194	4
Jumbo payload length			

Quando è utilizzata, il campo **jumbo payload length** nell'intestazione fissa è impostato a zero. Come nel caso di tutte le intestazioni estese, anche questa inizia con un byte che indica il tipo di intestazione che segue. Questo byte è seguito da un altro byte che indica la **lunghezza**, espressa in byte, dell'intestazione Hop-by-hop options, esclusi i primi 8 byte che sono obbligatori. Tutte le intestazioni iniziano in questo modo.

I successivi due byte indicano che questa opzione definisce la **dimensione del datagramma** (codice 194) e che la dimensione è un numero a **4 byte**. Gli ultimi 4 byte rappresentano la **dimensione del datagramma**. Dimensioni minori di 65.536 byte non sono consentite e, se utilizzate, costringono il primo router a scartare il pacchetto e a restituire un messaggio di errore ICMP.

I datagrammi che usano questa estensione di intestazione sono chiamati **jumbo grammi**. L'uso dei jumbo grammi è importante per applicazioni di supercomputer che devono trasferire gigabyte di dati attraverso Internet in modo efficiente;

- ✚ **Destination options** (opzioni di destinazione): è utilizzata per i campi che devono essere interpretati dall'host di destinazione. Nella versione originale di IPv6 le uniche opzioni definite sono opzioni nulle usate per riempire questa intestazione con multipli di 8 byte, perciò attualmente non viene utilizzata. È stata inclusa per essere certi che il software di routing e quello sull'host siano in grado di gestirla in caso qualcuno un giorno pensi a un'opzione di destinazione;
- ✚ **Routing**: elenca uno o più router che devono essere visitati lungo la strada che porta alla destinazione. È molto simile al *Loose source routing* di IPv4, in quanto tutti gli indirizzi elencati devono essere visitati nell'ordine indicato, ma in mezzo possono essere visitati anche altri router non elencati. Il formato di intestazione di Routing è quello della figura sottostante.



I primi **4 byte** dell'intestazione estesa di routing contengono quattro valori interi lunghi 1 byte. I campi **Next header** e **Header extension length** sono stati già descritti precedentemente.

Il campo **Routing type** (tipo di routing) indica il formato del resto dell'intestazione. Il tipo 0 indica che un gruppo specifico di 32 bit segue il primo, seguito da una serie di indirizzi IPv6. Altri tipi potranno essere inventati in futuro a seconda delle necessità. Infine, il campo **Segments left** (segmenti rimasti) tiene traccia del numero di indirizzi nell'elenco che non sono ancora stati visitati, ed è decrementato ogni volta che ne viene visitato uno. Quando il valore raggiunge lo 0, il pacchetto è lasciato libero e può seguire il percorso che desidera. Di solito a questo punto è talmente vicino alla destinazione che il percorso migliore appare ovvio;

- ✚ **Fragmentation**: si occupa delle informazioni di frammentazione in maniera simile alla gestione di IPv4. L'intestazione contiene l'identificatore di datagramma, la posizione del frammento all'interno del datagramma originale e un bit che indica se il frammento corrente è seguito da altri. In IPv6, a differenza di IPv4, *solo l'host sorgente può frammentare un pacchetto*; i router lungo il percorso non possono farlo.

Sebbene tale approccio rappresenti una rottura con la filosofia del passato, semplifica il lavoro dei router e rende più rapido il processo di routing. Come è stato spiegato precedentemente, se riceve un pacchetto troppo grande, il router scarta i dati e invia alla sorgente un pacchetto di risposta ICMP. Tale informazione permette all'host sorgente di spezzare il pacchetto in pezzi più piccoli, usando questa intestazione e tentando di nuovo la trasmissione;

- ✚ **Authentication**: fornisce un meccanismo che garantisce al ricevente di un pacchetto l'autenticità del mittente;
- ✚ **Carico utile cifrato**: permette di proteggere con la crittografia il contenuto del pacchetto in modo da renderlo leggibile solo ai destinatari desiderati. Queste intestazioni utilizzano allo scopo le tecniche crittografiche.

PROTOCOLLI DI CONTROLLO IN INTERNET

La rete Internet è molto complessa, sempre più grande, e per tenerla sotto controllo abbiamo bisogno di **protocolli di controllo**, che hanno diverse funzioni:

- ✚ **ICMP**: fa un controllo della rete, è una suite di protocolli che verificano la correttezza interna;
- ✚ **ARP e RARP**: risolvono la corrispondenza tra la scheda di livello 2, indirizzo IP e viceversa;
- ✚ **Bootstrap**: hanno all'interno un insieme di dati;
- ✚ **DHCP** (Dynamic Host Configuration Protocol): assegna un numero di indirizzi pubblici tra un numero di macchine che sia superiore al numero di macchine a disposizione, un'assegnazione di tipo dinamico.

INTERNET CONTROL MESSAGE PROTOCOL (ICMP)

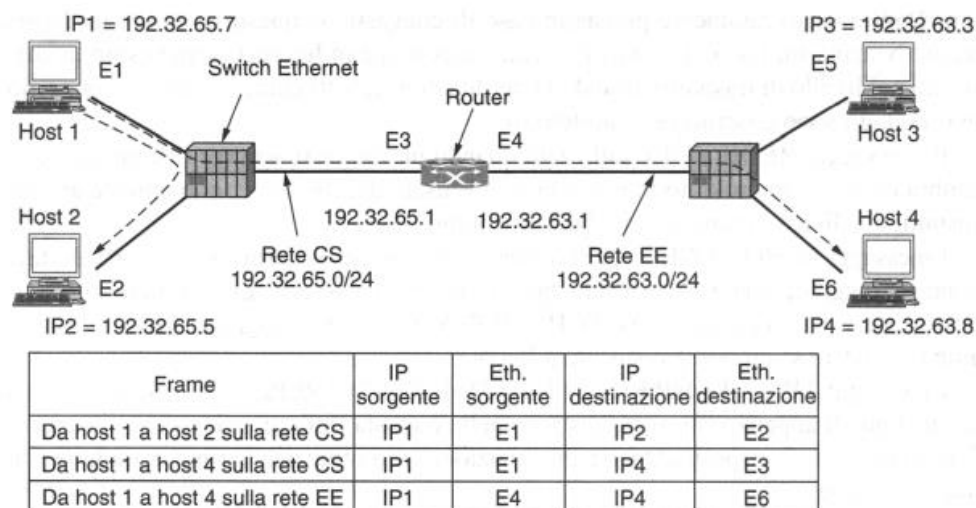
ICMP è un protocollo di controllo e di messaggio che ci consente di avere una serie di informazioni, ed è incapsulato nel pacchetto IP. Queste informazioni riguardano:

- ✚ **Destination unreachable:** è utilizzato quando il router non è in grado di individuare la destinazione. Viene inviato al mittente del pacchetto, ovvero abbiamo informazioni circa l'irraggiungibilità di una certa destinazione. Viene inviato al mittente del pacchetto;
- ✚ **Time exceeded:** è inviato quando il contatore del TimeToLive di un pacchetto è arrivato a zero, di conseguenza il pacchetto è stato scartato. Viene inviato al mittente del pacchetto;
- ✚ **Redirect** è utilizzato quando un router si accorge che un pacchetto sembra essere stato inoltrato in modo errato. È usato dal router per comunicare all'host trasmittente di aggiornare la sua tabella di routing;
- ✚ **Echo request, reply:** sono utilizzati per scoprire se una certa destinazione sia raggiungibile e attiva. Non appena riceve il messaggio **ECHO**, la destinazione deve rispondere con un messaggio **ECHO REPLY**.
- ✚ **Timestamp request, reply:** come il precedente, con in più la registrazione dell'istante di arrivo e partenza, per misurare le prestazioni della rete.
- ✚ **Router Advertisement, solicitation:** sono usati dagli host per trovare i router vicini. Un host deve conoscere l'indirizzo IP di almeno un router per uscire dalla rete locale;

ADDRESS RESOLUTION PROTOCOL (ARP)

Anche se ogni macchina di Internet ha uno o più indirizzi IP, in realtà questi non possono essere utilizzati per inviare pacchetti, in quanto le schede di rete, come le schede Ethernet che operano a livello data link, non comprendono gli indirizzi Internet. La domanda che sorge spontanea è: *in che modo gli indirizzi IP vengono associati agli indirizzi del livello data link (per esempio agli indirizzi Ethernet)?*

Per spiegare come funziona il meccanismo è utile l'esempio rappresentato nella figura sottostante, che mostra una piccola università contenente due reti di tipo /24.



Una rete (**CS**) è una Ethernet commutata e l'altra LAN (**EE**) è sempre una Ethernet commutata, quindi sono collegate ad uno switch ethernet, queste sono connesse fra di loro tramite un **router IP**; ogni macchina su Ethernet e ogni interfaccia del router hanno un unico indirizzo Ethernet, indicati da E1 a E6 e un indirizzo IP unico sulla rete CS o EE.

In che modo l'utente dell'host 1 invia un pacchetto all'utente 2 sulla rete CS?

- ✚ Prima bisogna scoprire *l'indirizzo IP di destinazione* (tramite DNS), il software del livello superiore sull'host 1 costruisce un pacchetto che riporta nel campo **Destination address** l'indirizzo IP appena

restituito dal DNS e lo passa al software che implementa IP per la trasmissione. Il software esamina l'indirizzo e scopre che la destinazione si trova sulla rete CS, la sua stessa.

✚ Poi bisogna *ottenere l'indirizzo Ethernet della macchina di cui si conosce solo l'IP*.

COME SI FA?

L'host 1 trasmette attraverso la Ethernet un pacchetto broadcast che chiede chi sia il proprietario dell'indirizzo IP con cui vogliamo comunicare, la trasmissione broadcast arriva a tutte le macchine della Ethernet CS e ognuna controlla il proprio indirizzo IP. Solo l'host 2 risponde inviando il proprio indirizzo Ethernet (E2). In questo modo l'host 1 scopre che l'indirizzo IP è assegnato all'host che ha l'indirizzo Ethernet E2.

Il protocollo utilizzato per fare questa domanda e ottenere una risposta si chiama **ARP** (address resolution protocol) e quasi tutte le macchine di Internet lo utilizzano. ARP è definito nell'RFC 826.

✚ A questo punto il software che si occupa di IP sull'host 1 *costruisce un frame Ethernet indirizzato a E2, inserisce il pacchetto IP nel payload e scarica il tutto sulla Ethernet*.

La scheda Ethernet dell'host 2 rileva il frame, si accorge di essere il destinatario della comunicazione, preleva i dati e genera un interrupt. Il driver Ethernet estrae il pacchetto IP dal payload e lo passa al software che verifica la correttezza dell'indirizzo ed elabora i dati.

Un altro esempio è questo:

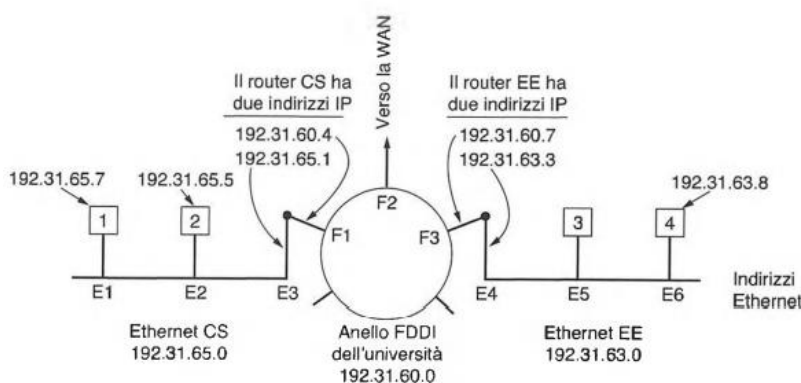


Figura 5.62. Tre reti /24 interconnesse: due Ethernet e un anello FDDI.

In questo caso le due ethernet sono collegate ad un **anello principale**, per esempio **FDDI** (Fiber distributed data interface) che ha un indirizzo IP.

Le CS e EE, inoltre sono **reti interconnesse**, quindi ogni host avrà un suo indirizzo ethernet, il quale è collegato al successivo che sarà dell'host 2 e così via, solo l'E3 è collegata direttamente all'anello.

Varie ottimizzazioni permettono ad ARP di funzionare in modo più efficiente. Tanto per cominciare, una volta eseguita l'operazione ARP, il computer memorizza in **cache** il risultato, caso mai dovesse essere necessario ricontattare lo stesso computer. La volta successiva, trovando l'informazione desiderata nella cache, il computer *non dovrà generare alcun messaggio broadcast*.

In molti casi l'host 2 potrebbe aver bisogno di inviare una richiesta per *determinare grazie ad ARP l'indirizzo Ethernet del mittente*; per evitare questa trasmissione ARP broadcast, è sufficiente che l'host 1 includa la propria associazione IP. Quando la trasmissione ARP raggiunge l'host 2, la coppia di valori del tipo (indirizzo IP, EI) viene inserita nella cache ARP dell'host 2 in previsione di un utilizzo futuro. In realtà, tutti i computer sulla Ethernet possono inserire queste associazioni nelle proprie cache ARP.

Per *permettere cambiamenti nelle associazioni*, per esempio quando un host è configurato per utilizzare un nuovo indirizzo IP, mantenendo quello Ethernet, le informazioni nella cache ARP **dovrebbero scadere in pochi minuti**. Un modo più intelligente per tenere le informazioni nella cache e ottimizzare le prestazioni è che *ogni computer trasmetta in broadcast la propria associazione durante l'accensione*. Questa trasmissione broadcast è generalmente eseguita sotto forma di una ricerca ARP mirata al proprio indirizzo IP. L'operazione, che non dovrebbe ricevere alcuna risposta, ottiene l'effetto di inserire una voce nella cache ARP di tutti gli altri computer. Questa operazione è chiamata **gratuitous ARP** (ARP gratuito).

Si osservi nuovamente la Figura 5.62, e si supponga che questa volta l'host 1 voglia inviare un pacchetto all'host 4. ARP non funziona perché *l'host 4 non riceve la trasmissione broadcast* (i router non inoltrano i pacchetti broadcast a livello Ethernet). Il problema può essere risolto in due modi.

- 1) Il router CS potrebbe essere configurato per rispondere alle richieste ARP della rete 192.31.63.0 (e possibilmente delle altre reti locali); se si adotta questa soluzione:
L'host 1 crea nella sua *cache ARP* una voce contenente la coppia di valori (192.31.63.8, E3) e trasmette al router CS tutto il traffico diretto all'host 4. Questa soluzione è chiamata **ARP proxy**.
- 2) Il secondo modo consiste nel fare in modo che l'host 1 si accorga immediatamente che la *destinazione si trova su una rete remota*; in questo caso tutto il traffico viene trasmesso a un indirizzo Ethernet predefinito (E3 nella figura) in grado di gestire il traffico remoto.
Questa soluzione non richiede che il router CS sappia quali sono le reti remote che sta servendo.

In entrambi i casi, ciò che accade è che *l'host 1 impacchetta il pacchetto IP nel campo carico utile di un frame Ethernet indirizzato a E3*. Quando riceve il frame Ethernet, il router CS rimuove il pacchetto IP dal carico utile e cerca l'indirizzo IP nelle sue tabelle di routing, scoprendo che i pacchetti per la rete 192.31.63.0 devono essere inviati al router 192.31.60.7.

Se non conosce già l'indirizzo FDDI di 192.31.60.7, il router *trasmette in modalità broadcast un pacchetto ARP nell'anello* e scopre che il suo indirizzo è F3. Quindi inserisce il pacchetto nel campo carico utile di un frame FDDI indirizzato a F3 e immette i dati nell'anello.

Sul router EE, il driver FDDI rimuove il pacchetto dal carico utile e lo passa al software IP, che scopre di dover inviare il pacchetto all'indirizzo 192.31.63.8. Se questo indirizzo non si trova nella sua cache ARP, il dispositivo trasmette in modalità broadcast una richiesta ARP attraverso la Ethernet EE e scopre che l'indirizzo di destinazione è E6, perciò costruisce un frame Ethernet indirizzato a E6, inserisce il pacchetto nel campo carico utile e invia i dati attraverso la Ethernet.

Quando il frame Ethernet raggiunge l'host 4, il pacchetto viene estratto dal frame e passato al software IP per l'elaborazione finale.

La trasmissione dall'host 1 verso una rete distante attraverso una **WAN** funziona essenzialmente nello stesso modo, tranne che questa volta le tabelle del router CS gli dicono di utilizzare il router WAN il cui indirizzo FDDI è F2.

REVERSE ADDRESS RESOLUTION PROTOCOL (RARP)

Nell'**RARP** viene fatto il processo inverso rispetto all'ARP, quindi: dato un indirizzo Ethernet, qual è il corrispondente indirizzo IP?

- ✚ Prima bisogna scoprire l'indirizzo Ethernet di destinazione avendo l'indirizzo IP;
- ✚ La richiesta inoltrata dalla macchina in possesso del suo solo indirizzo IP all'accensione riceve risposta dal Server RARP, si inviano le risposte per informare ogni macchina e creare la sua tabella;
- ✚ Il Server RARP, che utilizza una risposta broadcast (tutti 1), queste trasmissioni broadcast non sono inoltrate dai router, perciò è necessario installare un server RARP su ogni rete.

BOOT PROTOCOL (BOOTP)

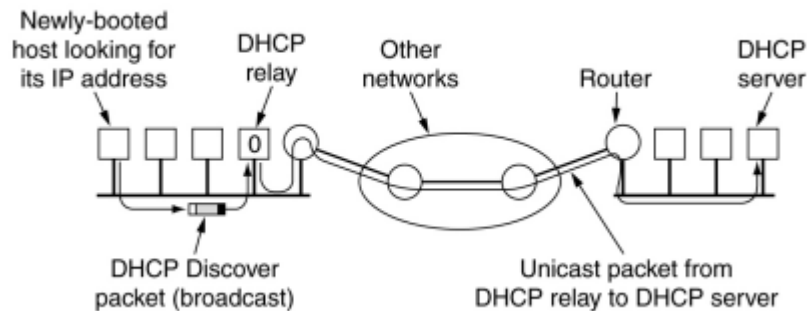
Per aggirare il problema del RARP è stato inventato un protocollo alternativo di bootstrap, chiamato **BOOTP**. A differenza di RARP, BOOTP *utilizza messaggi UDP*, che sono inoltrati attraverso i router. Non ha bisogno dell'ARP proxy e dunque non ha limiti dovuti all'impiego su una singola rete locale, quindi può direttamente "saltare" su una rete locale distante, e vien fatta al momento della partenza.

Un problema serio con BOOTP è che *richiede la configurazione manuale delle tabelle che associano gli indirizzi IP agli indirizzi Ethernet*. Quando si aggiunge alla LAN un nuovo host, non è possibile utilizzare BOOTP fino a quando l'amministratore non assegna alla macchina un indirizzo IP e non inserisce manualmente l'associazione del tipo (indirizzo Ethernet, indirizzo IP) nelle tabelle di configurazione di BOOTP.

DYNAMIC HOST CONTROL PROTOCOL (DHCP)

ARP, così come altri protocolli Internet, suppongono che gli host siano configurati in un certo modo e che abbiano alcune informazioni di base come gli indirizzi IP. **QUESTO COME AVVIENE?**

Il **DHCP** (Dynamic host configuration protocol) permette l'assegnazione manuale oppure automatica degli indirizzi IP pubblici.



Nell'esempio in figura, vediamo a sinistra 4 macchine più una macchina che fa da router e va verso la rete. Ho però **solo 2 indirizzi IP**. Il DHCP controlla quali sono le macchine vicine, vede chi sta chiedendo di uscire fuori per comunicare, e *assegna un indirizzo IP pubblico alla macchina*.

DHCP permette quindi che la macchina venga vista all'esterno, ma la macchina deve essere una **macchina attiva**. Se non è attiva le viene tolto l'indirizzo IP pubblico e viene assegnato a un'altra macchina. Fanno eccezione i DHCP con parametrizzazione statica (ad esempio un server del dipartimento), i server si terranno l'indirizzo pubblico assegnatogli, questo avviene in casi in cui ho necessità di non perdere dati, e ho quindi macchine sempre attive come i server appunto.

Un problema che si presenta quando si assegnano automaticamente indirizzi IP estratti da una riserva comune riguarda *la durata dell'allocazione*. Se un host abbandona la rete e non restituisce il proprio indirizzo IP al server DHCP, *quell'indirizzo andrà definitivamente perso*, perciò dopo un po' di tempo molti indirizzi IP potrebbero non essere più disponibili. Per impedire che ciò accada, l'assegnazione dell'indirizzo IP può essere fissata per un periodo di tempo, mediante una tecnica chiamata **leasing**. Poco prima della scadenza del leasing, l'host deve chiedere al server DHCP il rinnovo dell'indirizzo. Se non riesce a fare la richiesta o la richiesta viene rifiutata, l'host non può più utilizzare l'indirizzo IP che gli era stato assegnato precedentemente.

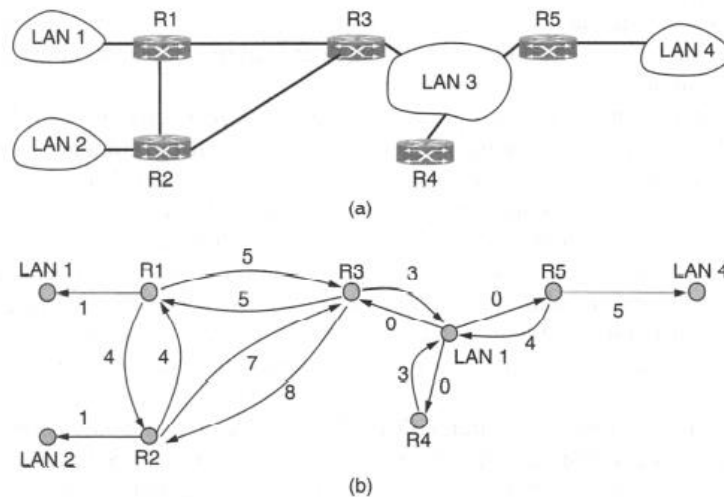
Anche qui abbiamo il caso di reti distanti che per consentire un uso ideale dell'DHCP hanno un **agente di inoltro DHCP**, cioè l'equivalente dell'ARP Proxy visto in precedenza.

PROTOCOLLI INTERNI ALLE RETI (IGP)

OSPF è un esempio di **IGP** (interior gateway protocol), funziona per collegare reti locali differenti attraverso collegamenti di router interni e dà vita a un modello a grafo, dove il collegamento tra due macchine ha valori di costo amministrativo differenti, questo è un tipo di interconnessione tra router che stanno all'interno di un **Autonomous System** (abbiamo visto che Internet è una rete composta da un gran numero di reti indipendenti o AS).

Possiamo ipotizzare che sia un AS 137 (quello del GARR che copre tutte l'Italia). Possiamo vedere la figura in basso come il sistema di comunicazione interno all'AS 137.

Un AS (Autonomous System) si può suddividere in più **Aree**. Un AS può avere un **Area dorsale** detta anche Area 0 (area centrale), oppure **Aree periferiche** (Area 1,2,3 e così via) **multiprotocollo**.



La maggior parte dei router nella figura è connessa ad altri router da collegamenti punto a punto e a reti di cui raggiungere gli host. Tuttavia, i router R3, R4 e R5 sono connessi da una LAN broadcast come una Ethernet commutata.

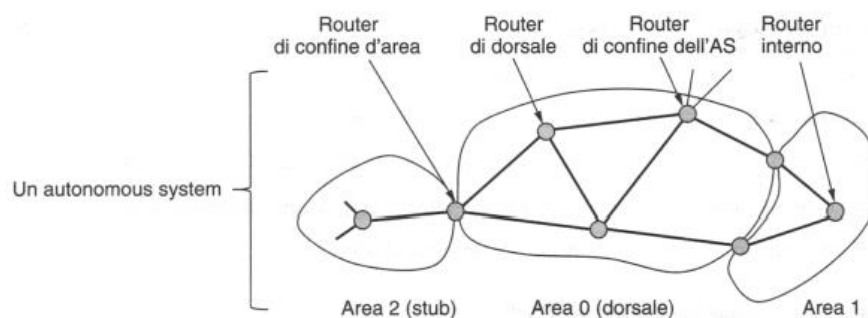
OSPF opera riassumendo l'insieme di reti reali, router e linee in un *grafo orientato* in cui a ogni arco è assegnato un **costo** (distanza, ritardo e così via), quindi calcola il *percorso più breve* in base al peso degli archi. Una connessione punto a punto tra due router è rappresentata da una coppia di archi, uno per ogni direzione; i loro pesi possono essere diversi.

Una rete broadcast è rappresentata da un nodo per la rete stessa, più un nodo per ogni router. Gli archi dal nodo di rete ai router hanno peso 0 e sono senza dubbio importanti, perché altrimenti non esisterebbero percorsi attraverso la rete. Altre reti, che hanno solo host, hanno solo archi che li raggiungono e non che partano da essi; questa struttura definisce percorsi verso gli host, ma non attraverso essi.

OSPF distingue dunque *quattro tipi di categorie di router*:

- 1) **Internal router**: è il router più vicino a noi (Ad es. quello al palazzo delle Scienze è l'internal router: in figura potrebbe essere il router interno ad Area 2 se noi ci troviamo lì);
- 2) **Border router** (router tra aree di uno stesso AS): un border router può essere il router di confine di uno stesso AS (router di confine d'area tra Area 2 e Area Dorsale in figura ad esempio);
- 3) **Backbone router** (router di dorsale): i router che stanno nella dorsale.
- 4) **Boundary router** (router di confine verso altri AS, che ha collegamenti che vanno verso altri AS): in Italia ne abbiamo uno a Roma e uno a Milano.

In Sardegna non ci sono boundary router, neanche backbone router, ma abbiamo border router e anche internal router su ogni edificio.



Area 2 (stub) è l'area finale, possiamo notare poi il border router (router di confine). I backbone router stanno nell'area dorsale. Il **border router** è l'unico che ha collegamenti verso un AS completamente diverso.

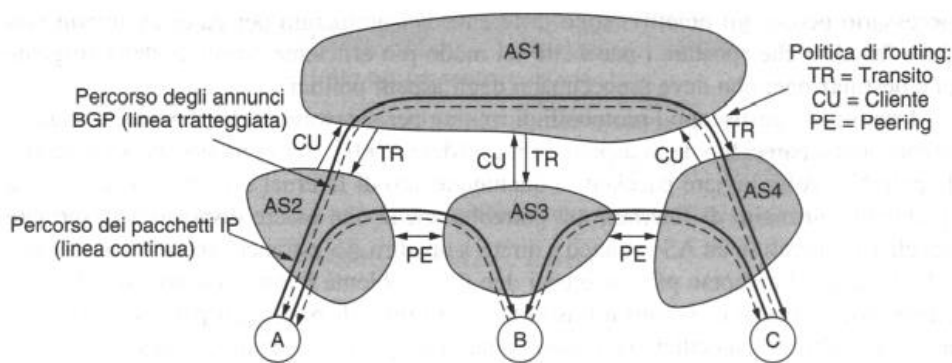
BGP - PROTOCOLLO DI ROUTING PER GATEWAY ESTERNI (EGP)

Dentro un singolo AS i protocolli di routing comunemente usati sono OSPF e IS-IS. Tra i vari AS si utilizza un protocollo diverso chiamato **BGP** (border gateway protocol), necessario perché gli obiettivi sono differenti. Un protocollo per gateway interni non deve far altro che spostare i pacchetti nel modo più efficiente possibile dalla sorgente alla destinazione; non deve preoccuparsi degli aspetti politici. I router che gestiscono i protocolli di routing per gateway esterni *devono, di contro, preoccuparsi parecchio degli aspetti politici*.

Dato che bisogna tener conto degli aspetti politici si ha la necessità di creare delle politiche di routing che rispettino i vari vincoli imposti, una politica di routing viene implementata decidendo quale traffico può fluire su quali linee tra AS. Una politica comune consiste nel fatto che un *cliente di un ISP paghi un altro ISP per consegnare pacchetti a qualunque altra destinazione di Internet* e ricevere i pacchetti inviati da qualunque altra destinazione. Si dice che l'ISP cliente compra un **servizio di transito** (transit service) da un ISP fornitore. È esattamente come quando un cliente domestico compra un servizio di accesso Internet da un ISP.

Perché funzioni, il provider dovrebbe diffondere i percorsi per tutte le destinazioni di Internet al cliente sulle linee che li collegano. In questo modo il cliente avrà un percorso da usare per spedire pacchetti ovunque voglia. Dall'altra parte, il cliente dovrebbe diffondere i percorsi solo delle destinazioni sulla propria rete al provider. Questo permetterebbe al fornitore di inviare traffico al cliente solo per questi indirizzi; *il cliente non vuole dover gestire traffico per altre destinazioni*.

Gli EGP quindi collegano i vari AS con politiche di transito (se io devo passare solo da una parte all'altra TR), oppure client (sono protocolli che consentono il collegamento solo di macchine cliente remoto) o peering se i collegamenti sono equipollenti.



Un esempio di **traffico** è questo che troviamo a sinistra, Ci sono quattro AS connessi.

La connessione spesso viene effettuata tramite una linea a un **IXP** (internet exchange point) a cui molti ISP si collegano per collegarsi ad altri ISP.

AS2, AS3 e AS4 sono clienti di AS1 perché comprano un servizio di transito da esso. Quindi, quando la sorgente A trasmette alla destinazione C, i pacchetti viaggiano da AS2 ad AS1 e finalmente arrivano ad AS4. *La diffusione del routing viaggia in direzione opposta ai pacchetti*. AS4 avverte C come destinazione al suo provider di transito, AS1, di lasciare che le sorgenti raggiungano C attraverso AS1. Dopo AS1 diffonde un percorso verso C a tutti gli altri clienti, incluso AS2, perché i clienti sappiano che possono inviare traffico a C attraverso AS1.

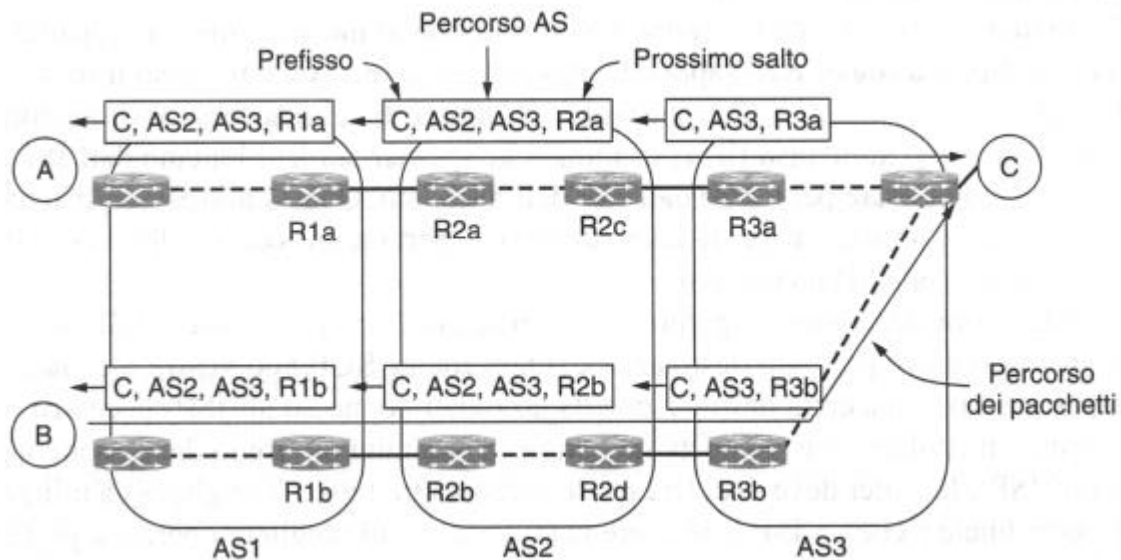
Nella Figura sopra *tutti gli altri AS comprano un servizio di transito da AS1*. Questo fornisce **connettività** in modo da poter interagire con qualunque host di Internet. Tuttavia per avere questo privilegio devono pagare.

Si supponga che AS2 e AS3 si scambino molto traffico. Dato che le loro reti sono già connesse, se lo vogliono, possono usare una differente politica: *possono scambiarsi traffico direttamente tra di loro senza pagare*. Questa procedura riduce la quantità di traffico che AS1 consegna al loro posto e si spera riduca il loro conto da pagare. Questa politica è chiamata **peering**.

Per implementare il peering due AS *si scambiano le tabelle di routing* per gli indirizzi che stanno nelle loro reti. Ciò permette ad AS2 di inviare pacchetti di AS3 da A a B e viceversa. Tuttavia si noti **che il peering non è transitivo**.

BGP è un protocollo di tipo **distance vector**, ma è abbastanza diverso dai protocolli interdominio basati anch'essi su distance vector. Una grande differenza è che i router BGP, invece di tenere traccia solo del costo di un cammino verso ogni destinazione, *memorizzano tutto il cammino usato*. Questo approccio è

chiamato **path vector protocol** (protocollo a vettore di percorso). Il cammino consiste nel router del prossimo salto (che potrebbe trovarsi dall'altra parte dell'ISP, non adiacente) e la sequenza di AS, chiamata anche **AS path**, che il percorso ha seguito (data in ordine inverso). Infine coppie di router BGP comunicano tra di loro stabilendo **connessioni TCP**. Questo modo di operare fornisce comunicazioni affidabili e nasconde tutti i dettagli della rete che si attraversa.



Questo è un esempio applicativo di quello che abbiamo visto precedentemente, dove abbiamo un treno di comunicazione dei pacchetti dove è riportato il percorso e il tipo di salto successivo.

INTERNET MULTI-CASTING

Sono degli indirizzi speciali in cui ogni indirizzo di classe D ha *28 bit di identificazione gruppi*:

- ✚ 224.0.0.1 dispositivo di una stessa LAN;
- ✚ 224.0.0.2 router di una stessa LAN
- ✚ 224.0.0.5 router OSPF su una LAN;
- ✚ 224.0.0.6 router OSPF specificati su una stessa LAN;
- ✚ 224.0.0.251 server DNS specificati su una stessa LAN.

L'**IGMP** (Internet Group Management Protocol) è un protocollo legato al multi-casting che permette di eseguire ulteriori *controlli interni*. Il multi-casting usa algoritmi Spanning tree con protocollo **PIM** (Protocol Independent Multicast).

IL LIVELLO DI TRASPORTO

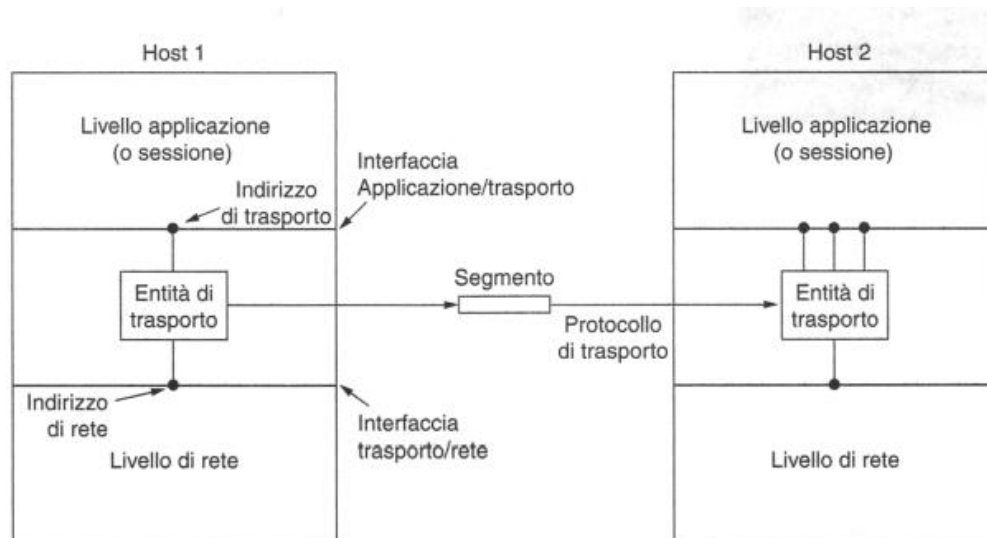
Il livello di Trasporto è l'ultimo che governa le sessioni effettive tra due host che comunicano tra di loro, nel TCP/IP oltre abbiamo il livello applicativo, se fossimo nell'ISO/OSI sopra il livello di trasporto avremmo anche il livello di sessione, presentazione e applicazione.

Parte delle funzioni del modello ISO/OSI dei livelli superiori sono contenute nel livello di trasporto TCP/IP.

Il livello di trasporto si basa sul livello di rete per fornire il trasporto dei dati da un processo su una macchina sorgente a un processo su una macchina destinazione con un livello di affidabilità desiderato e indipendente dalle reti fisiche attualmente in uso.

È costituito da **servizi orientati e non orientati alla connessione**, i dati prendono il nome di **Segmento** (TPDU) e abbiamo una connessione **END-TO-END** (l'ultima volta che due host distanti colloquiano per una stessa sessione, la possibilità di uno di trasmettere o dell'altro è in capo proprio a livello di trasporto).

L'hardware e/o il software all'interno del livello di trasporto che svolge il lavoro è chiamato **entità di trasporto** (transport entity). L'entità di trasporto può essere situata nel kernel del sistema operativo, in un processo utente separato, in librerie associate alle applicazioni di rete o addirittura nella scheda di rete.



Ogni servizio che viene richiesto a livello superiore non ha un NSAP ma una **TSAP** e questa risponde a un numero di porta (da 0 a 65536, di cui le più famose sono le *well-know port*).

Un elemento chiave che fa capire l'utilità del livello di trasporto sta nel fatto *che il codice eseguito nel livello di trasporto, viene eseguito sulla macchina*, mentre tutta l'architettura dal livello di rete fino al livello fisico, dipende dalla rete (router, linee e tutti gli elementi propri dei provider, non dipendono dalla macchina su cui stiamo cercando di gestire la comunicazione).

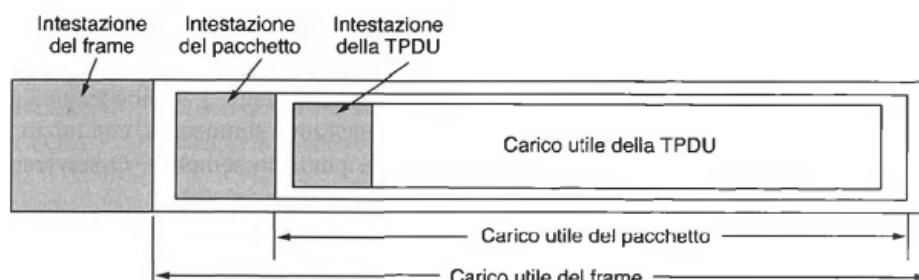
In un mondo in cui non esistono errori, il livello di trasporto sarebbe ridondante in quanto svolge per la maggior parte delle funzioni che il livello di rete potrebbe assolvere, ma dato che *in caso di errore da parte dei router non possiamo gestire la situazione dalla macchina*, è utile avere questa ulteriore stratificazione tra livelli.

L'intera struttura del trasporto è retta da dei servizi con delle **primitive**: permettono agli utenti di accedere al servizio. Presentano servizi simili ai servizi di rete, e normalmente si utilizzano dei modelli di servizio in genere **affidabile**, nonostante anche la forma di servizio datagramma sia comunque possibile. Come vediamo dalla figura sopra, un'entità di trasporto trasferisce il segmento prodotto dal livello applicativo, una volta passato tramite l'interfaccia applicazione/trasporto al livello di trasporto, al suo pari (cioè nell'entità di trasporto equivalente nell'host 2). Tra l'entità di trasporto e il livello di rete come sappiamo c'è un **N-SAP**, ovvero un indirizzo di rete per noi, mentre il SAP superiore è un T-SAP che definiremo più avanti come porta.

Queste primitive sono simili ai servizi di rete e in genere garantiscono un servizio affidabile. Generalmente il livello di trasporto si basa su scambi di informazioni tra due macchine. Questi host, per poter fare bene il loro lavoro devono adattarsi ad un insieme di servizi messi a disposizione che sono codificati dentro queste primitive sopra. Queste fanno da ambasciatori tra due host per definirne il comportamento. Vediamone alcune:

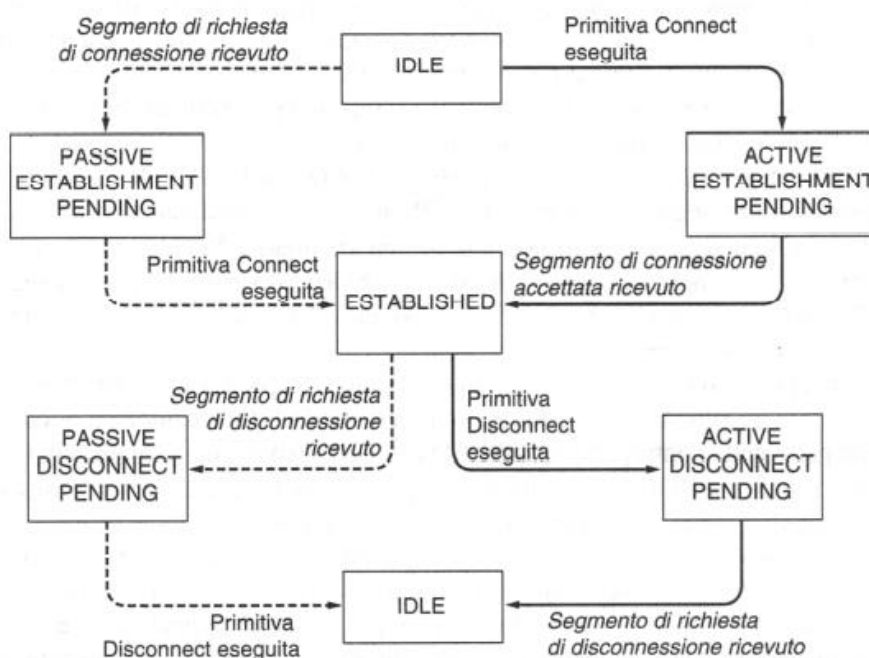
- ✚ La **LISTEN** è la primitiva d'ascolto. Questa blocca il server fino a quando un processo cerca di connettersi su una porta specifica, invocando uno specifico TSAP;
- ✚ Una **CONNECT** è la primitiva che cerca di instaurare la connessione, e il suo pacchetto correlato è una *connection request*;
- ✚ Gli scambi che arrivano attraverso un invio data (**SEND**) e una ricevuta che può dare una conferma di ciò che è avvenuto (**RECEIVE**). Gli scambi avvengono tramite un invio (**Data**) e una ricevuta.
- ✚ La **DISCONNECT** rilascia la connessione con una *disconnection request*.

Il livello di trasporto è in grado con il suo **TPDU** (Transport Protocol Data Unit, unità dati del protocollo di trasporto) di inserire all'interno un insieme di pacchetti come hanno fatto i livelli sottostanti, *fa un'ulteriore operazione di imbustamento* e quindi vediamo come partendo da un'intestazione iniziale abbiamo **un'intestazione di pacchetto e di trama**, dunque quando viene spaccettato questo insieme, arriva sino al livello di trasporto, viene aperto e viene trasferito il dato (il payload qui è veramente il dato, perché nel pacchetto e nella trama abbiamo contenute anche le intestazioni: nel payload del frame abbiamo sia l'intestazione del pacchetto che del segmento, mentre nel payload del pacchetto abbiamo contenuta solo l'intestazione del segmento).



Quindi, è come se avessimo una **macchina a stati finiti**. Una macchina a stati finiti definisce delle posizioni obbligate raggiunte o non raggiunte attraverso azioni specifiche; sulla destra abbiamo una primitiva di connessione, lo stabilimento della connessione, e dopo la disconnessione ritorniamo allo stato di IDLE.

Le *transizioni in corsivo* sono provocate dall'arrivo dei pacchetti. Le *linee continue* mostrano la sequenza di stato del client; le *linee tratteggiate* mostrano la sequenza di stato del server:



Nel livello di trasporto è importante capire quali sono le correlazioni tra due host che comunicano tra di loro, due configurazioni principali:

- 1) Capire se è un'architettura di **peering**, allora avremo uno scambio equilibrato. Dobbiamo capire se le macchine funzionano in maniera paritetica.
- 2) Capire se l'architettura è basata su due macchine che hanno funzionalità diverse (**ClientServer**), che poi è il caso di gran lunga più frequente.

Questo governo nel TCP/IP è basato sulle primitive che chiamiamo Socket. Sono primitive base affianco nella figura la C indica che è di tipo Client la S di tipo Server.

SOCKET BERKLEY

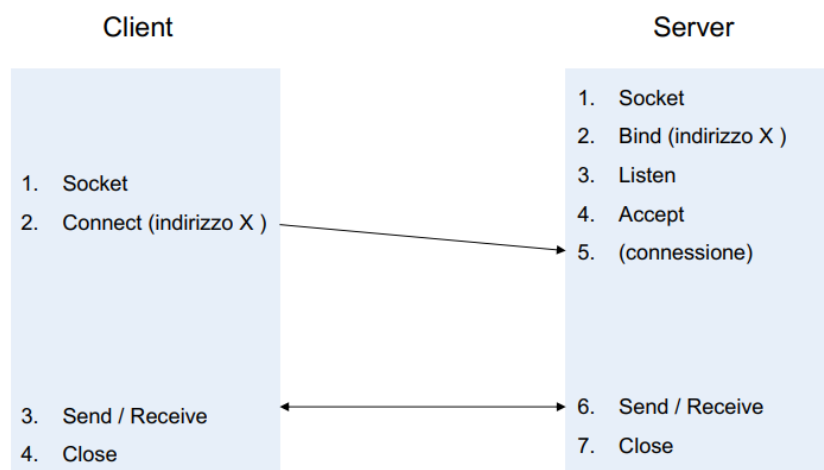
Una **socket** è una rappresentazione astratta di una comunicazione tra processi locali e processi remoti, attraverso questi un'applicazione può ricevere ed inviare dati.

In berkeley un socket è rappresentato da un file descriptor, che fornisce un'interfaccia comune per l'I/O dei dati. Sono un insieme di **primitive di trasporto** usate per la programmazione Internet, tali primitive sono utilizzate per TCP (Transmission Control Protocol) in Berkeley UNIX. Offrono maggiore flessibilità rispetto all'esempio precedente pur essendo simili.

Le primitive sono:

- ✚ **SOCKET ***: crea un nuovo punto finale di comunicazione, quindi, alloca le risorse per creare il punto di accesso;
- ✚ **BIND***: associa un indirizzo locale ad un socket e costruisce questo punto di accesso unico visibile dalla rete per accedere al processo ed i suoi servizi;
- ✚ **LISTEN**: annuncia la capacità di accettare connessioni; dà la dimensione della coda;
- ✚ **ACCEPT***: blocca il chiamante fino all'arrivo di un tentativo di connessione;
- ✚ **CONNECT**: tenta in modo attivo di stabilire una connessione;
- ✚ **SEND**: invia alcuni dati sulla connessione;
- ✚ **RECEIVE**: riceve alcuni dati sulla connessione;
- ✚ **CLOSE**: rilascia la connessione.

L'asterisco indica che sono primitive che nell'esempio prima non c'erano.



Il processo server inizierà a inizializzare il suo **socket** associandoli un indirizzo visibile dalla rete ed una porta, in modo da ricevere le connessioni. una volta che è pronto, il server è bloccato nell'attesa che qualcuno lo chiami dalla rete.

Parallelamente li client esegue una **Socket** ma non è necessario che il client esegua la *bind*, la *listen* e l'*accept* perché il client non mette a disposizione dei servizi e può seguire subito una **Connect** allo stesso indirizzo che è stato reso disponibile come punto di accesso al servizio da parte del server.

Sostanzialmente un indirizzo deve essere noto al client, a questo punto il protocollo TCP/IP invierà una richiesta di connessione e saranno disponibili le procedure **send** e **receive** per scambiare i dati infine avremo la **CLOSE** per chiudere la connessione.

La primitiva **SOCKET** crea un punto finale nella comunicazione, il socket viene identificato da un numero intero. È usata sia dal server che dal client. I suoi parametri di input specificano:

- ✚ *Formato di indirizzamento da utilizzare (es: 256 byte);*
- ✚ *Tipo di servizio desiderato (es: flusso affidabile);*
- ✚ *Protocollo da utilizzare (TCP? UDP?)*

I risultati di output sono un descrittore di file da usare per le chiamate successive.

La primitiva **BIND** è una chiamata utilizzata prettamente lato server. Quando si usa la chiamata SOCKET, ai punti finali creati non viene assegnato un indirizzo ma solo un descrittore. La chiamata BIND permette di assegnare un indirizzo di rete ed un'eventuale porta. I client possono connettersi ad un server solo dopo che quest'ultimo ha eseguito la BIND, il client non la usa perché il suo indirizzo non interessa al server.

La primitiva non bloccante **LISTEN** (usata solo lato server) alloca spazio in memoria per le future richieste in ingresso nel caso più client si connettano contemporaneamente. A differenza dell'esempio base, LISTEN non è bloccante.

La primitiva bloccante **ACCEPT** (usata solo lato server) permette di accettare le connessioni in ingresso, ricevuta una connessione, l'entità di trasporto crea un nuovo socket con le stesse proprietà dell'originale restituendo un nuovo descrittore. Il server a questo punto genera un processo o un thread per gestire la connessione sul nuovo socket. Il server torna al socket originale per verificare se ci sono altre connessioni in ingresso.

La primitiva **CONNECT** è utilizzata dal client, e assegna una porta libera al socket in input, permette di instaurare la connessione, è bloccante per chi la chiama ma sbloccante per chi la riceve.

Le primitive **SEND** e **RECEIVE** vengono usate per scambiare informazioni da entrambi i lati.

La primitiva **CLOSE** permette di rilasciare la connessione.

ELEMENTI DEI PROTOCOLLI DI TRASPORTO E INDIRIZZAMENTO

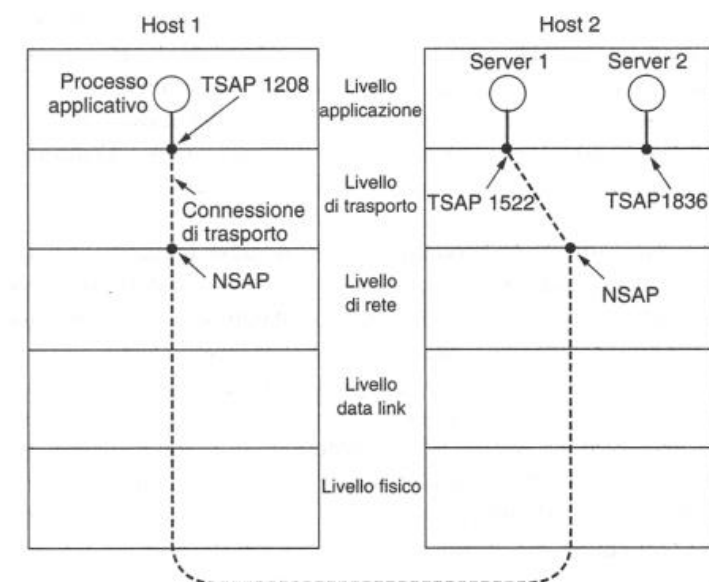
Il servizio di trasporto è implementato da un **protocollo di trasporto** utilizzato tra le due entità di trasporto. Per certi aspetti i protocolli di trasporto ricordano i protocolli data link, entrambi hanno a che fare, tra le altre cose, con il controllo degli errori, l'ordinamento e il controllo di flusso.

Quando un processo applicativo vuole creare una connessione verso un processo applicativo remoto, *deve specificare a quale intende connettersi*. Il trasporto non orientato alla connessione presenta lo stesso problema: a chi bisogna inviare ogni singolo messaggio? Il metodo normalmente utilizzato consiste nel definire indirizzi di trasporto su cui i processi possono restare in ascolto delle richieste di connessione. Su Internet questi endpoint (o punti terminali) sono chiamati **porte** (pori).

Quindi quando ho a che fare con un tipo di colloquio continuativo, devo individuare come viene fatto. Questo colloquio avviene tramite uno specifico endpoint, i **T-SAP** (Transport Service Access Point). I TSAP (detti anche porte) definiscono il punto d'accesso verso il livello superiore. Sono l'equivalente dell'N-SAP che però è tipico del livello 2/3. Il TSAP è una porta tipicamente di lunghezza di **16 bit** che individua, al variare del suo valore, *specifiche di servizio offerte diverse*. Un possibile scenario per una connessione di trasporto è il seguente:

Questo rappresentato è quello canonico (modello classico):

Abbiamo un host 1 che ha un processo applicativo che richiede attraverso un TSAP di avere l'ora esatta (*porta 1522*), le porte sono messe a disposizione come TSAP all'interno della macchina che offre il servizio (vediamo 1522, 1836 etc..).



Ad ognuno di questi (TSAP 1522 e TSAP 1836) corrisponde un **processo server attivo**.

L'host 1 attiva una socket, aggancia una richiesta dal suo TSAP (in questo caso 1208), la porta all'NSAP e la trasferisce fino ad arrivare all'altro NSAP dove viene spaccchettato e arriva la richiesta dell'ora (1522), NSAP interroga, trova la porta 1522 la aggancia al server in funzione (Server 1) e *stabilisce la connessione inviando i dati tramite la primitiva SEND*.

I TSAP possibili (porte) sono costituiti da 16 bit, quindi dovrei avere sempre attivi sui server **65536 server attivi**, uno spreco. Anziché mantenere tutti i server attivi verso la macchina invocata, si utilizza un modello diverso:

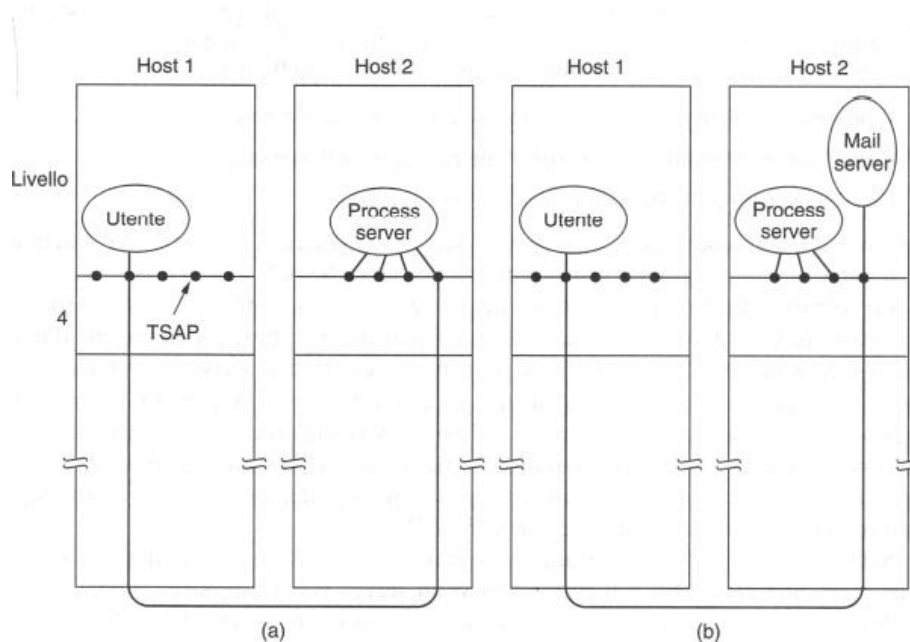
MODELLO PROCESS SERVER

Invece di avere tutti i possibili server in ascolto su un TSAP noto, ogni macchina che vuole offrire servizi ad utenti remoti ha un *process server che agisce da proxy per i servizi meno utilizzati*. Questo server, chiamato **inetd**, ascolta contemporaneamente su un insieme di porte aspettando una richiesta di connessione, questo protocollo è noto come **protocollo di connessione iniziale**.

I potenziali utenti di un servizio iniziano facendo una richiesta con la primitiva **CONNECT** specificando l'indirizzo TSAP del servizio che vogliono e ottenendo una connessione al process server. Dopo aver ricevuto la richiesta, il process server genera il server richiesto da host 1, permettendogli di ereditare la connessione esistente con l'utente. Il nuovo server svolgerà il lavoro richiesto dunque.

Una volta che arrivo su NSAP questo interroga TSAP, ma TSAP non risponde con il suo server attivo, *ma invoca un process server attivo su più porte*, legge la richiesta, attiva qual è il servizio (in questo caso l'ora del giorno) e aggancia quel servizio.

Da un punto di vista elaborativo, *abbiamo la possibilità di sganciare il momento di risposta e l'attivazione del server di risposta dalla necessità di dover avere tutti i server attivi* (capacità elaborativa, risparmio di energia, efficienza macchina). Perdiamo qualche frazione di secondo in questo passaggio, ma è un danno calcolato e ne vale la pena rispetto all'efficienza del sistema che segue questo modello. Questo modello si applica se è possibile creare server su richiesta.



MODELLO DIRECTORY SERVER

Il protocollo di connessione iniziale funziona bene per i server che possono essere creati in caso di necessità, ma si verificano spesso situazioni *dove i servizi esistono indipendentemente dal process server*. Un file server, per esempio, deve essere eseguito su hardware speciale (una macchina con un disco) e non può essere creato al volo quando qualcuno desidera comunicare con esso.

Per gestire questa situazione, viene spesso utilizzato uno schema alternativo. In questo modello esiste un processo speciale chiamato **server dei nomi** (name server) o a volte **directory server**. Per trovare l'indirizzo TSAP corrispondente a un dato nome di servizio, come "ora esatta", l'utente apre una connessione con il server dei nomi (che ascolta uno TSAP noto). L'utente invia poi un messaggio che specifica il nome del servizio, mentre il server dei nomi risponde con l'indirizzo TSAP.

In questo modello un *nuovo servizio appena creato si deve registrare sul server dei nomi*, presentando il nome del servizio (generalmente una stringa ASCII) e uno TSAP. Il server dei nomi registra queste informazioni nel suo database interno, per poter rispondere alle successive interrogazioni.

ATTIVAZIONE DI UNA CONNESSIONE

Sembra facile stabilire una connessione, ma in realtà l'operazione è sorprendentemente complessa. A prima vista, sembra sufficiente che un'entità di trasporto invii una TPDU CONNECTION REQUEST alla destinazione e attenda una risposta CONNECTION. Il problema si verifica *se la rete può perdere, memorizzare e duplicare i pacchetti*. Questo comportamento provoca serie complicazioni.

Quando abbiamo parlato del livello rete abbiamo detto che vi era un problema di gestione del tempo di vita del pacchetto, in quanto bisogna trovare *un modo per distruggere i pacchetti obsoleti che sono ancora in circolo*. Come possiamo essere certi che un pacchetto non sia morto? Con il Link State Routing avevamo messo un numero di sequenza e un'età per ciascun pacchetto. Il problema non si porrebbe se:

- 1) **Avessi una rete ristretta**: se è abbastanza piccola posso lavorare con un clock unico e vedo per tempo se il pacchetto è in circolazione o è defunto. Questa sarebbe una soluzione attuabile con una rete abbastanza piccola. Supponendo di avere una rete abbastanza grande invece, si intende un metodo che impedisca ai pacchetti di percorrere dei cicli e un modo per contenere il ritardo dovuto anche alla congestione su un percorso più lungo possibile;
- 2) Se metto un **contatore di salto in ogni pacchetto**: che può essere decrementato a ogni salto;
- 3) Posso inserire un **orario in ogni pacchetto** (timestamp): ma da dove lo prendo l'orario su una rete vasta? Tutti i router dovrebbero essere sincronizzati, ma la cosa è praticamente impossibile da attuare.

È necessario poi garantire che il pacchetto sia realmente defunto che non sia ancora in circolazione. Questo sia per quanto riguarda il pacchetto vero e proprio, più tutti gli eventuali acknowledgement a lui legati (tempo lungo). Dato che il livello di trasporto è l'ultimo livello che si occupa di gestire la connessione tra due host, allora si capisce quanto questo non sia un problema banale. Bisogna fare in modo che l'attivazione di quella connessione venga fatta in maniera seria.

Quando un pacchetto muore, devo attendere **un tempo T** (che definisce la cosiddetta **area proibita**), che è il *tempo necessario per evitare che l'ultimo dei potenziali acknowledgement ancora in circolazione del pacchetto defunto non sia ancora in circolazione all'interno della linea*. Questo tempo è stato definito come di **120 secondi** nella rete Internet.

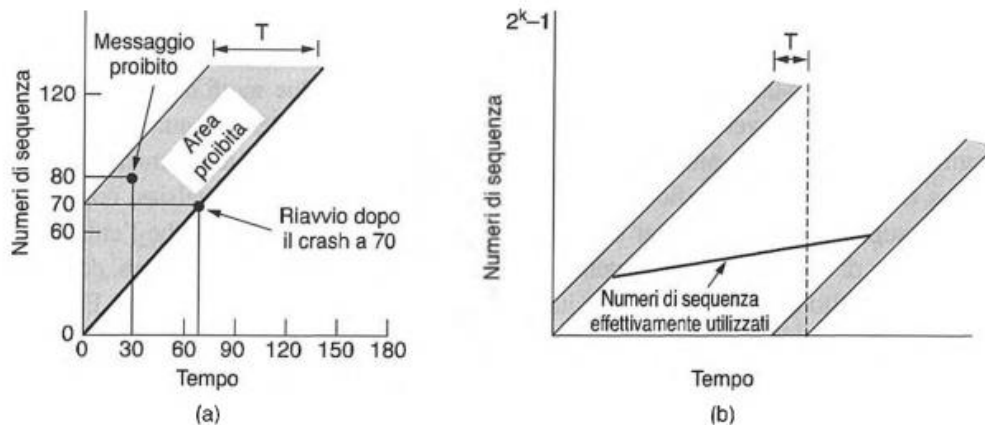
Atteso questo tempo T fissato, sarà quindi possibile dare per morto un pacchetto che viaggia nella rete.

Questo metodo per risolvere il problema è stato descritto da **Tomilson**, e il cuore di questo metodo sta nella sorgente che *deve etichettare i segmenti con numeri di sequenza che non saranno riutilizzati per un numero di secondi pari a T*. Il periodo T e il tasso dei pacchetti al secondo andranno a delimitare la **dimensione dei numeri di sequenza**. In questo modo soltanto un pacchetto con un dato numero di sequenza può essere in sospeso in un certo istante. Possono però ancora verificarsi dei duplicati ma almeno non può capitare che il duplicato in ritardo di un vecchio pacchetto batta sul tempo un pacchetto nuovo con lo stesso numero di sequenza.

Un altro metodo potrebbe essere quello di tenere *inattiva l'unità di trasporto di una macchina che ha subito un malfunzionamento*. L'inattività sarà di **T**, ma per reti molto estese potrebbe essere un tempo fin troppo lungo e dunque poco accettabile. Un'altra alternativa proposta da **Tomilson** insieme a **Sunshine e Dalai**, fu

quella di installare degli **orologi** sugli host che funzionano come dei *contatori binari*, dunque non hanno necessariamente bisogno di essere sincronizzati. L'orologio deve continuare a funzionare anche quando l'host è inattivo però, questa deve essere una assunzione da fare a monte. **QUINDI COSA SUCCEDDE?**

Quando viene instaurata una connessione, i **k bit di ordine più basso** nell'orologio vengono usati come primo *numero di sequenza*. Quindi, diversamente dai protocolli di rete, ogni connessione inizia numerando i segmenti in modo diverso. Lo spazio della sequenza deve essere abbastanza grande da fare in modo che quando questi vengono esauriti, e si ricomincia a contare, i segmenti con il numero di sequenza iniziale devono essere già spariti da parecchio all'interno della rete. Questa relazione è mostrata dalla figura



La parte della **regione proibita**, indica il tempo per il quale non è possibile riutilizzare il numero di sequenza di un segmento. Se un certo segmento fosse spedito dentro questa regione temporale, potrebbe accadere che questo venga ritardato e che possa essere scambiato poi per un altro segmento spedito più tardi. L'host ad esempio potrebbe crashare e riavviarsi nell'istante in cui l'orologio segna 70 secondi.

Utilizzerà quindi dei numeri per iniziare le sequenze in base all'istante in cui torna a funzionare e non userà numeri dentro alla regione proibita. Una volta che entrambe le entità di trasporto si sono messe d'accordo sul numero di sequenza iniziale, può essere usato qualunque **protocollo a finestra scorrevole** per il controllo del flusso. Questo troverà e scarcerà i duplicati dei pacchetti nel caso in cui siano già stati accettati.

Per garantire che i numeri di sequenza dei pacchetti siano fuori dalla regione proibita abbiamo bisogno di fare attenzione a due aspetti:

- 1) Se un host spedisce troppo velocemente troppi dati su una connessione appena aperta, l'andamento del numero di sequenza attuale rispetto al tempo può crescere più rapidamente del numero di sequenza iniziale, sempre rispetto al tempo, facendo entrare il numero di sequenza nella regione proibita. Per evitare che questo accada, *il massimo tasso di invio dei dati per qualsiasi connessione è di un segmento per ogni ciclo di clock.*

Questo vuol dire anche che l'entità di trasporto *dovrà aspettare fino al prossimo avanzamento dell'orologio prima di aprire una nuova connessione* dopo il riavvio dovuto a un malfunzionamento, altrimenti lo stesso numero potrebbe essere utilizzato due volte.

Entrambe queste argomentazioni sono in favore di un **ciclo di clock molto breve** (1 ps o meno), ma l'orologio non può procedere troppo velocemente rispetto al numero di sequenza. Per una **velocità C** dell'orologio e uno spazio dei **numeri di sequenza S**, si deve avere $S/C > T$ così che la sequenza numerica non possa ricominciare da capo troppo velocemente.

Nella figura 6 (b) è possibile notare che, per qualsiasi cadenza dei dati inferiore a quella dell'orologio, la curva dei numeri di sequenza effettivi entra nell'area proibita da sinistra. Maggiore è la pendenza della curva dei numeri di sequenza effettivi, *più lungo sarà ritardato questo evento.*

Come affermato in precedenza, appena prima di inviare ogni TPDU *l'entità di trasporto deve controllare se sta per entrare nell'area proibita*; in tal caso, deve ritardare la TPDU per T secondi o risincronizzare i numeri di

sequenza. Il metodo basato sull'orologio risolve per le TPDU dati *il problema del duplicato in ritardo*; tuttavia, affinché questo metodo sia utile bisogna prima stabilire una connessione.

Dal momento che anche le TPDU di controllo possono essere ritardate, esiste un problema potenziale per accordare le due parti sul numero di sequenza iniziale. Supponiamo, per esempio, che le connessioni vengano stabilite mediante l'host 1 che invia a un peer remoto (l'host 2) una TPDU CONNECTION REQUEST, contenente il numero di sequenza iniziale proposto e un numero di porta di destinazione. Il ricevitore host 2 riconosce la richiesta restituendo una TPDU CONNECTION ACCEPTED. Se la *TPDU CONNECTION REQUEST viene persa*, ma nell'host 2 appare improvvisamente una CONNECTION REQUEST duplicata e ritardata, la connessione viene stabilita in modo non corretto.

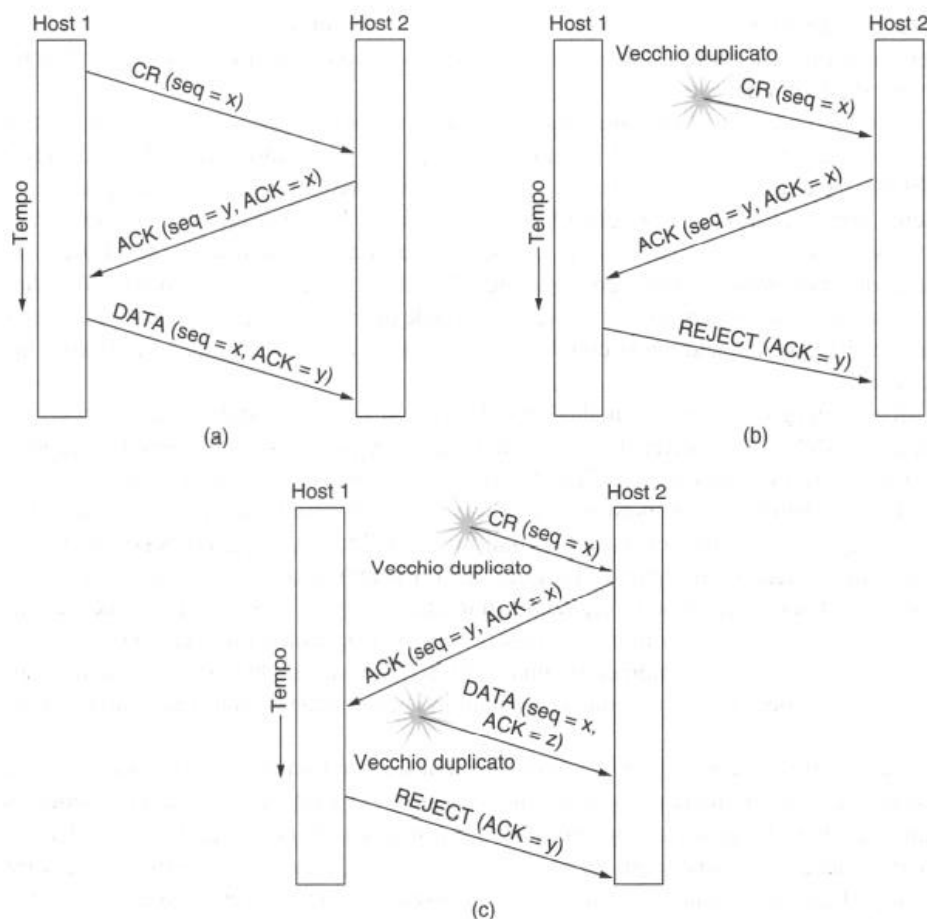
Per risolvere questo problema, sempre Tomilson ha introdotto la cosiddetta **Tripla stretta di mano**.

THREE-WAY HANDSHAKING

Per risolvere il problema della duplicazione di pacchetti, Tomlinson nel 1975 si è occupato del problema inventando l'algoritmo **three-way handshaking** (triplo stretta di mano). L'idea di base sta nella verifica reciproca da parte dei peer che l'attuale richiesta di connessione non sia un duplicato. Nel 1975 le università stanno utilizzando da poco internet ed è da poco finito il periodo di ARPANET, teniamolo a mente. Ma come funziona quindi la tripla stretta di mano?

- ✚ Il richiedente invia un TPDU di tipo *connection-request* con sequenza **x**, dall'host 1 all'host2.
- ✚ L'host 2 (destinatario) invia un TPDU che dice *ack.x* (ti do l'ack di x, è arrivata correttamente) e la proposta di un proprio **numero y progressivo** (esempio y), che è un numero di inizio sequenza.
- ✚ A quel punto il richiedente invia un TPDU di tipo dati che conterrà:
 - i primi dati del dialogo
 - la conferma di y (ack di y).

I valori di x e y possono essere generati sfruttando l'orologio di sistema, in modo da avere valori ogni volta diversi.



Vediamo in verticale il fluire del tempo, abbiamo 3 casistiche:

- a) Si vede host 1 che invia il suo **numero di sequenza x** e invia un segmento CONNECTION REQUEST contenente x all'host 2. L'host 2 la riceve e invia l'ack di x verso host 1 per dare conferma di ricevuta. Host 2 invia questo dato di ack la cui **sequenza è y**, e poi host 1 invia l'ack di y che ha ricevuto da host 2 con un nuovo numero di sequenza che è x. Questa è *l'instaurazione normale*;
- b) In questo caso abbiamo che il primo segmento è una CONNECTION REQUEST **duplicata e ritardata** che si riferisce a una vecchia connessione. Questo segmento arriva all'host 2 senza che l'host 1 lo sappia. L'host 2 reagisce inviando all'host 1 *un segmento ACK*, chiedendo in definitiva una conferma del fatto che l'host 1 stia tentando di creare una nuova connessione. Quando l'host 1 rifiuta il tentativo dell'host 2 di stabilire una connessione, *l'host 2 comprende di essere stato ingannato da un duplicato in ritardo e abbandona la connessione*; in questo modo un duplicato in ritardo non fa danni.
- c) Il caso C è il peggiore: si verifica quando abbiamo la presenza duplicata di una **CR ritardata e dell'ack ritardato**. Host 2 riceve una CR ritardata e risponde ad essa. A questo punto, host 2 ha proposto di utilizzare **y** come numero di sequenza iniziale per il traffico tra le macchine, sapendo che nessun segmento contenente il numero di sequenza y o l'ack y è ancora in circolazione. Quando il *secondo segmento ritardato* giunge ad host 2, il fatto che sia stato confermato **z** e non **y**, *suggerisce ad host 2 che questo era un vecchio duplicato*. Viene quindi anche in questo caso rigettata la richiesta. Bisogna notare il fatto che in questo caso non esiste una combinazione di vecchi segmenti che possano accidentalmente instaurare una connessione non richiesta.

RILASCIO DI UNA CONNESSIONE

Questo problema si ripropone nella parte di **rilascio della connessione**. Il rilascio al livello di trasporto deve essere un rilascio netto, perché le due macchine non devono avere nulla di pendente che carichi la rete. L'entità di trasporto rimuove le informazioni sulla connessione e informa l'utente di livello superiore che la connessione è chiusa.

Questo modello lo rendiamo rigoroso tramite due tipi di rilascio:

- ✚ **Rilascio Asimmetrico**: ad esempio, quando si chiude il telefono, la comunicazione resta comunque aperta (comunicazione asimmetrica). Se io faccio un rilascio asimmetrico è possibile che *io lo rilasci la connessione mentre ci siano ancora dati in viaggio*. Vediamo nella figura cosa può succedere se faccio un rilascio asimmetrico (DR che parte quando sto trasferendo i dati).
- ✚ **Rilascio Simmetrico**: l'ideale quindi è un rilascio simmetrico in cui le azioni combinate degli host condividono azioni reciproche realizzando un rilascio completo.

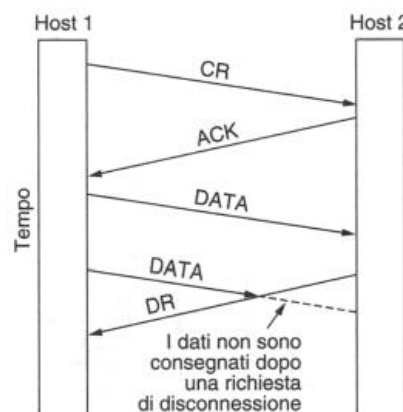


Figura 6.12 Una disconnessione improvvisa con perdita di dati.

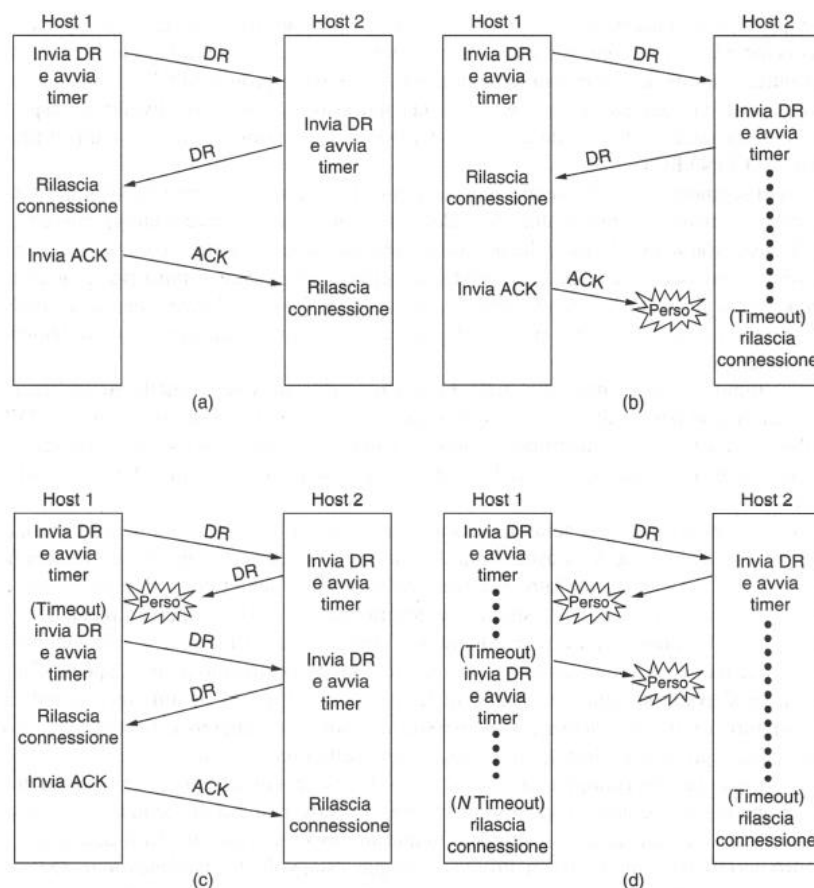
ESEMPIO DEI DUE ESERCITI

Questo **esempio dei due eserciti** è abbastanza esplicativo circa il problema del **rilascio asimmetrico**. L'**esercito bianco** ha un numero di componenti che è da solo superiore all'esercito 1 (blu) e all'esercito 2 (blu), ma se i due eserciti blu attaccassero simultaneamente l'esercito bianco, allora avrebbero la vittoria sicura. Per esempio, l'esercito 1 per attaccare contemporaneamente con il 2 manda un messaggio "Partiamo domani alle 6" all'esercito 2. L'esercito 1 non attaccherà se l'esercito 2 non gli darà conferma. Se anche l'esercito 2 inviasse la conferma, 2 non sarà certo che 1 abbia ricevuto la conferma e così via... (questo è un processo iterativo senza fine, perché nessuno è certo che l'altro abbia ricevuto la comunicazione. Si arriva quindi al paradosso). La tecnica che è stata elaborata per il rilascio asimmetrico che tiene conto di queste particolarità:



Figura 6.13. Il problema dei due eserciti.

L'idea è quella di evitare questa situazione non imponendo la necessità di un accordo e *demandando il problema a chi usa il livello di trasporto, facendo decidere in modo indipendente i partecipanti*. La figura in alto illustra 4 scenari possibili.



- a) In un caso normale di rilascio con **three-handshake**: L'host 1 vuole disconnettere, invia una **DISCONNECTION REQUEST** (DR) all'host 2. Quando arriva all'host 2, il destinatario invia anche lui una DR con un timer utile nel caso in cui il suo DR vada perso. All'arrivo di questo DR, il mittente originale invia un'ack e viene rilasciata la connessione. Il rilascio della connessione significa che l'entità di trasporto *rimuove le informazioni sulla connessione dalla propria tabella delle connessioni aperte* e lo segnala al proprietario della connessione (l'utente al livello di trasporto). Ciò è diverso dal rilascio della connessione tramite DISCONNECT.
- b) In questo caso vediamo cosa succede se l'ack di ricevuta DR viene perso. *DR fa avviare un timer in modo che se non ho la conferma, la connessione viene comunque rilasciata*. Se non lo faccio torno al caso del paradosso dei due eserciti;
- c) Se viene persa una risposta ho bisogno che sia comunque possibile andare avanti, e si ipotizza che la seconda volta nessuna DR venga persa;
- d) Il caso D equivale a quello C tranne per il fatto che ora si presume che *i tentativi ripetuti di ritrasmettere la DR falliscano a causa delle TPDU perse*. Dopo N tentativi il mittente rilascia la connessione, e nel frattempo si verifica un timeout nel ricevitore che a sua volta esce. Questo protocollo è generalmente sufficiente, ma in teoria può fallire se vengono perse la DR iniziale e le N ritrasmissioni. Il mittente abbandonerà i tentativi e rilascerà la connessione, mentre *l'altro lato non saprà nulla di tutti i tentativi di disconnessione e sarà pienamente attivo*. Questa situazione provoca una connessione aperta a metà.
- Un modo per interrompere tutte le connessioni aperte a metà può essere la definizione di una regola: se non arrivano TPDU per un certo numero di secondi, *la connessione viene automaticamente terminata*. In questo modo, se un lato si disconnette, l'altro lato rileverà la mancanza di attività ed eseguirà a sua volta la disconnessione.

CONTROLLO DEL FLUSSO E GESTIONE DEL BUFFER

Nel controllo del flusso abbiamo il solito problema studiato anche nel livello data link, ovvero dobbiamo evitare che un *ricevente lento venga affogato da una macchina che spedisce molto velocemente*. Nel sistema di trasporto però abbiamo una particolarità: i segmenti non sono sempre uguali, e questo può creare problemi. Se avessimo un solo segmento si preferisce utilizzare lo **stop-and-wait**, altrimenti **sliding window**.

Mettiamo caso che io abbia una connessione di livello di trasporto tra il terminale di una banca e un server centrale. Molte volte il terminale della banca carica in alcuni casi solo 8 bit per dare un certo comando, abbiamo quindi dei piccoli traffici, mentre in altri abbiamo traffici molto grandi. Abbiamo quindi un *problema di gestione dovuto alla grande variabilità di dimensione del TPDU*:

- ✚ **Traffico bursty ma poco gravoso** (ad esempio in una sessione di emulazione di terminale): niente buffer a destinazione, bastano buffer alla sorgente;
- ✚ **Traffico gravoso** (per esempio il file transfer): buffer cospicui a destinazione, per poter sempre accettare ciò che arriva.

Usiamo in genere dei modelli di tipo sliding window come quelli a livello data link:

- ✚ Può capitare che io invii pochi dati, e questi vengono messi nel segmento che arriva a destinazione e viene letto, controllato e ceduto dalla macchina destinataria a livello superiore, e poi la connessione continua;
- ✚ Può capitare che io invii una quantità di dati enorme ma solo per un momento.

Se il **servizio di rete è inaffidabile**, il mittente deve inserire nel buffer tutte le TPDU inviate, come nello strato data link, ma con un servizio di rete affidabile diventano possibili altri compromessi. In particolare, *se il mittente sa che il ricevitore ha sempre uno spazio nei buffer, non deve mantenere copie delle TPDU inviate*. Tuttavia, se il ricevitore non può garantire l'accettazione di ogni TPDU in ingresso, il mittente dovrà comunque utilizzare il buffer. Nell'ultimo caso il mittente non può fidarsi dell'acknowledgement dello strato network, perché indica solo l'arrivo della TPDU e non la sua accettazione.

Per compensare questa differenza abbiamo 3 modelli di buffer da utilizzare per il controllo del flusso:

- a) **Buffer a dimensione fissa:** se la maggior parte delle TPDU ha una dimensione simile, è naturale organizzare i buffer sotto forma di pool di buffer con dimensione identica e una TPDU per buffer;
- b) **Buffer a dimensione variabile:** dove utilizzo porzioni di spazio differenti, il vantaggio è un migliore utilizzo della memoria, al prezzo di una gestione più complicata dei buffer;
- c) **Buffer circolari per ogni tipo di connessione:** questa tecnica consiste nel dedicare un singolo buffer circolare di grandi dimensioni per ogni connessione. Questo sistema fa un valido utilizzo della memoria quando tutte le connessioni sono caricate in modo pesante, ma è scadente se alcune connessioni hanno poco carico.

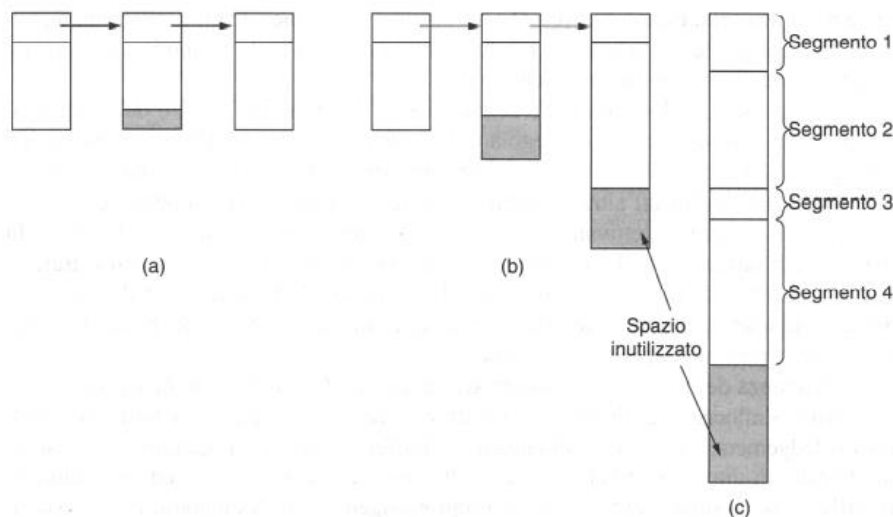
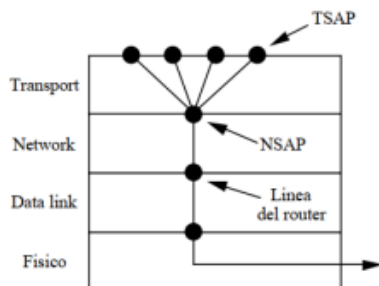


Figura 6.15 (a) Buffer concatenati a dimensione fissa. (b) Buffer concatenati a dimensione variabile. (c) Un grosso buffer circolare per ogni connessione.

MULTIPLEXING

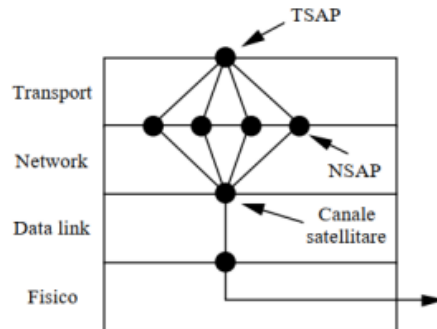
Può capitare che se richiedo anche un solo servizio, questo può essere gravoso, oppure posso avere una sola connessione sulla quale voglio appoggiare tanti servizi magari anche non gravosi, ma che potrebbero esserlo dato che sono tanti. Introduciamo quindi il multiplexing, che si divide in due categorie:

- ✚ **Upward multiplexing:** abbiamo un collegamento da una determinata macchina, abbiamo un indirizzo MAC, e un indirizzo NSAP (internet): abbiamo quindi un'unica connessione. Posso però avere diverse richieste di servizi contemporaneamente (FTP, Telnet, http, HTTPS etc...). Posso farlo con l'upward: *da un solo NSAP attivo più TSAP a livello superiore*, applicando quindi il multiplexing a livello superiore (Vedi figura);



- ✚ **Downward multiplexing:** può capitare il caso contrario, ossia che io debba fare una quantità di traffico dati impressionante mentre sto scaricando il nostro solito HDD da 500 GB. In questo caso ho una sola FTP che sta funzionando, ma per poter mettere in piedi la il trasferimento non posso usare una sola connessione IP. **QUINDI CHE COSA FACCIO?** Posso mettere in piedi più connessioni IP in modo da dividere almeno linearmente il carico. Per esempio, vediamo il caso in figura: *avendo un solo TSAP, instauro più connessioni IP per dividere il carico del FTP*. In questo caso ipotizziamo che ognuna di queste linee che diverge fino al livello

dell'NSAP sia una linea ISDN da 64 kbps. Scarico ma non è sufficiente, quindi viene attivato il secondo e arrivo a 128, e così via fino a 256. Arrivo fino a 256 perché non mi basterebbero gli altri da soli. *Ho attivato 4 differenti indirizzi IP di connessione per dividere il carico da scaricare.* Questo è il caso del downward multiplexing (In pratica mi collego a una singola TSAP perché sto richiedendo una sola FTP, dunque mi serve un solo tipo di servizio, ma una singola connessione IP non mi basta per svolgere il compito, quindi la ripartisco il carico su diverse linee che vanno sull' NSAP che convergono verso l'alto a richiedere l'unico TSAP di cui ho fatto richiesta).



RIPRISTINO DOPO IL CRASH

Il livello di trasporto deve funzionare benissimo dato che è l'ultimo livello nel quale avviene il controllo tra le macchine. Ipotizziamo di avere vinto a una lotteria, e sul mio conto siano stati caricati 1.000.000 €. Devo scaricare i soldi in home banking, e la mia macchina è l'host 1 e la banca è la macchina host 2. Lancio il trasferimento e crolla la linea. Succede la casistica classica del **ripristino dopo il crash**.

Il ripristino dopo il crash serve a capire che cosa succede una volta che le linee vengono ritirate su. In particolare, il problema si presenta quando il *server crasha per qualche secondo e reinizializza le sue tabelle*, quindi, *non sa precisamente a che punto del trasferimento era arrivato*. Questo capita spesso, e si possono verificare diverse eventualità che tengono conto, intanto, se esistono dei *segmenti ancora in circolazione che non hanno ricevuto ack (\$1)*, e qui c'è necessità di trasmissione, oppure se *segmenti in circolazione non ce ne sono (\$0)*.

Strategia utilizzata dall'host che invia	Strategia utilizzata dall'host che riceve					
	Prima ACK, poi scrittura			Prima scrittura, poi ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Ritrasmetti sempre	OK	DUP	OK	OK	DUP	DUP
Non ritrasmettere mai	LOST	OK	LOST	LOST	OK	OK
Ritrasmette in S0	OK	DUP	LOST	LOST	DUP	OK
Ritrasmette in S1	LOST	OK	OK	OK	OK	DUP

OK = il protocollo funziona correttamente
 DUP = il protocollo genera un messaggio duplicato
 LOST = il protocollo perde un messaggio

Le sequenze possono essere di **Acknowledgement** (A), **Crash** (C) e **scrittura** (W) e possono variare. Negli esempi sopra, le *parentesi* seguono il crash C e indicano che né A né W possono seguire C (una volta che l'host ha subito un crash non c'è più nulla da fare).

Il server può essere implementato in due modi: **prima ack e scrittura**, oppure **prima scrittura e poi ack**. A seconda di questo, si vanno a creare diverse possibilità per la gestione dei crash. Nel caso in cui avvenga prima l'acknowledgement e poi la scrittura, avremo queste tre casistiche che spiegano la tabella riportata in alto in figura:

- ✚ **AC(W):** ho dato l'ack, c'è stato il crash, e non so se è avvenuta la scrittura. Devo ritrasmettere per forza. Se non ritrasmetto in questo caso il protocollo perde un messaggio. Se si verifica un crash dopo l'invio dell'acknowledgement ma prima della scrittura (come in questo caso), il client riceverà l'acknowledgement e pertanto si troverà nello stato **S0** quando giunge l'annuncio di ripristino dal crash. Il client non eseguirà di nuovo la trasmissione, pensando (erroneamente) che il segmento sia arrivato. Questa decisione del client genera un segmento mancante;
- ✚ **AWC:** è stato dato l'ok, ho scritto (cioè ho prelevato dal livello di trasporto e portato al livello applicativo), e solo dopo si è verificato il crash. In questo caso ho ricevuto tutto a livello superiore, posso ritrasmettere ma avrò un duplicato. Se non trasmettessi sarebbe tutto ok, perché ho già ricevuto tutto a livello superiore;
- ✚ **C(AW):** c'è stato un crash e gli ultimi due non ne so granché. Devo ritrasmettere per forza altrimenti perderei il protocollo perde un messaggio.

Se nel server avviene prima la scrittura in output al livello applicativo e poi l'ack avremo questi casi:

- ✚ **C(WA):** Avviene prima il crash, quindi la soluzione è sempre quella di ritrasmettere sempre per evitare la perdita di segmenti. Il caso è del tutto analogo a CAW visto in precedenza;
- ✚ **WAC:** Questo caso è invece analogo a quello AWC; se ho già eseguito entrambe le operazioni e il sistema va in crash, non è necessaria la ritrasmissione in quanto avrei semplicemente un duplicato superfluo;
- ✚ **WC(A):** In questo caso, si supponga che la scrittura sia stata eseguita, ma che il crash avvenga prima dell'invio dell'acknowledgement. Il client sarà nello stato **S1** e se eseguirà di nuovo la trasmissione, genererà un segmento duplicato non rilevato nel flusso di output verso il processo applicativo del server. Dunque, la soluzione in questo caso è quella di non trasmettere in modo che il protocollo funzioni bene, oppure di ritrasmettere in stato S0 ma non in stato S1.

PROTOCOLLI DI TRASPORTO DI INTERNET (UDP E TCP)

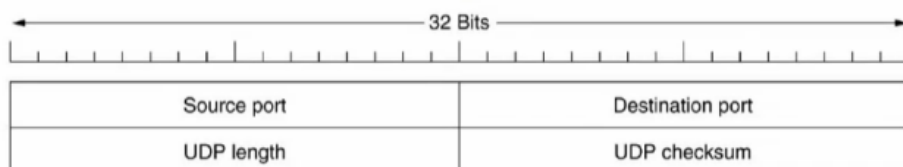
Il livello trasporto di Internet è basato su due protocolli:

- ✚ **UDP** (User data Protocol): principio di commutazione a datagramma, RFC 768;
- ✚ **TCP** (Transmission Control Protocol): principio di commutazione circuito virtuale RFC 793, 1122 e 1323.

UDP (USER DATA PROTOCOL)

L'UDP è un **datagramma** quindi è veloce, ha bassa conferma e vede dove può andare a seconda della sequenza che viene attivata e si attiva per arrivarci nel modo più rapido secondo le disponibilità di linee nella rete verso il destinatario. Queste sue caratteristiche presuppongono che abbia sotto una rete piuttosto efficiente. L'UDP lo usiamo quindi in **reti abbastanza affidabili**, o se abbiamo a che fare con dati che se andassero persi non stiamo perdendo dati importanti.

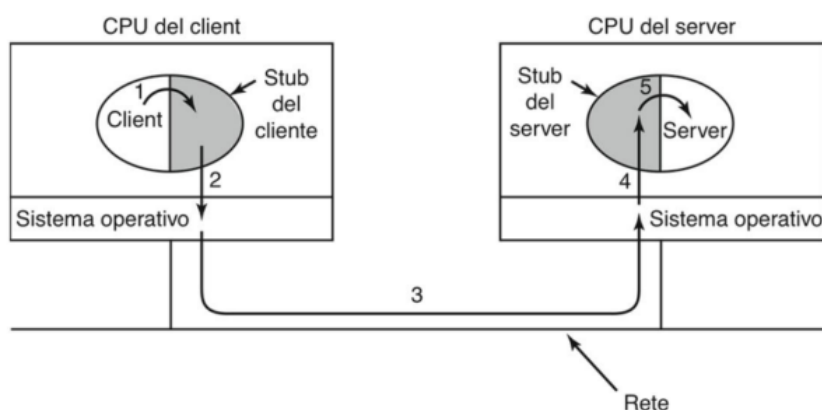
L'**header dell'UDP** è uno dei più piccoli esistenti proprio per garantire questa sua caratteristica di snellezza (più piccolo c'è solo l'intestazione di ATM). Ha i **16 bit** che definiscono la porta numero 1 o **Source port**, e i 16 che definiscono la porta numero 2 o la **destinazione**. Un campo con la **lunghezza dell'UDP** che impieghiamo e un **campo di checksum**. Il campo di checksum può non essere considerato, e ciò avviene tipicamente nel caso di sessioni real-time.



RPC (REMOTE PROCEDURE CALL)

Utilizziamo UDP anche in **macchine terminali**, e ne è un esempio l'impiego nelle procedure di chiamata dette **RCP** (Remote Procedure Call). La macchina Client lavora e utilizza una porzione dello **stub** del client che rappresenta la *procedura del server nello spazio di indirizzamento del client*, anche il server avrà uno stub del server che è speculare. per invocare dei dati su una macchina server remota.

Avviene una vera e propria **chiamata di procedura remota** da client a server per svolgere un qualche tipo di funzione. Questo client funziona benissimo se funziona bene la rete UDP, perché UDP è velocissimo e mi scarica i dati che mi servono interrogando il server tanto che non si rende conto che sta utilizzando una macchina distante, se ne rende conto solo se dovesse cadere la comunicazione. L'idea dietro a questo modello è quella di nascondere il meccanismo di chiamata di procedura remoto in modo tale che sia trasparente sulla macchina client a livello di velocità. Ci serviamo dell'UDP proprio per la sua velocità nell'eseguire questo tipo di operazioni.



Il funzionamento è semplice:

1. Inizialmente il *client chiama una procedura dello stub del client*, facendo una chiamata di procedura convenzionale con i parametri inseriti nello stack.
2. Dopodiché, il passaggio successivo sarà il *client stub che inserisce i parametri in un messaggio* che effettua una chiamata di sistema per inviare il messaggio al server (**marshalling**).
3. Questo messaggio viene spedito dal kernel e arriva al server.
4. Nel passaggio successivo vediamo che il messaggio passa dal kernel del server in ingresso allo stub del server.
5. Questo restituisce la richiesta che avevamo fatto partire dal client, e la risposta torna indietro esattamente come è arrivata.

RTP (REAL TIME TRANSPORT PROTOCOL)

Un altro utilizzo può essere quello impiegato per un *protocollo real-time*, che deve essere caratterizzato da una certa efficienza che ci consenta una certa velocità. È preferibile perdere qualche segmento ma avere una certa continuità di invio dei dati per garantirne la continuità della percezione.

All'interno del segmento UDP viene annegato un payload di tipo **RTP** (Real time transport protocol), un tipo di protocollo che può essere visto garantendo quella parte di fibrillazione (**jittering**) interna del dato trasferito che è in grado pur perdendo un datagramma, di fornirci una certa continuità nella ricezione. La funzione base di RTP è quella di *eseguire il multiplexing di flussi di dati real-time in un singolo flusso di pacchetti UDP*. Questo può essere inviato singolarmente o su più direzioni (unicasting e multicasting).

Dato che RTP utilizza un normale UDP, i pacchetti RTP sono trattati come gli altri dai router. Ogni pacchetto di un flusso è numerato, questo perché a seconda del traffico è possibile o scartare i pacchetti più vecchi in favore di quelli nuovi, o approssimare il segnale tramite interpolazione se perdiamo qualche pacchetto di messo (ad esempio per il traffico audio). Non si applica in nessun caso la ritrasmissione, sia per la natura di UDP che sappiamo essere datagramma, sia perché altrimenti non sarebbe un protocollo di trasporto real-time.

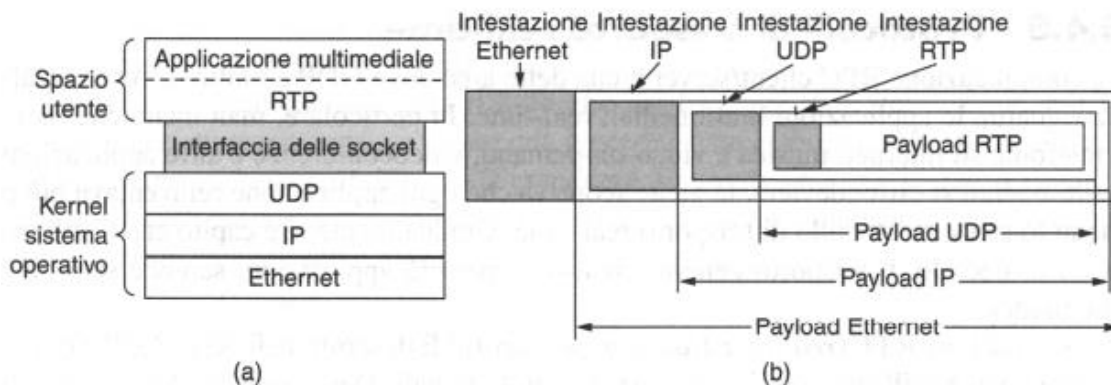


Figura 6.30 (a) La posizione di RTP nello stack dei protocolli. (b) La nidificazione dei pacchetti.

RTCP (REAL-TIME TRANSPORT CONTROL PROTOCOL)

Un altro protocollo agganciato al primo è **RTCP** (Real-time Transport Control Protocol) che controlla le reti. È descritto insieme al primo nella RFC 3550 e *gestisce le retroazioni verso la sorgente, la sincronizzazione e l'interfaccia utente*.

Quando le informazioni multimediali raggiungono il destinatario, abbiamo una determinata quantità di segmenti definita ed un certo tempo di buffering. Questi segmenti vengono inviati in rete in modo differenziale e man mano che arrivano vengono rimossi dal buffer però arrivano anche in maniera molto distanziata. Questo perché anche se invio al destinatario i segmenti con lo stesso intervallo, questi raggiungeranno il destinatario con diversi tempi relativi e ci sarà dunque una certa *variazione del ritardo* chiamata **Jitter**.

La soluzione per colmare il problema del jitter è proprio *fare uso di un buffer su cui salvare l'informazione*. In figura vediamo il disallineamento tra il tempo di spedizione e il segmento che arriva al buffer. Vi è un certo disallineamento ma facendo in modo che il segmento venga rimosso dal buffer con un certo periodo di attesa dopo l'arrivo del primo segmento, possiamo avere sulla macchina utente una *rappresentazione abbastanza continua*.

Se però il segmento numero 8 ad esempio arriva con un ritardo troppo alto lo perderò, e avrò allora un **playback point**, ovvero un punto da cui partire perché la sequenza non è più rispettata. Il playback point è un concetto chiave che ci permette di avere una trasmissione di dati continuativa. Questo meccanismo è il motivo dei **delay la trasmissione via satellite e televisiva classica**.

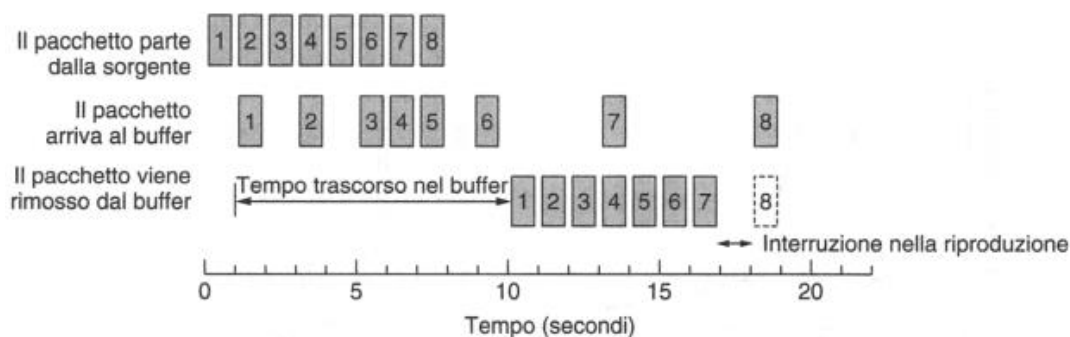


Figura 6.32 Regularizzazione del flusso in uscita tramite buffer.

Per poter lavorare con correttezza facciamo il controllo del jitter (tremolio). In merito al punto di playback, decidere quanto attendere prima di riprodurre i dati dipende fortemente dal jitter. Nel grafico in basso vediamo in (a) un **jitter alto** e in (b) un **jitter basso**.

Il ritardo medio non deve differire molto tra i due ma se c'è un punto di jitter molto alto il punto di playback può avere bisogno di essere abbondantemente lontano per catturare il 99% dei segmenti. Per scegliere un buon punto di playback l'applicazione può misurare il jitter guardando la differenza tra i timestamp RTP e il

tempo in arrivo. Ogni differenza ci dà un campione di ritardo ma questo può cambiare a causa del traffico di rete in competizione o di un cambio di percorso. Per accomodare questo cambiamento, le applicazioni possono *adottare i loro punti di playback durante l'esecuzione*. Se non è fatto in modo corretto però, il punto di playback può produrre artefatti visibili all'utente finale, quindi si può optare (ad esempio in una comunicazione audio) per una prolungata pausa di silenzio che è difficilmente notabile rispetto a un disallineamento o a un errore audio. Se il ritardo assoluto previa riproduzione è troppo alto il contenuto ne soffrirà, e non si può fare niente per evitare questo se non utilizzare una linea diretta o affidarsi a una QoS migliore.

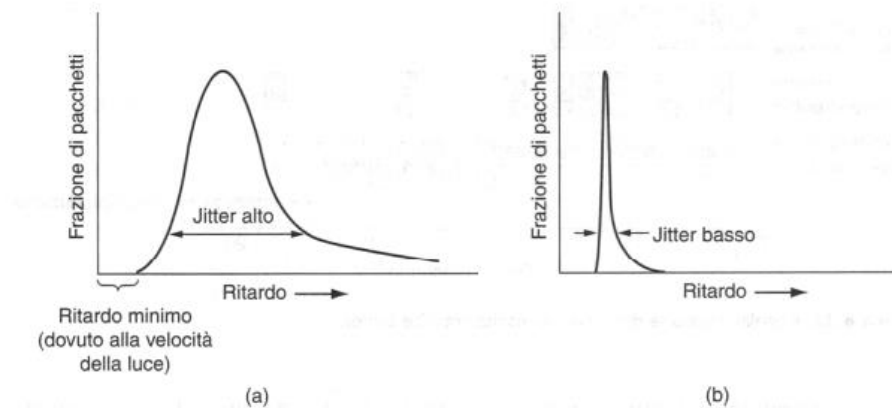


Figura 6.33 (a) Jitter alto. (b) Jitter basso.

TCP (TRANSMISSION CONTROL PROTOCOL)

Il TCP è il protocollo di gran lunga più utilizzato. Tutte le grandi funzionalità, le primitive e le porte principali sono appoggiate al TCP. Questo deve accettare dati dal livello applicazione, li deve spezzare e inserirli in segmenti, e invia la sua TDPU (segmento di dimensione massima di **64Kb**, ma tipicamente di dimensione di 1500 byte). Li consegna a livello di rete, se le cose non vanno bene li ritrasmetto (TCP si tiene nel buffer), se tutto va bene scarichiamo il buffer e passiamo al segmento successivo.

Riceve segmenti dal livello di rete e li rimette in ordine all'arrivo (eliminando buchi e doppioni), infine consegna i dati al livello applicazione dopo averli ordinati.

Il protocollo TCP è stato progettato per fornire un **flusso di dati di tipo affidabile**, da sorgente a destinazione, su una rete non affidabile (IP) e un servizio **full-duplex** con gestione di **ack** e **controllo del flusso di dati**. TCP è fondamentale per il buon funzionamento della rete.

I servizi TCP si ottengono creando connessione a livello di trasporto identificata da una coppia di punti di accesso detti socket. Ogni socket ha un **socket number** che aggancia il numero della porta 16 bit con l'IP address (32 bit). Il socket number costituisce il nostro TSAP.

Questa associazione (la porta da 16 bit) ci porta ad aver **65535** (ovvero 2¹⁶) **primitive**, che sono un'enormità. Non posso tenerle attive tutte ma alcune sono essenziali (HTTP, Telnet, FTP...), e quasi tutti questi servizi stanno sotto la porta 1024 e fanno parte delle già citate **Well-known-ports** che sono riservate ai servizi standard.

Porta	Protocollo	Utilizzo
20, 21	FTP	Trasferimento di file
22	SSH	Login remoto, rimpiazzamento di Telnet
25	SMTP	Posta elettronica
80	HTTP	World Wide Web
110	POP-3	Accesso remoto alla posta elettronica
143	IMAP	Accesso remoto alla posta elettronica
443	HTTPS	Web sicuro (HTTP su SSL/TLS)
543	RTSP	Controllo di riproduttori multimediali
631	IPP	Condivisione di stampanti

Le connessioni TCP sono full duplex e point to point, il tipo di connessione è identificata da **una coppia di socket number alle due estremità**. È possibile che su un singolo host più connessioni siano attestate localmente sullo stesso socket number. Se ho necessità posso mettere un **flag URGENT** per interrompere una computazione remota già iniziata (esempio CTRL-C di una sessione di emulazione terminale), per dire che dobbiamo andare veloci.

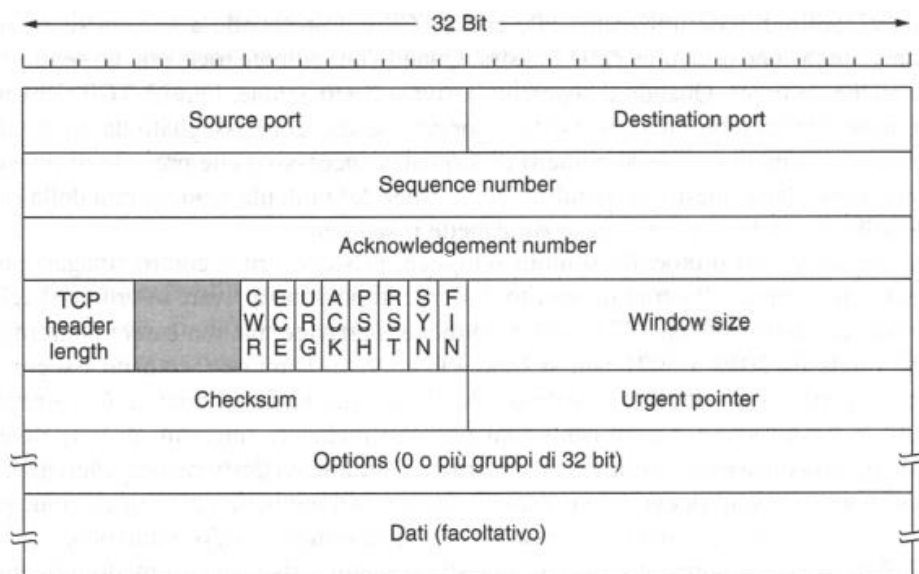
STRUTTURA DEL PROTOCOLLO E DELLA SUA INTESTAZIONE

Ogni byte del flusso TCP è numerato con un **numero di sequenza a 32 bit**, usato sia per il controllo di flusso che per la gestione degli ack. Un **segmento TCP** non può superare i 65535 byte.

Un segmento TCP è formato da:

- ✚ Un **header**, a sua volta costituito da:
 - Una parte fissa di **20 byte**;
 - Una parte opzionale;
- ✚ I **dati** da trasportare.

La struttura dell'intestazione del protocollo è questa:



- ✚ **Source Port**: identifica i 16 bit della porta sorgente;
- ✚ **Destination Port**: identifica i 16 bit della porta destinazione;
- ✚ **Sequence Number**: identifica il numero di sequenza, in modo tale che io capisca qual è la sequenza che arriva in modo che quando rispondo lo faccio diventare il numero dell'acknowledgement. Se mi

arriva la sequenza generica "x" risponderò con "ack.x" quando rispondo alla macchina che ha inviato;

- ✚ **Acknowledgement number:** come detto, se arriva una sequence number "x", risponderemo con "ack.x" in questo campo. Occorre notare che quest'ack specifica il *successivo byte previsto*, non l'ultimo byte ricevuto correttamente; si tratta di un **acknowledgement cumulativo**, perché riassume i dati ricevuti con un solo numero e non va al di là dei dati persi.
- ✚ **TCP header length:** una piccola porzione di 4 bit che identifica la lunghezza dell'header, l'informazione è necessaria perché il campo Options (opzioni) ha una lunghezza variabile e, di conseguenza, anche l'intestazione.

Seguono diversi bit di **flag**:

- ✚ **CWR ed ECE** sono usati per segnalare la congestione quando viene utilizzata **ECN** (explicit congestion notification):
 - **CWR** è usato per segnalare una *condizione di riduzione della finestra di congestione* dal mittente al destinatario TCP;
 - **ECE** è regolata per mandare un **ECN-Echo** a un mittente TCP per indicargli di rallentare il flusso di inoltro quando il destinatario TCP riceve un'indicazione di congestione dalla rete;
- ✚ **URG** è impostato a 1 quando si usa **Urgent Pointer**, che indica lo spiazzamento in byte in cui si trovano dati urgenti. Questa funzionalità si utilizza al posto degli interrupt ed è un metodo rudimentale per consentire al *mittente di inviare segnali al ricevente senza coinvolgere TCP nel motivo dell'interrupt*. Viene usato piuttosto raramente;
- ✚ **ACK** è impostato a 1 per indicare che Acknowledgement number è valido e questo è il caso di quasi tutti i pacchetti. Se vale 0, il segmento non contiene Acknowledgement number e quindi quel campo viene ignorato;
- ✚ **PSH** segnala la presenza di **dati PUSH**. Al ricevente viene chiesto di consegnare i dati dell'applicazione all'arrivo e di non archivarli nel buffer per poi trasmetterle un buffer completo;
- ✚ **RST** viene utilizzato per reimpostare una connessione che è diventata confusa a causa di un malfunzionamento dell'host o per altre ragioni. È anche utilizzato per *rifiutare un segmento non valido* o un *tentativo di aprire una connessione*. In generale, se si riceve un segmento con il bit RST attivato, si è di fronte a un problema;
- ✚ **SYN** viene utilizzato per *stabilire le connessioni*. La richiesta di connessione presenta **SYN = 1 e ACK = 0** per indicare che il campo Acknowledgement non è utilizzato. La risposta alla richiesta di connessione porta un acknowledgement, pertanto possiede **SYN = 1 e ACK = 1**. In pratica il bit SYN è utilizzato per segnalare entrambi **CONNECTION REQUEST e CONNECTION ACCEPTED**, mentre il bit ACK distingue tra le due possibilità;
- ✚ **FIN** viene utilizzato per *rilasciare una connessione*. Specifica che il mittente non ha altri dati da trasmettere. Tuttavia, dopo avere chiuso una connessione, il processo di chiusura potrebbe continuare a ricevere dati all'infinito. Entrambi i segmenti SYN e FIN possiedono numeri di sequenza ed è quindi garantito che verranno elaborati nell'ordine corretto.

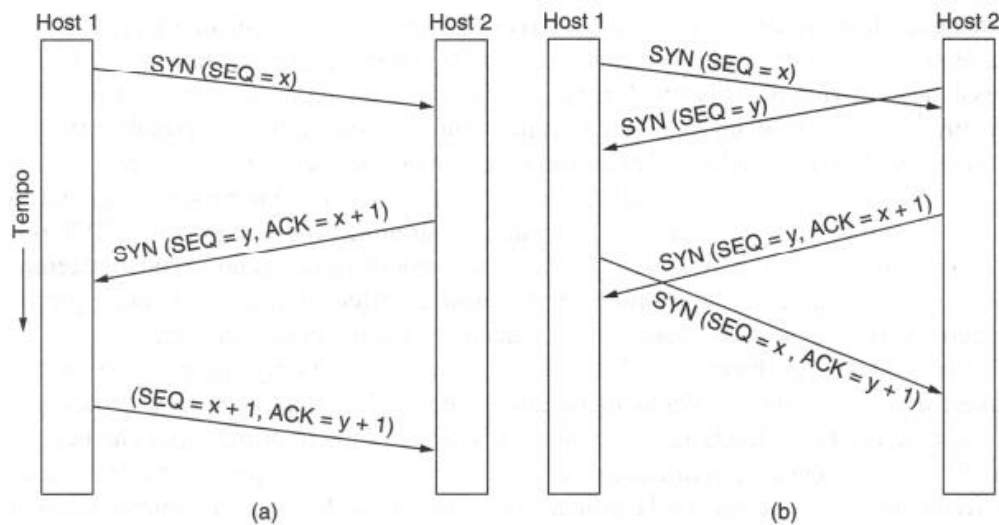
Seguono poi:

- ✚ **Window Size:** ci dà una dimensione complessiva della finestra ed indica quanti byte possono essere inviati a partire da quello di ack. Un campo Window Size con valore 0 afferma che i byte fino a **ACK number -1** sono stati ricevuti, ma che il ricevente non ha avuto modo di consumarli e non vuole altri dati per il momento (permette di gestire il flusso);
- ✚ **Checksum** è un campo per aumentare l'affidabilità. Questo fa un controllo dell'intestazione esattamente come in UDP;
- ✚ **Urgent Pointer:** spiazzamento in byte in cui si trovano i dati urgenti;
- ✚ Anche qui possono esserci dei campi opzione denominati **Options** che sono 0 o più gruppi di 32 bit;
- ✚ Seguirà infine il campo **Dati**.

INSTAURAZIONE DI UNA CONNESSIONE TCP

Nell'instaurazione di una connessione TCP, si utilizza la già discussa **hand-shake a tre vie**. Avrò una CONNECTION REQUEST sincronizzata con un **numero di sequenza x**, una risposta della sincronizzazione che sarà **y** che ci dà l'ack di **x** e poi la conferma della connessione.

Se *due richieste partono simultaneamente* ci può essere qualche problema come visto a destra nella figura.



Ma definiamo per bene che cosa succede quando instauriamo una connessione TCP.

Innanzitutto, per stabilire una connessione, da un lato (per esempio il server) attende in modo passivo una connessione in ingresso eseguendo le primitive **LISTEN e ACCEPT**. L'altro lato (diciamo il client) esegue le primitive **CONNECT** e invia un segmento TCP con il **bit SYN** posto a 1, perché sappiamo che quel bit serve apposta per le richieste di connessione TCP, e l'ack di quel segmento sarà posto a 0 in una prima richiesta per una CONNECTION REQUEST, per distinguere dalla CONNECTION ACCEPTED che presenta sia il bit SYN che ACK posti a 1.

Dopo aver inviato il segmento, il client attende risposta. Quando questo segmento arriva al server, l'entità TCP controlla se esiste un processo che ha eseguito una **LISTEN** sulla porta indicata da **Destination Port** e in caso negativo invia una risposta con il bit **RST** posto a 1, in caso positivo invece aggancia la richiesta. Se il processo è in ascolto gli viene passata la richiesta TCP sulla porta e se accetta la richiesta, viene mandata indietro una conferma tramite un segmento di ack.

La sequenza di segmenti TCP inviati nel caso normale è mostrata in figura (a) in alto. Nel caso che *due host tentino insieme di connettersi* a vicenda come nel caso di (b) dove avviene una richiesta simultanea, ne **risulta una sola connessione** e non due, dato che le connessioni sono identificate dai loro punti terminali. Se entrambe le richieste generano una connessione identificata da **(x,y)**, *viene creata una sola voce nella tabella delle connessioni aperte*.

Tuttavia, una vulnerabilità nell'implementazione dell'handshake a tre vie è che il *processo in ascolto deve ricordare il proprio numero di sequenza* dal momento in cui risponde con il proprio segmento SYN. Questo significa che un mittente malintenzionato può bloccare le risorse su un host spedendo un flusso di segmenti SYN e non completando mai la connessione. Questo attacco è chiamato SYN flood (inondazione di SYN).

Un modo per difendersi da questo attacco è utilizzare i **SYN cookie**. Invece di ricordare il numero di sequenza, un host *sceglie un numero di sequenza, generato facendo uso di un algoritmo tipico della crittografia*, lo mette nel segmento in uscita e se lo dimentica. Se l'handshake a tre vie arriva a termine, questo numero di sequenza (più 1) verrà restituito *all'host che potrà quindi rigenerarlo eseguendo la stessa funzione crittografica*, fintantoché siano noti gli input di quella funzione; per esempio gli indirizzi IP le porte dell'altro host e un segreto locale. Questa procedura permette all'host di verificare che un numero di sequenza per cui si riceve un acknowledgement è corretto senza doverselo ricordare separatamente.

RILASCIO DI UNA CONNESSIONE TCP

Quando dobbiamo disconnettere la connessione, il rilascio avviene considerando la connessione full duplex come coppia di connessioni simplex indipendenti proprio per evitare che uno dei due rimanga appeso:

- ✚ Quando una delle due parti non ha più nulla da trasmettere, *invia un segmento TCP con il bit FIN posto a 1* per confermare che non si ha più niente da trasmettere;
- ✚ Quando esso viene confermato, la connessione in uscita viene rilasciata ma l'altra rimane pendente;
- ✚ Quando anche l'altra parte completa lo stesso procedimento e rilascia la connessione nell'altra direzione, la connessione full-duplex termina.

È come se rilasciassimo quindi due connessioni (come nel caso della chiusura della chiamata con il cellulare). Per completare l'intero processo, sono necessari *4 segmenti TCP in totale con un FIN e un ACK per ciascuna direzione*. Per evitare il problema dei due eserciti si usano i **timer**, impostati al *doppio della vita massima di un pacchetto*. Il protocollo di gestione delle connessioni si rappresenta comunemente come una macchina a stati finiti.

CRITERIO DI TRASMISSIONE A FINESTRA SCORREVOLE IN TCP

Come detto in precedenza, la gestione della finestra in TCP disaccoppia il problema degli acknowledgement relativi alla corretta ricezione dei segmenti e l'allocazione dei buffer lato ricevente, la *finestra quindi viene continuamente adattata mediante un dialogo fra destinazione e sorgente*. Quando la destinazione invia un ack di conferma, dice anche quanti ulteriori byte possono essere spediti.

Abbiamo due host: *host 1 (mittente)* e *host 2 (ricevente)*, abbiamo un determinato buffer del ricevente (capacità della macchina ricevente di immagazzinare dati prima di verificarli e cederli al livello applicativo sovrastante).

In un determinato tempo **10** l'applicazione che sta sul mittente scarica sul livello di trasporto dei dati da inviare. Ne scarica **2KB**, poi parte un segmento di peso 2KB e con una **sequenza 0**. Questo segmento arriva su host 2 e viene preso e *caricato sul buffer disponibile* (di 2KB). *L'host 2 invia l'ack* dei 2KB che host 1 ha inviato, e comunica che ha una finestra disponibile di altri 2KB, e invia questa comunicazione alla macchina mittente. La macchina host 1 riceve poi dal livello applicativo altri 2KB da inviare, quindi li carica su una primitiva di SEND che spedisce all'host 2. La macchina ricevente carica gli altri 2KB (ma non ha ancora scaricato

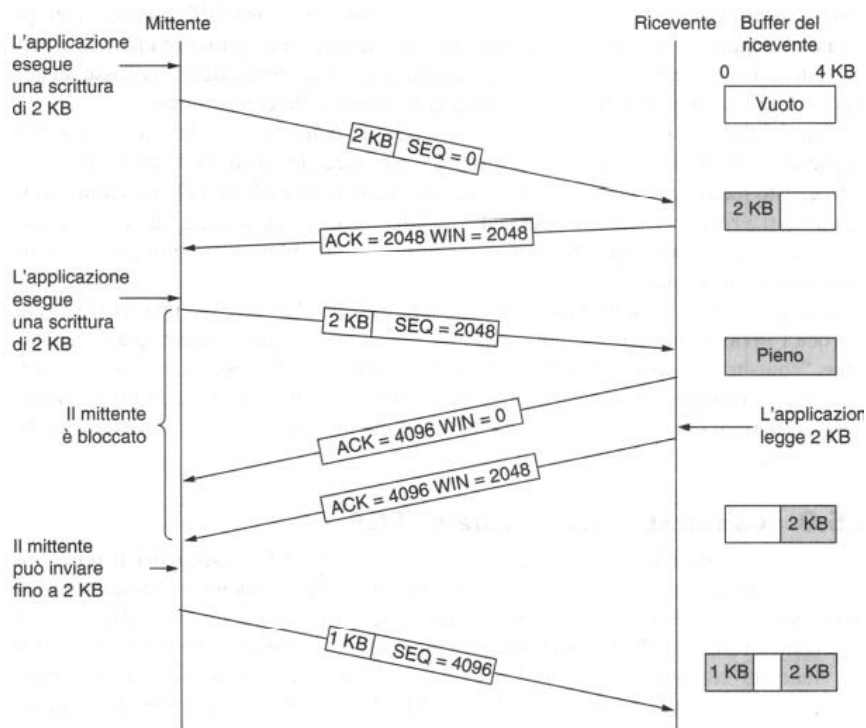


Figura 6.40 Gestione della finestra in TCP.

neanche i 2KB precedenti), ora il buffer è pieno. Quindi la *velocità di trasmissione è superiore a quella di elaborazione della macchina ricevente*.

A quel punto l'host 2 *rinvia all'host 1 l'ack ma avvisa che ha il buffer pieno e che la sua finestra di ricezione è 0*. La macchina host 1 quindi si blocca.

Dopo un po' l'applicazione nell'host 2 legge i primi 2KB, quindi host 2 avvisa host 1 che potrà inviare altri 2KB. Nella figura per ipotesi l'host 1 invia 1 KB e viene caricato nell'host 2. La gestione del flusso è quindi importante, bisogna dare attenzione al mittente veloce e al destinatario lento.

Ci sono delle eccezioni in questo modello: *è possibile inviare dati urgenti per consentire ad esempio di interrompere il processo su macchina remota*, oppure il mittente può inviare un segmento di 1 byte e per fare in modo che il *ricevente annunci di nuovo il successivo byte atteso e la dimensione della sua finestra*, questo pacchetto si chiama pacchetto di **window probe** (sonda). Lo standard TCP fornisce questa possibilità in modo che si prevenga un blocco nel caso dovesse essere perso un aggiornamento nella dimensione della finestra.

Il livello di trasporto deve avere dei modi pratici per gestire queste finestre scorrevoli. Ne vediamo 4:

- 1) **Delayed acknowledgement** (acknowledgement ritardati): posso fare in modo che se sto arrivando a una congestione, posso ritardare fino a **500ms** (tempo enorme per una rete) l'invio degli ack. Lo faccio perché se io non invio l'ack il mittente non può inviare o comunque non ho dati nuovi sulla finestra e questo consente di scaricare le finestre di buffer che si stanno saturando e regolare la trasmissione con un ritardo studiato.
- 2) **Algoritmo di Nagle**: posso fare in modo che quando dei dati arrivano in gruppo, è sufficiente inviare il primo gruppo e inserire il resto nel buffer finché non arriva l'ack. A questo punto possiamo poi *inviare le informazioni nel buffer usando un singolo segmento TCP* per poi ricominciare ad inserire i dati nel buffer fino all'arrivo del successivo ack. Il risultato, è che solo un pacchetto di piccole dimensioni può essere in circolazione in ogni dato momento. Se molti dati vengono spediti dall'applicazione, l'algoritmo di Nagle li metterà tutti in un segmento riducendo la banda utilizzata.
- 3) **Silly window syndrome**: Questo problema si verifica quando i dati vengono passati all'entità TCP di invio in *blocchi grandi*, ma un'applicazione interattiva dal lato ricevente legge i dati **1 byte alla volta**. Inizialmente il buffer TCP lato ricevente è pieno (ha una finestra con dimensione 0) e il mittente ne è a conoscenza.
L'applicazione interattiva legge quindi un carattere dal flusso TCP. Questa azione fa felice il TCP di ricezione, che invia un aggiornamento della finestra al mittente comunicandogli che può inviare 1 byte. Il mittente obbedisce e invia 1 byte. Ora il buffer è pieno, pertanto il ricevente manda un acknowledgement per il segmento di 1 byte, ma imposta la finestra a 0. Questo comportamento può andare avanti per sempre.

La **soluzione di Clark** sta *nell'impedire che il ricevente invii un aggiornamento della finestra per 1 byte*; invece è obbligato ad attendere la disponibilità di una certa quantità di spazio. Nello *specifico il ricevente può inviare un aggiornamento della finestra solo quando è in grado di gestire la dimensione massima del segmento* (concordata quando la connessione è stata instaurata) o quando il suo buffer è vuoto per metà. Inoltre, il mittente può essere di aiuto se non invia segmenti di piccole dimensioni; dovrebbe invece provare ad attendere fino a quando ha accumulato un segmento completo o almeno un segmento contenente dati per la metà della dimensione del buffer del ricevente.

L'**algoritmo di Nagle** e la **soluzione di Clark** al problema della silly window syndrome *sono complementari*. Nagle stava provando a risolvere il problema provocato dal fatto che l'applicazione di invio consegna i dati a TCP un byte alla volta. Clark stava provando a risolvere il problema di un'applicazione ricevente che legge i dati da TCP un byte alla volta.

Entrambe le soluzioni sono valide e possono cooperare. Lo scopo è che *il mittente non invii segmenti piccoli e che il ricevente non li richieda*. Il TCP di ricezione può andare oltre nel miglioramento delle prestazioni. Proprio come il TCP di invio, può fare uso di un buffer per tenere bloccata una richiesta READ dell'applicazione finché non possa fornirle un blocco di dati abbastanza grande. In questo modo si riducono

il numero di chiamate a TCP e il corrispondente overhead. Naturalmente aumenta anche il tempo di risposta, ma per applicazioni non interattive come il trasferimento di file l'efficienza potrebbe essere più importante del tempo di risposta alle singole richieste.

- 4) **Cumulative acknowledgement:** Un altro problema che il destinatario deve gestire è che *i segmenti possono arrivare in disordine*. Il destinatario deve memorizzare i dati in un buffer fino al momento in cui possono essere passati all'applicazione nel giusto ordine. In realtà se fossero scartati segmenti in disordine non succedrebbe nulla, dal momento che alla fine sarebbero ritrasmessi dal mittente. Ci sarebbe però uno spreco di risorse.

Gli acknowledgement possono essere inviati solo quando sono stati ricevuti tutti i dati fino all'ultimo byte; questo modello prende il nome di cumulative acknowledgement (acknowledgement cumulativo o ack cumulativo). Se il destinatario riceve i segmenti 0,1,2,4,5,6 e 7, può mandare acknowledgement fino all'ultimo byte del segmento 2 incluso. Al timeout del mittente, la trasmissione riprende dal segmento 3. Se il ricevente ha inserito nel buffer i segmenti da 4 a 7, dopo la ricezione del segmento 3 può mandare acknowledgement per tutti i byte fino alla fine del segmento 7.

GESTIONE DEI TIMER DI TCP

TCP utilizza più **timer** per svolgere il proprio lavoro. Il più importante è quello che *fa scattare la ritrasmissione*, e lo chiamiamo timer **RTO** (Retransmission Timeout).

Quando viene inviato un segmento, si avvia un timer di ritrasmissione. Se il segmento riceve un acknowledgement prima della scadenza del timer, TCP lo ferma; se invece il timer scade prima dell'arrivo dell'acknowledgement, *il segmento viene ritrasmesso* (e il timer riavviato). La questione che si pone è:

QUANTO DOVREBBE ESSERE LUNGO L'INTERVALLO DI TIMEOUT?

La soluzione è utilizzare un *algoritmo dinamico* che adatta costantemente l'intervallo di timeout in base a continue misurazioni delle prestazioni di rete. Per ogni connessione TCP, mantiene una variabile **SRTT** che rappresenta la stima corrente del **round trip time**.

Vediamo i 4 timer principali:

🚦 Timer di ritrasmissione RTT:

$$SRTT = \alpha SRTT + (1 - \alpha) R$$

Round Trip Time o Smoothed Round Trip Time è pari al *tempo totale di attraversamento di una rete*. L'algoritmo in questione presenta la scelta migliore per quanto concerne il round-trip di destinazione. Se uso questo tipo di timer, so che se per esempio il tempo di attraversamento totale di una rete corrisponde ad un certo **Tau**, quando ho *2 volte Tau* vuol dire che *l'ack deve essere morto* perché non fa in tempo a tornare indietro.

A seconda del funzionamento della rete però, devo essere in grado di parametrizzarlo e per farlo utilizzo un **valore R** che tiene conto delle condizioni differenti. È come se verificassi il trend di questo timer. R è il tempo di attraversamento, mentre α è un fattore di perequazione che determina quanto velocemente i vecchi valori vengono dimenticati, è pari a 7/8 per poter implementare la moltiplicazione per α e $\alpha-1$ come uno scorrimento a bit per rendere l'operazione più leggera alle CPU;

🚦 Timer di persistenza:

Il timer di persistenza ci dice *quante volte devo reiterare una determinata azione*. È progettato per impedire la seguente situazione di stallo: il ricevente invia un acknowledgement con una dimensione della finestra pari a 0, chiedendo al mittente di aspettare. In seguito, il mittente aggiorna la finestra, ma il pacchetto con l'aggiornamento viene perso.

Ora sia il mittente sia il ricevente attendono la prossima mossa dell'altro. Quando il timer di persistenza scade, il mittente trasmette una sonda (window probe) al ricevente. La risposta alla sonda fornisce la dimensione della finestra; se la dimensione è ancora zero, il timer di persistenza viene reimpostato e il ciclo si ripete; se non è zero è ora possibile inviare i dati;

🚦 Timer keep alive:

Quando una connessione è stata inattiva per lungo tempo, il timer di keep alive scade in modo che una delle parti controlli se l'altra sia ancora presente. Se non ottiene risposta la

connessione viene terminata. Questa funzionalità è controversa, perché aggiunge carico di lavoro e può terminare una connessione attiva a causa di una partizione temporanea della rete;

- ✚ **Timed wait:** può essere stimato su singole comunicazioni ed è l'ultimo timer utilizzato da ogni connessione TCP nello stato **TIMED WAIT** appunto. Il suo conteggio prosegue per un tempo pari al doppio del tempo di vita massimo del pacchetto, per garantire che alla chiusura di una connessione tutti i pacchetti creati siano stati rimossi.

CONTROLLO DELLA CONGESTIONE TCP

Quando il carico applicato a una rete è superiore a quello che può gestire, si verifica una congestione. Anche se lo strato network tenta di gestire le congestioni, la maggior parte del lavoro è svolta da TCP, perché la vera soluzione alle congestioni è la diminuzione della velocità dei dati.

In teoria, la congestione può essere affrontata prendendo in prestito un principio dalla fisica: la legge della conservazione dei pacchetti. L'idea è d'impedire l'inserimento di un nuovo pacchetto nella rete fino a quando un pacchetto vecchio non la lascia (vale a dire che viene consegnato). TCP tenta di raggiungere l'obiettivo manipolando dinamicamente la dimensione della finestra.

Il primo passo per la gestione delle congestioni è il **rilevamento**. Un tempo era difficile rilevare le congestioni. Un timeout provocato da un pacchetto perso avrebbe potuto essere causato da un disturbo su una linea di trasmissione o un pacchetto scartato da un router congestionato. Scoprire la differenza era difficile.

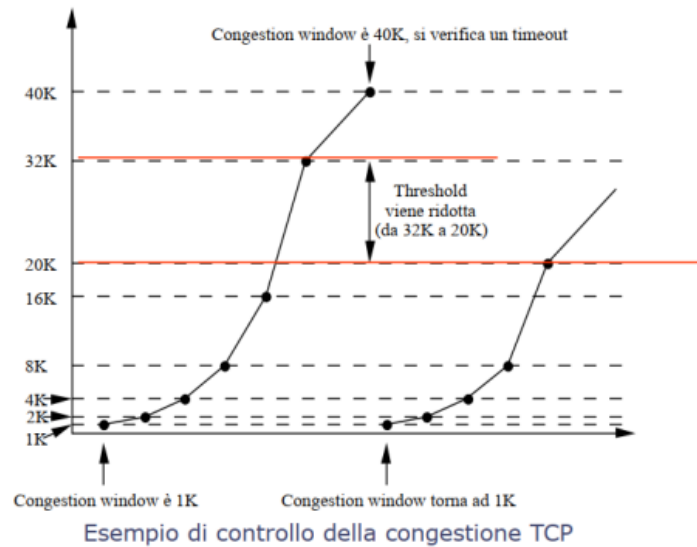
Oggi la perdita di pacchetti dovuta a errori di trasmissione è relativamente rara, perché le linee più lunghe sono realizzate in fibra ottica, di conseguenza, la maggior parte dei timeout di trasmissione su Internet è dovuta alla congestione. Tutti gli algoritmi TCP Internet presumono che i timeout siano provocati dalla congestione, e controllano i timeout per riscontrare i problemi.

Quando viene stabilita una connessione si deve scegliere una dimensione adatta per la finestra. Il ricevente può specificare una finestra in base alla dimensione del suo buffer; se il mittente si attiene a questa dimensione, non si verificheranno problemi dovuti al traboccamento del buffer all'estremità ricevente, ma potrebbero ancora verificarsi a causa della congestione interna alla rete.

La soluzione Internet consiste nel comprendere l'esistenza di due distinti problemi potenziali (*capacità della rete e capacità del ricevente*) e nel gestirli separatamente. A questo scopo ogni mittente mantiene due finestre: quella che il ricevente ha garantito e una seconda, la **finestra di congestione**. Ognuna riflette il numero di byte che il mittente può trasmettere. Il numero di byte che possono essere inviati corrisponde alla più piccola delle due finestre.

Se il ricevente afferma "Invia 8 KB" ma il mittente sa che un pacchetto con dimensione superiore a 4 KB congestionerebbe la rete, viene inviato un pacchetto di 4 KB. D'altra parte, se il ricevente afferma "Invia 8 KB" e il mittente sa che l'invio di un pacchetto di 32 KB non provocherebbe problemi, vengono inviati comunque gli 8 KB richiesti.

Per poter consentire di far funzionare questo modello, conviene andare per gradi. Vediamo la figura:



Innanzitutto, prendo e invio con una prima finestra di congestione di 1KB, arriva alla finestra di bufferizzazione dall'altra parte che prende questo dato e lo analizza velocemente passandolo a livello applicativo e ottengo immediatamente un ack. Con 1 Kb va benissimo. Dato che con 1 Kb è andata bene proviamo ad inviarne 2. Se inviamo 2Kb, questi arrivano sulla macchina di destinazione con lo stesso processo di prima. Usiamo le potenze di 2 e forziamo ulteriormente il procedimento provando ad inviare 4Kb e così via. Andrà tutto a buon fine aumentando sempre di più l'invio dei dati seguendo le potenze di 2, ma arrivati a *40 Kb collassa*. Si verifica un timeout, non arriva l'ack e siamo costretti a riavviare il processo ripartendo da 1.

Non useremo più la stessa tecnica, ma *useremo l'ultima potenza di 2 corretta* (cioè per la quale l'operazione è andata a buon fine), e la diluiamo di una certa quantità e il nuovo valore limite sarà quindi nella finestra rossa in figura. *Cerchiamo di posizionare la finestra in funzione della capacità che questa linea ha di poter ricevere quei dati in maniera normale* senza dover rischiare un rinvio.

Questo algoritmo è chiamato **avvio lento**, ma non è per nulla lento è esponenziale. Tutte le implementazioni TCP devono supportarlo.